

Handboek Informaticawetenschappen

Stijn De Vusser

Inhoudsopgave

Inleiding	7
Hardware	7
Software	8
Programmeren in tijden van AI	9
Over deze cursustekst	10
Praktisch	10
 1 Wiskundige bewerkingen	 13
1.1 Instap	13
1.2 Theorie	13
1.3 Verwerking	16
1.4 Opdrachten	17
 2 Variabelen en commentaar	 21
2.1 Instap	21
2.2 Theorie	22
2.3 Verwerking	24
2.4 Opdrachten	25

3	Datatypes en invoer van het toetsenbord	31
3.1	Instap	31
3.2	Theorie	31
3.3	Verwerking	35
3.4	Opdrachten	37
4	Functies	41
4.1	Instap	41
4.2	Theorie	42
4.3	Overzicht	46
4.4	Verwerking	47
4.5	Opdrachten	49
5	Voorwaardelijke uitvoering	51
5.1	Instap	51
5.2	Theorie	52
5.3	Overzicht	58
5.4	Verwerking	60
5.5	Opdrachten	63
6	Begrensde herhaling	69
6.1	Instap	69
6.2	Theorie	70
6.3	Overzicht	73
6.4	Verwerking	75
6.5	Opdrachten	76
7	Voorwaardelijke herhaling	79
7.1	Instap	79
7.2	Theorie	79
7.3	Overzicht	81
7.4	Verwerking	83
7.5	Opdrachten	85

8	Tuples	89
8.1	Instap	90
8.2	Theorie	90
9	Verzamelingen	95
9.1	Instap	95
9.2	Theorie	96
10	Lijsten	101
10.1	Instap	101
10.2	Theorie	102
10.3	Overzicht	111
10.4	Verwerking	115
10.5	Opdrachten	116
11	Geneste lijsten	123
11.1	Instap	123
11.2	Theorie	124
11.3	Verwerking	129
11.4	Opdrachten	130
12	Bestanden	133
12.1	Inleiding	133
12.2	Gegevens inlezen vanuit een tekstbestand	136
12.3	Gegevens inlezen vanuit een csv-bestand	138
12.4	Gegevens wegschrijven naar een tekstbestand	141
12.5	Overzicht	144
13	Recurisie	147
13.1	Instap	147
13.2	Theorie	149
13.3	Opdrachten	151

14 Numerieke wiskunde	155
14.1 Inleiding	155
14.2 Wat is numerieke wiskunde?	156
14.3 Numerieke wiskunde in deze cursus	156
14.4 Opdrachten	157
15 Sorteeralgoritmen	161
15.1 Opdrachten	161

Je bent geboren in een tijdperk waarin computers overal zijn. Dat is best wel een recente ontwikkeling, en het loont de moeite om even stil te staan bij de spectaculaire evolutie die tot ons computertijdperk heeft geleid. In deze inleiding duiken we in zowel de hardware (de toestellen zelf) als de software (de programma's die op die toestellen draaien). We staan daarnaast ook even stil bij de razendsnelle evolutie van artificiële intelligentie, en meer specifiek bij de rol die de mens moet blijven spelen.

Hardware

Charles Babbage (1791 – 1871) was een Brits wiskundige en ingenieur. Halverwege de 19de eeuw bedacht hij de zogenaamde **Analytical Engine**. Jammer genoeg is het bij een ontwerp gebleven: hij heeft deze door stoom aangedreven, volledig mechanische machine nooit kunnen bouwen. Toch was er niets mis met het ontwerp zelf, weten we nu. Daarom wordt Babbage algemeen beschouwd als de vader van de moderne computer.

Een kleine eeuw later gaf de Tweede Wereldoorlog een flinke duw in de rug aan de ontwikkeling van elektronische computers. In Duitsland werd vanaf 1941 de **Z3** gebruikt. Groot-Brittannië kon vanaf 1943 versleutelde berichten kraken dankzij de **Colossus**. In de Verenigde Staten zag **ENIAC** het levenslicht in 1945: een reusachtige machine ter grootte van een grote kamer, 30 ton zwaar, met een elektrisch vermogen van 150 kW.

In de jaren 1960 zorgde het Apollo-ruimtevaartprogramma van de Verenigde Staten voor een nieuwe uitdaging: computers moesten kleiner en lichter worden. Duizenden knappe koppen ontwikkelden daarom de **Apollo Guidance Computer** (AGC). Deze computer, die in 1969 mee instond voor de navigatie van Apollo 11 naar het maanoppervlak, had het formaat van ongeveer drie schoendozen. Hij woog nog 32 kg en verbruikte 55 W. De AGC had een werkgeheugen van 4 kB, een harde schijf van 72 kB en een kloksnelheid van 1

MHz. Het is niet overdreven om te stellen dat je grafische rekenmachine meer rekenkracht heeft dan de computer die de mens naar de maan bracht.

In de jaren 1970 werden computers steeds kleiner, zuiniger én krachtiger. Toch bleven ze voorlopig nog het speeltje van grote bedrijven, overheden en (militaire) onderzoeksinstellingen.

Vanaf de jaren 1980 kwam de **Personal Computer** (PC) op de markt, waardoor ook gewone gezinnen stilaan een computer in huis kregen. Vanaf het midden van de jaren 1990 werden al die computers met elkaar verbonden via het internet.

Kijk nu even om je heen: je ziet overal computers. Misschien denk je meteen aan een laptop of desktop, maar het gaat veel verder dan dat. De smartphone in je zak is onvoorstelbaar veel krachtiger dan de computer waarmee we in 1969 naar de maan reisden. Misschien heb je een smartwatch om je pols waarop je je mails en berichten leest. Je televisie is eigenlijk ook een computer geworden. En je spelconsole al helemaal: die kan dan wel niet zo goed tekstverwerken, maar blinkt weer uit op grafisch vlak.

Software

Software, de programma's die door de computer worden uitgevoerd, is minstens even belangrijk als hardware. Opvallend genoeg maakte men pas vanaf de jaren 1960 echt een onderscheid tussen hardware en software. Daarvoor werd een programma gewoon gezien als een onlosmakelijk deel van de machine.

Ada Lovelace (1815 – 1852) was een tijdgenote van Charles Babbage en, net als hij, een Brits wiskundige. Zij bedacht programma's die door zijn Analytical Engine uitgevoerd zouden kunnen worden. Daarom wordt Lovelace algemeen beschouwd als de allereerste computerprogrammeur.

Computerprogramma's voeren een **algoritme** uit: een stappenplan dat in een eindig aantal stappen altijd tot het gewenste resultaat leidt. Het woord klinkt misschien nieuw, maar het concept ken je al langer dan vandaag.

- In de lagere school leerde je cijferen. De juf of meester leerde je een vast stappenplan om een natuurlijk getal (het deeltal) te delen door een ander natuurlijk getal (de deler). Volg je dat stappenplan correct, dan kom je gegarandeerd bij de juiste oplossing uit. De staartdeling van natuurlijke getallen is dus eigenlijk gewoon een algoritme.
- In de eerste graad secundair leerde je een getal ontbinden in priemfactoren en het kleinste gemene veelvoud van twee getallen bepalen. Ook dat zijn twee mooie voorbeelden van algoritmes.
- Een recept in een kookboek is ook een stappenplan dat in een eindig aantal stappen tot een bepaald resultaat leidt: een algoritme dus. Zelfs als je nog nooit een pannenkoek hebt gezien, kan je er toch een perfecte bakken, zolang het recept maar stap voor stap duidelijk en ondubbelzinnig zegt wat je moet doen.

Algoritmes zitten tegenwoordig werkelijk overal verstopt:

- Luister je via een streamingdienst naar muziek? Dan leert het onderliggende algoritme je muzikale smaak steeds beter kennen, en op basis daarvan stelt het nieuwe nummers voor waarvan het denkt dat je ze wel zult smaken.
- Sociale media werken op precies dezelfde manier. Elke klik en elke *like* leert het bedrijf je weer wat beter kennen, zodat het je steeds gericht kan bestoken met informatie (en advertenties) waarvan ze denken dat die jou wel zullen aanspreken.
- Heb je je ooit gevraagd hoe een routeplanner werkt? Hoe vind je gegarandeerd de snelste weg van A naar B? Het klinkt ingewikkeld, maar in essentie pas je gewoon een relatief eenvoudig algoritme toe dat de Nederlandse wiskundige en informaticus Edsger Dijkstra (1930 – 2002) al in 1959 bedacht.
- Ook in sectoren waar je het misschien niet meteen verwacht, worden algoritmes almaar belangrijker. Artsen worden bijgestaan door software bij het stellen van een diagnose. Wil je een lening aangaan bij de bank, dan is het een algoritme dat je kredietwaardigheid inschat. Zelfs taalkundigen, archeologen en bijbelwetenschappers gebruiken tegenwoordig algoritmes in hun onderzoek.

Programmeren in tijden van AI

Je hebt het ongetwijfeld al gemerkt: artificiële intelligentie (AI) is niet meer weg te denken uit onze wereld. Chatbots kunnen op vraag een volledig computerprogramma schrijven, en dat wordt met de dag beter. Misschien vraag je je dan af: waarom zou ik nog leren programmeren, als AI dat straks toch overneemt?

Dat is een terechte vraag, maar het antwoord is simpel. Je moet leren programmeren, precies omdat AI het anders volledig zou overnemen. Een AI-model kan inderdaad code genereren, maar het begrijpt niet altijd wat er precies gevraagd wordt, en het maakt (net als mensen) fouten. Iemand moet die code kunnen lezen, begrijpen, analyseren, testen en bijsturen. Zonder basiskennis van programmeren en computationeel denken sta je compleet machteloos wanneer een AI-tool niet doet wat het zou moeten doen, of erger, wanneer het iets doet wat het helemaal niet zou mogen doen.

In de wereld van cybersecurity wordt het onderscheid gemaakt tussen een **red team** en een **blue team**. Het red team probeert een systeem aan te vallen en zwaktes bloot te leggen, terwijl het blue team dat systeem net moet verdedigen en versterken. Doordat beide teams elkaar tot het uiterste drijven, blijven digitale systemen in het algemeen vrij veilig.

Heel wat experts stellen dat programmeurs en AI in de toekomst een gelijkaardige dubbelrol gaan spelen. Een AI-tool speelt de rol van het red team en genereert code, terwijl de programmeur (in de rol van het blue team) die code kritisch beoordeelt, test, begrijpt waarom ze werkt (of net niet), en verantwoordelijkheid opneemt voor het eindresultaat.

Wie zelf geen idee heeft hoe programmeren in elkaar zit, kan onmogelijk die verdedigende rol op zich nemen, en kan geen verantwoordelijkheid opnemen voor de goede werking van de software.

Met andere woorden: net omdat AI zo goed wordt in programmeren, is het extra belangrijk dat ook jij begrijpt hoe programmeren werkt. Alleen dan kan je AI op een slimme, kritische en veilige manier inzetten, in plaats van er blindelings op te vertrouwen.

Over deze cursustekst

Er bestaan honderden programmeertalen, en er komen er voortdurend nieuwe bij. Sommige zijn inzetbaar voor een breed scala aan toepassingen, andere zijn erg krachtig maar beperkt in hun toepassingsgebied. Sommige talen hebben een eenvoudige syntax (vergelijk het met de “grammatica” van een taal) en zijn daardoor uitstekend geschikt om mee te starten. Voor andere moet je je dan weer eerst inwerken in een soms behoorlijk complexe syntax.

In deze lessenreeks werk je met Python 3. Daar zijn enkele goede redenen voor:

- Python heeft een relatief eenvoudige syntax en is daardoor makkelijk aan te leren. Een uitstekende keuze dus om de basisprincipes van computationeel denken onder de knie te krijgen.
- Ondanks die eenvoudige syntax is Python bijzonder krachtig. Door **libraries** of **modules** te importeren, kan je Python inzetten voor de meest uiteenlopende toepassingen. Dankzij die veelzijdigheid is Python wellicht een van de meest gebruikte programmeertalen ter wereld.
- Python is volledig **open source**. Dat betekent dat iedereen Python helemaal gratis mag gebruiken. De meest recente versie vind je steeds op <https://www.python.org/downloads/>.

Let er wel op dat je steeds in Python 3 werkt. Python 2.7 is ook nog beschikbaar en wordt hier en daar nog gebruikt, maar de functie **print()**, die je voortdurend nodig zult hebben, heeft een andere syntax in beide versies. Een correct programma in Python 3 zal daardoor niet altijd werken in Python 2.

Praktisch

Elk deel in deze tekst volgt dezelfde opbouw. Je start telkens met een instapoefening (meestal een uitgewerkt voorbeeld), gevolgd door een stukje theorie. Daarna volgen enkele verwerkingsopdrachten waarbij je met kleine stapjes naar een werkend programma toewerkt. Elk deel sluit af met een reeks opgaves die je volledig zelfstandig oplost.

Je leert niet fietsen door een boek over fietsen te lezen. Programmeren leer je op dezelfde manier niet uit een boek. Je moet zelf achter de computer kruipen en aan de slag gaan. Bij deze cursustekst hoort een Dodona (<https://dodona.ugent.be>), een platform dat ontwikkeld werd door de UGent. Je leerkracht bezorgt je de precieze url en legt uit hoe je je voor deze cursus registreert.

Om een programma vanaf nul te schrijven, heb je in het algemeen een IDE (*Integrated Development Environment*) nodig. Je gebruikt daarvoor de *Sandbox* in Dodona. In deze omgeving schrijf je je programma, kan je het zelf uitvoeren en testen. Om fouten op te sporen kan je in de *Sandbox* ook *debuggen*: het programma wordt stap voor stap uitgevoerd, zodat je goed kan opvolgen wat er precies gebeurt.

Als je denkt dat je programma helemaal aan de opgave voldoet, dan kan je het indienen. Dodona voert je code meermaals uit, met verschillende testgevallen, en vergelijkt jouw uitvoer met de verwachte uitvoer.

Let op: als je de *Sandbox* afsluit, wordt je code niet bewaard. Ook al ben je nog niet klaar en voldoet je code nog niet aan de opgave, toch doe je er goed aan om regelmatig code in te dienen. Je code blijft bewaard; je kunt op gelijk welk moment terug naar alle code die je ooit ingediend hebt.

1.1 Instap

- 1** Een voetbalclub wil 30 nieuwe ballen kopen. Een bal kost normaal gezien 15 euro, maar de verkoper geeft een korting van 20%. Daarnaast is er een vaste administratiekost van 5 euro per bestelling. De verzendingskosten bedragen 0,5 euro per bal voor de eerste 10 ballen. Alle andere ballen worden gratis verzonden.

Welk budget moet de club vrijmaken voor deze bestelling?

Typ volgende lijn over in je IDE:

```
1 30*15 - 20/100*(30*15) + 5 + 10*0.5
```

Let op: decimale getallen noteer je in Python met een punt, niet met een komma. Bewaar je programma en voer het uit.

Noteer hier de uitvoer die de computer geeft:

1.2 Theorie

De functie `print()`

De functie `print()` wordt in Python gebruikt om resultaten te tonen.

```
1 print(30*15 - 20/100*(30*15) + 5 + 10*0.5)
```

1. WISKUNDIGE BEWERKINGEN

De computer geeft nu het juiste antwoord:

```
370.0
```

Je kunt verschillende resultaten op een zelfde lijn tonen door ze met een komma van elkaar te scheiden. Python zorgt dan automatisch voor een spatie tussen de verschillende resultaten.

Voorbeeld 1:

```
1 print(30*15 - 20/100*(30*15) + 5 + 10*0.5)
2 print(3+5, 2-8, 3*(1+4))
```

```
370.0
8 -6 15
```

Python begint bovenaan je programma en voert de code lijn per lijn uit. Elke lijn wordt precies één keer uitgevoerd. Het programma wordt beëindigd na de laatste lijn code.

Wiskundige operatoren

De tabel geeft aan hoe je in Python de hoofdbewerkingen kan gebruiken. Bij het berekenen van een uitdrukking houdt Python rekening met de volgorde van bewerkingen.

bewerking	operator	voorbeeld	resultaat
som	+	3+4	7
verschil	-	2-5	-3
product	*	7*6.5	45.5
quotiënt	/	13/4	3.25
machtsverheffing	**	(-3)**2	9

Een staartdeling is de deling van een natuurlijk deeltal D door een natuurlijke deler d . Het resultaat van de staartdeling is het quotiënt q en de rest r .

Python kent operatoren om het quotiënt en de rest van de staartdeling te berekenen:

bewerking	operator	voorbeeld	resultaat
natuurlijk quotiënt	//	43//8	5
natuurlijke rest	%	43%8	3

Wiskundige functies

In Python bestaat er eigenlijk geen commando om de vierkantswortel van een getal te berekenen. Het getal π is ook niet gedefinieerd. Om $\sqrt{\pi}$ te berekenen, importeer je de functie `sqrt()` (van **square root**, de Engelse vertaling van vierkantswortel) en de constante `pi` eerst uit de `math`-module.

Voorbeeld 2:

```
1 from math import sqrt, pi
2 print(sqrt(pi))
```

```
1.7724538509055159
```

In de tabel vind je nog een aantal functies die van pas zullen komen bij het schrijven van een programma.

bewerking	operator	voorbeeld	resultaat
vierkantswortel van <code>a</code>	<code>from math import sqrt</code> <code>sqrt(a)</code>	<code>sqrt(25)</code>	5.0
absolute waarde van <code>a</code>	<code>abs(a)</code>	<code>abs(-5)</code>	5
<code>a</code> afronden naar een natuurlijk getal	<code>round(a)</code>	<code>round(4/3)</code> <code>round(5/3)</code>	1 2
<code>a</code> afronden op <code>n</code> decimalen	<code>round(a, n)</code>	<code>round(5/3, 2)</code>	1.67
<code>a</code> afronden naar beneden	<code>from math import floor</code> <code>floor(a)</code>	<code>floor(3.99)</code>	3
<code>a</code> afronden naar boven	<code>from math import ceil</code> <code>ceil(a)</code>	<code>ceil(3.01)</code>	4
maximum van een rij getallen	<code>max(a, b, c, ...)</code>	<code>max(5, -3, 7)</code>	7
minimum van een rij getallen	<code>min(a, b, c, ...)</code>	<code>min(5, -3, 7)</code>	-3

1.3 Verwerking

2 De rechthoekszijden in een rechthoekige driehoek hebben een lengte van 5 cm en 7 cm.

- a. Bereken de lengte van de schuine zijde. Rond je resultaat af op 1 millimeter.
- b. Je krijgt een aantal Python-uitdrukkingen om deze opgave op te lossen. Kies de juiste uitdrukkingen. Rangschik ze in de juiste volgorde.

(a) `sqrt(5**2 + 7**2)`

(b) `from math import sqrt`

(c) `print(sqrt(5**2 + 7**2))`

(d) `round(print(sqrt(5**2 + 7**2), 1))`

(e) `print(round(sqrt(5**2 + 7**2), 1))`

- c. Je vindt deze uitdrukkingen in Dodona. Schrijf ze in de juiste volgorde. Test je programma in Dodona.

3 Een cirkel heeft een oppervlakte van 100 cm².

- a. Bereken de straal van deze cirkel in cm. Rond af op 1 millimeter.
- b. Schrijf een programma dat de straal van deze cirkel berekent in cm en dat de straal toont (afgerond op 1 millimeter). Test je programma in Dodona.

4 Voorspel de uitvoer van de volgende programma's. Je mag je rekenmachine gebruiken. Controleer je antwoord door de code te kopiëren naar je IDE en daar uit te voeren.

a. `from math import sqrt`
`print(round(sqrt(2), 3))`

b. `print(65 // 7)`

c. `print(65 % 7)`

1.4 Opdrachten

Reeks 1

- 5** De formule voor het verband tussen een temperatuur C in graden Celsius en een temperatuur F in graden Fahrenheit is:

$$F = \frac{9}{5} \cdot C + 32.$$

Schrijf een programma dat een temperatuur van -5°C omzet in $^{\circ}\text{F}$ en het resultaat toont. Test je programma in Dodona.

- 6** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Alexander is 190 cm groot en weegt 85 kg. Schrijf een programma dat zijn BMI berekent. Toon de berekende waarde, afgerond naar een natuurlijk getal. Test je programma in Dodona.

- 7** Een cilinder heeft een straal van 2 cm en een hoogte van 5 cm. Schrijf een programma dat het volume van de cilinder berekent. Toon het resultaat, uitgedrukt in cm^3 . Rond af op 1 mm^3 . Test je programma in Dodona.

- 8** Rayane heeft van zijn ouders 30 euro gekregen waarmee hij iets mag kopen om in de klas te trakteren voor zijn verjaardag. Een zak met mini-chocoladerepen kost 12,99 euro. Schrijf een programma dat berekent hoeveel zakken Rayane kan kopen en hoeveel geld hij overhoudt. Toon beide waarden op één lijn (eerst het aantal zakken, dan het bedrag). Test je programma in Dodona.

Reeks 2

- 9** Een ruit heeft een grote diagonaal van 8 cm en een kleine diagonaal van 6 cm.

Schrijf een programma dat de oppervlakte en de omtrek van deze ruit berekent. Toon de resultaten in deze volgorde, telkens op een aparte lijn. Test je programma in Dodona.

- 10** Annabel heeft 5700 m gelopen op een atletiekpiste. Ze heeft daarvoor precies 31 minuten nodig gehad. Schrijf een programma dat haar gemiddelde snelheid berekent. Druk deze uit in km/u en rond af op 0,1 km/u . Bereken ook het aantal hele rondjes dat ze gelopen heeft en toon dit op een tweede lijn. Test je programma in Dodona.

1. WISKUNDIGE BEWERKINGEN

- 11** De formule voor het verband tussen een temperatuur C in graden Celsius en een temperatuur F in graden Fahrenheit is:

$$F = \frac{9}{5} \cdot C + 32.$$

In 1913 werd een recordtemperatuur van 134°F waargenomen in Death Valley (VS).

Schrijf een programma dat deze temperatuur omzet naar $^{\circ}\text{C}$. Rond de temperatuur af op $0,1^{\circ}\text{C}$ en toon het resultaat. Test je programma in Dodona.

- 12** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Alix weegt 63 kg en heeft een BMI van 21. Schrijf een programma dat haar lengte berekent en toont. Druk de lengte uit in cm en rond af op 1 mm. Test je programma in Dodona.

- 13** Een cilinder heeft een hoogte van 10 cm en een volume van 300 cm^3 . Schrijf een programma dat de straal van de cilinder berekent. Toon het resultaat, uitgedrukt in cm. Rond af op 1 mm. Test je programma in Dodona.

- 14** Een rechte met voorschrift $y = ax + b$ gaat door de punten $P(1, 1)$ en $Q(-2, 7)$.

Schrijf een programma dat de waarde berekent van a en b en deze op één lijn toont (eerst a , dan b). Test je programma in Dodona.

- 15** Driehoek ABC is rechthoekig in B . De rechthoekszijden hebben een lengte $|AB| = 5 \text{ cm}$ en $|BC| = 7 \text{ cm}$.

Schrijf een programma dat $\sin \hat{A}$, $\cos \hat{A}$ en $\tan \hat{A}$ berekent en afrondt op 2 cijfers na de komma. Toon de resultaten in deze volgorde en telkens op een aparte lijn. Test je programma in Dodona.

Reeks 3

- 16** Boris wil zijn kamer opnieuw schilderen. Hij heeft berekend dat de totale oppervlakte van de muren in zijn kamer 48 m^2 is. Hij weet dat hij 2 lagen verf moet aanbrengen en dat hij 10 m^2 kan verven met 1 liter verf. Een pot verf van 1 liter kost 35 euro. Een pot verf van 4 liter kost 110 euro.

Boris wil zo weinig mogelijk geld uitgeven. Schrijf een programma dat berekent hoeveel potten verf van 4 liter en hoeveel van 1 liter hij moet kopen. Het programma toont deze hoeveelheden op één lijn (eerst het aantal potten van 4 liter, dan het aantal van 1 liter).

Op een volgende lijn toont het programma de totale prijs van de verf. Test je programma in Dodona.

- 17** Op kerstavond om 19u00 stelt Ranya een timer in die 143 uur later zal aflopen.

Schrijf een programma dat berekent op welke datum de timer zal aflopen en hoe laat het zal zijn wanneer de timer afloopt. Het programma toont deze twee natuurlijke getallen op één lijn (eerst de dag, dan het uur). Test je programma in Dodona.

- 18** Een analoge klok wijst precies 13u00 aan. Ewout draait de klok precies 1293 uur verder.

Schrijf een programma dat de hoek tussen de kleine en de grote wijzer berekent en toont. Test je programma in Dodona.

2.1 Instap

1 Een voetbalclub wil 30 nieuwe ballen kopen. Een bal kost normaal gezien 15 euro, maar de verkoper geeft een korting van 20%. Daarnaast is er een vaste administratiekost van 5 euro per bestelling. De verzendingskosten bedragen 0,5 euro per bal voor de eerste 10 ballen. Alle andere ballen worden gratis verzonden.

- a. Welk budget moet de club vrijmaken voor deze bestelling? Noteer het programma dat je in 1.1 gemaakt hebt om deze opgave op te lossen.
- b. Het bestuur van de club beslist om 40 ballen van 10 euro te bestellen. Voor de rest blijft de opgave ongewijzigd. Bereken welk budget er nu moet voorzien worden.

Pas de Python-code aan om dit nieuwe bedrag te berekenen.

Typ deze code in je IDE. Bewaar het programma en voer het uit. Test de code.

Noteer de uitvoer van het programma:



- c. Het bestuur van de club beslist om 70 ballen van 5 euro te bestellen. Voor de rest blijft de opgave ongewijzigd. Bereken welk budget er nu moet voorzien worden.

Pas de Python-code aan om dit nieuwe bedrag te berekenen.

Typ deze code in je IDE. Bewaar het programma en voer het uit. Test de code.

Noteer de uitvoer van het programma:



2.2 Theorie

Een programma aanpassen vraagt tijd. Als het programma door iemand anders geschreven is, moet je de code eerst interpreteren. Pas dan kan je de code aanpassen. Hoe langer het programma is, hoe moeilijker het is om de code te interpreteren.

Om een programma leesbaarder te maken, kan je **commentaar** toevoegen en **variabelen** gebruiken. Een variabele is een plaats in het geheugen van de computer waar informatie tijdelijk kan bewaard worden. Je kunt een variabele zien als de naam van een lade in een kast. Door een waarde toe te kennen aan een variabele, plaats je iets in de lade. Door de variabele later opnieuw op te roepen in je programma, lees je als het ware de waarde uit die in de lade verborgen zat.

Voorbeeld 1:

```
1 # dit programma berekent en toont het budget voor de bestelling van
   nieuwe ballen
2 aantal_ballen = 40
3 eenheidsprijs = 10
4 prijs_ballen = aantal_ballen * eenheidsprijs
5 kortingspercentage = 20
6 korting = kortingspercentage / 100 * prijs_ballen
7 administratiekost = 5
8 verzendingskosten = 0.5*min(10, aantal_ballen)
9 totaalprijs = prijs_ballen - korting + administratiekost +
   verzendingskosten
10 print(totaalprijs)
```



330.0

Commentaar

Lijn 1 begint met een **hekje** (**#**). Python houdt bij het uitvoeren van code geen rekening met alles wat er achter een hekje staat. Een deel van een programma dat achter een hekje staat, wordt daarom **commentaar** genoemd. Maak er een gewoonte van om je code netjes en duidelijk te houden door voldoende commentaar te schrijven.

Variabelen

Op lijnen 2-9 worden enkele variabelen gedefinieerd. De code past nu niet meer op één lijn, maar dankzij het gebruik van variabelen is de code wel duidelijker geworden. Ook een persoon die de code niet heeft geschreven, zou dit programma zonder veel moeite moeten kunnen begrijpen en aanpassen.

Hoe definieer je een variabele?

- Kies een zinvolle naam voor de variabele. Een variabele heeft een naam die bestaat uit een opeenvolging van letters, cijfers of het symbool `_`. Spaties zijn niet toegelaten. Het eerste karakter mag ook geen cijfer zijn. Python is hoofdlettergevoelig, dus de variabele `a` kan andere informatie bevatten dan de variabele `A`. Je mag geen bestaand Python-commando kiezen als naam van een variabele: zo zul je geen variabele `print` kunnen definiëren.
- De waarde van een variabele wordt toegekend met het gelijkheidsteken (`=`).
- Geef de waarde van een variabele.

Je kunt een variabele definiëren aan de hand van andere variabelen. Je kunt een variabele zelfs definiëren aan de hand van de variabele zelf.

Voorbeeld 2:

```
1 # dit programma toont verschillende mogelijkheden om de waarde van een
  variabele te definiëren
2 a = 2
3 b = 4
4 c = a + b          # de waarde van c is gelijk aan 2+4 = 6
5 a = a + 1          # de (nieuwe) waarde van a is gelijk aan 2+1 = 3
6 print(a, b, c)
```

```
3 4 6
```

Als je lijn 5 bekijkt vanuit een puur wiskundig standpunt, zie je een vergelijking die geen oplossing heeft. Geen enkel getal is gelijk aan zichzelf plus één.

Wanneer je er vanuit het standpunt van een computerwetenschapper naar kijkt, krijgt de uitdrukking op lijn 5 wel een zinvolle betekenis. Op die lijn wordt een waarde toegekend aan de variabele `a`. De (nieuwe) waarde die aan `a` toegekend wordt, is gelijk aan de (oude) waarde van `a`, plus één. Oorspronkelijk was de waarde van `a` gelijk aan 2. Na het uitvoeren van lijn 5 is de nieuwe waarde van `a` dus gelijk aan $2 + 1 = 3$.

2.3 Verwerking

- 2** Linde heeft 213 euro in haar spaarpot. Op 1 januari krijgt ze 100 euro van haar peter. Elke maand krijgt ze 20 euro zakgeld van haar ouders. Voor haar verjaardag krijgt ze 50 euro. Elke woensdag koopt ze een broodje van 3 euro. Om de twee weken koopt ze daar een drankje bij van 1,50 euro. Verder geeft ze geen geld uit. Hoeveel geld heeft Linde in haar spaarpot op 31 december? (Je mag aannemen dat er altijd 52 woensdagen in een jaar zijn.)

Schrijf op papier een programma waarmee je dit bedrag kunt berekenen en tonen. Schrijf minstens één lijn commentaar en gebruik zo veel mogelijk variabelen. Schrijf de lijn commentaar in het rood. Schrijf de naam van de variabelen in het blauw. Schrijf de naam van een functie in het groen. Schrijf al de rest in het zwart.

- 3** De rechthoekszijden in een rechthoekige driehoek hebben een lengte van 8 cm en 3 cm.

- Bereken de lengte van de schuine zijde. Rond je resultaat af op 0,1 mm.
- Je krijgt een aantal Python-uitdrukkingen om deze opgave op te lossen. Rangschik ze in een juiste volgorde.

(a) `c = sqrt(a**2 + b**2)`

(b) `from math import sqrt`

(c) `print(c)`

(d) `b = 3`

(e) `c = round(c, 2)`

(f) `a = 8`

- Je vindt deze uitdrukkingen in Dodona. Schrijf ze in de juiste volgorde. Test je programma in Dodona.

- 4** Een cirkel heeft een oppervlakte van 500 cm².

- Bereken de straal van deze cirkel in cm. Rond je resultaat af op 1 mm.
- Schrijf een programma dat de straal van deze cirkel berekent in cm. Toon het resultaat, afgerond op 1 mm. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je programma in Dodona.

2.4 Opdrachten

Reeks 1

- 5 De formule voor het verband tussen een temperatuur C in graden Celsius en een temperatuur F in graden Fahrenheit is:

$$F = \frac{9}{5} \cdot C + 32.$$

Schrijf een programma dat een temperatuur van 18°C omzet in °F en het resultaat toont. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je programma in Dodona.

- 6 De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Jacoba is 176 cm groot en weegt 68 kg. Schrijf een programma dat haar BMI berekent. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Toon de berekende waarde, afgerond op 1 cijfer na de komma. Test je programma in Dodona.

- 7 Een kegel heeft een straal van 3 cm en een hoogte van 8 cm. Schrijf een programma dat het volume van de kegel berekent. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Toon het resultaat, uitgedrukt in cm³. Rond af op 1 mm³. Test je code in Dodona.

- 8 Janne heeft van haar ouders 12 euro gekregen waarmee ze iets mag kopen om in de klas te trakteren voor haar verjaardag. Een pak met 6 appels kost 2,25 euro. Schrijf een programma dat berekent hoeveel pakken appels Janne kan kopen en hoeveel geld ze overhoudt. Toon beide waarden op één lijn (eerst het aantal pakken appels, dan het bedrag). Schrijf minstens één lijn commentaar en gebruik minstens vier variabelen. Test je programma in Dodona.

Reeks 2

- 9 Een ruit heeft een grote diagonaal van 12 cm en een kleine diagonaal van 7 cm.

Schrijf een programma dat de oppervlakte en de omtrek van deze ruit berekent. Rond de omtrek af op 1 mm. Toon de resultaten in deze volgorde, telkens op een aparte lijn. Schrijf minstens één lijn commentaar en gebruik minstens vier variabelen. Test je programma in Dodona.

2. VARIABELEN EN COMMENTAAR

- 10** Nadia wil haar kamer opnieuw schilderen. Ze heeft berekend dat de totale oppervlakte van de muren in haar kamer 52 m^2 is. Ze weet dat ze 2 lagen verf moet aanbrengen en dat ze 12 m^2 kan verven met 1 liter verf. Een pot verf van 1 liter kost 35 euro. Een pot verf van 4 liter kost 110 euro.

Nadia wil zo weinig mogelijk geld uitgeven. Schrijf een programma dat berekent hoeveel potten verf van 4 liter en hoeveel van 1 liter ze moet kopen. Het programma toont deze hoeveelheden op één lijn (eerst het aantal potten van 4 liter, dan het aantal van 1 liter). Op een volgende lijn toont het programma de totale prijs van de verf. Schrijf minstens één lijn commentaar en gebruik minstens zes variabelen. Test je programma in Dodona.

- 11** In 1913 werd een recordtemperatuur van 134°F waargenomen in Death Valley (VS).

Schrijf een programma dat deze temperatuur omzet naar $^\circ\text{C}$. Rond de temperatuur af naar een natuurlijk getal en toon het resultaat. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je programma in Dodona.

- 12** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Louise weegt 55 kg en heeft een BMI van 22. Bereken en toon haar lengte, uitgedrukt in cm, afgerond naar een natuurlijk getal. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Toon de berekende waarde, afgerond naar een natuurlijk getal. Test je programma in Dodona.

- 13** Een kegel heeft een straal van 8 cm en een volume van 400 cm^3 . Schrijf een programma dat de hoogte van de kegel berekent. Toon het resultaat, uitgedrukt in cm. Rond af op 1 mm. Test je programma in Dodona.

- 14** Een rechte met voorschrift $y = ax + b$ gaat door de punten $P(-1, 1)$ en $Q(7, 5)$.

Schrijf een programma dat de waarde berekent van a en b . Toon de resultaten op één lijn (eerst a , dan b). Schrijf minstens één lijn commentaar en gebruik minstens zes variabelen. Test je programma in Dodona.

- 15** Driehoek ABC is rechthoekig in B . De rechthoekszijden hebben een lengte $|AB| = 8$ en $|BC| = 4$.

Schrijf een programma dat $\sin \hat{A}$, $\cos \hat{A}$ en $\tan \hat{A}$ berekent en afrondt op 3 cijfers na de komma. Toon de resultaten in deze volgorde, telkens op een aparte lijn. Schrijf minstens één lijn commentaar en gebruik minstens zes variabelen. Test je programma in Dodona.

Reeks 3

- 16** Filip wil een bedrag van 298 euro betalen met zo weinig mogelijk biljetten en munten. Hij heeft voldoende biljetten van 50, 20, 10 en 5 euro en munten van 2 en 1 euro ter beschikking.

Schrijf een programma dat berekent hoeveel biljetten van 50, 20, 10 en 5 en hoeveel munten van 2 en 1 hij nodig heeft. Schrijf minstens één lijn commentaar en gebruik minstens zeven variabelen. Toon de aantallen in deze volgorde op één lijn. Test je programma in Dodona.

- 17** Op 1 december om 21u00 stelt Willemijn een timer in die 537 uur later zal aflopen.

Schrijf een programma dat berekent op welke datum de timer zal aflopen en hoe laat het zal zijn wanneer de timer afloopt. Het programma toont deze twee natuurlijke getallen (eerst datum, dan tijdstip). Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je programma in Dodona.

- 18** Een analoge klok wijst precies 13u00 aan. Korneel draait de klok precies 2467 uur verder.

Schrijf een programma dat de hoek berekent tussen de kleine en de grote wijzer en deze hoek ook toont. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je programma in Dodona.

- 19** In de Angelsaksische wereld worden maten en gewichten uitgedrukt in het Imperial Standard System. Waar wij de lengte van een persoon uitdrukken in (centi)meter, wordt in bijvoorbeeld Groot-Brittannië en de Verenigde Staten feet (voet) en inch (duim) gebruikt. 1 foot komt overeen met 30,48 cm en 1 inch komt overeen met 2,54 cm. In 1 foot passen dus precies 12 inches.

Lena is 1m68. Schrijf een programma dat haar lengte omzet in foot (een natuurlijk getal) en inches (afgerond op 0,1 inch). Toon deze getallen op één lijn (eerst foot, dan inches). Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je programma in Dodona.

- 20** Sinds 2014 maken alle Europese banken gebruik van het International Bank Account Number (IBAN). Een voorbeeld van een geldig Belgisch IBAN-bankrekeningnummer is BE69 7331 2345 6778. Een Belgisch IBAN-nummer wordt als volgt opgebouwd:

- BE is een internationaal afgesproken code voor Belgische banken.
- 69 is een eerste controlegetal.
- De eerste drie cijfers (733) vormen de bankcode. Dit getal verwijst naar de bank die de rekening beheert.

2. VARIABELEN EN COMMENTAAR

- De volgende zeven cijfers (1234567) vormen het rekeningnummer van de klant bij deze bank.
- De twee laatste cijfers (78) vormen een tweede controlegetal.

Het eigenlijke rekeningnummer heeft dus maar 10 cijfers, zoals 733-1234567. De controlegetallen worden als volgt berekend:

- Laat het liggend streepje weg. Bereken de rest bij deling van het eigenlijke rekeningnummer door 97. Het getal dat je zo bekomt, is het **tweede** controlegetal.
- Schrijf twee keer het tweede controlegetal en vul aan met 111400. Bereken van dit getal opnieuw de rest bij deling door 97. Trek deze rest af van 98. Het getal dat je zo bekomt, is het **eerste** controlegetal.

Met de twee controlegetallen kan snel gecontroleerd worden of een ingevoerd rekeningnummer wel geldig is. Als je een cijfer verkeerd ingeeft, zal dat dankzij deze controlegetallen altijd gedetecteerd worden. Zo verkleint de kans dat je per ongeluk geld overschrijft naar een verkeerd rekeningnummer.

- a. Controleer dat de twee controlegetallen voor het rekeningnummer 733-1234567 respectievelijk 69 en 78 zijn.
- b. Schrijf een programma waarmee je de twee controlegetallen berekent voor het rekeningnummer 135-7924680. Als uitvoer geeft het programma het volledige IBAN-nummer in de vorm **BE 69 7331234567 78**. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je programma in Dodona.

21 Pasen valt op de eerste zondag na de eerste volle maan van de lente. In zijn boek “Astronomical Algorithms” geeft de Belgische wiskundige en astronoom Jean Meeus (1928) een algoritme om de datum van Pasen in het jaar x ($x > 1582$) te berekenen. Schematisch ziet het er uit als volgt:

De waarde van n die je zo bekomt, is gelijk aan de maand waarin Pasen valt in het jaar x (3 = maart, 4 = april). De dag waarop Pasen valt, is gelijk aan $p + 1$.

Implementeer dit algoritme in Python. Bereken de datum van Pasen in 2022. Als uitvoer geeft het programma de paasdatum in de vorm **12 4 2020**. Test je programma in Dodona.

Deel	door	(gehele) quotiënt	rest
het jaartal x	19	$/$	a
het jaartal x	100	b	c
b	4	d	e
$b + 8$	25	f	$/$
$b - f + 1$	3	g	$/$
$19a + b - d - g + 15$	30	$/$	h
c	4	i	k
$32 + 2e + 2i - h - k$	7	$/$	l
$a + 11h + 22l$	451	m	$/$
$h + l - 7m + 114$	31	n	p

Datatypes en invoer van het toetsenbord

3.1 Instap

- 1 Je vindt dit programma in Dodona.

```
1 # vraag de naam van de gebruiker en geef een beleefde begroeting
2 naam = input('Wat is je naam? ')
3 print('Aangename kennismaking,', naam)
```

Kopieer en plak het programma in een IDE. Bewaar het programma en voer het uit. Noteer de uitvoer:

```
Wat is je naam?
```

3.2 Theorie

De functie `input()`

Je kunt in Python gebruik maken van de functie `input()` om invoer te vragen aan de gebruiker van een programma. De uitvoer van het programma wordt onderbroken tot wanneer de gebruiker een waarde invoert. Zodra de invoer ingegeven is, wordt de code verder uitgevoerd.

Je kunt de functie `input()` best uitvoeren door duidelijk een vraag te stellen. Zo ziet de gebruiker bij het uitvoeren van de code duidelijk dat het programma wacht op invoer. De vraag die je wilt stellen, zet je tussen aanhalingstekens binnen de haakjes van `input()`. Als je geen vraag stelt, zie je ook niet dat je programma uitgevoerd wordt. Je denkt dan misschien dat er iets niet werkt, terwijl het programma in feite wacht tot je iets intypt.

Datatypes

Python maakt een strikt onderscheid tussen verschillende soorten variabelen. De verschillende soorten variabelen worden **datatypes** genoemd. In dit hoofdstuk zal gebruik gemaakt worden van de volgende datatypes:

- Een **integer** is een geheel getal.
- Een **float** is een kommagetal.
- Een **string** is een stuk tekst. Strings worden altijd aangeduid met aanhalingstekens. Een string kan bestaan uit één letter, maar het mag ook een getal zijn, een woord, een zin, of zelfs het verzamelde werk van William Shakespeare. In Opgave 3.1 is de uitdrukking `'Wat is je naam?'` dus een string.

Python bepaalt het datatype van een variabele op basis van de manier waarop hij gedefinieerd wordt.

Voorbeeld 1:

```
1 # Python herkent datatypes aan de manier waarop de variabele
   gedefinieerd is:
2 woord = 'Python'    # de waarde van de variabele woord staat tussen
   aanhalingstekens. De variabele is dus van het type string.
3 x = -3              # de variabele x is een geheel getal, en is dus van
   het type integer
4 y = 5 / 2           # de variabele y is een kommagetal, en is dus van het
   type float
5 z = 4 / 2           # hoewel z een geheel getal is, is deze variabele
   gedefinieerd als een quotient. z is dus van het type float.
```

Bewerkingen met variabelen van het type string

De tabel vat een aantal mogelijkheden samen om te werken met variabelen van het type string.

Let op: wat wij het eerste karakter van de variabele `woord` noemen, is voor Python het nulde karakter. `woord[0]` verwijst dus naar het eerste karakter van `woord`. Wat wij in het algemeen het *i*-de karakter noemen, vind je dus met `woord[i-1]`.

Omzetten naar een ander datatype

De functie `input()` geeft altijd een waarde terug van het type string. Ook als je een getal ingeeft, interpreteert Python dit getal dus toch als een string.

Voorbeeld 2:

bewerking	operator	voorbeeld	resultaat
lengte van de variabele <code>woord</code>	<code>len()</code>	<code>len('Python')</code>	6
<code>woord_2</code> achter <code>woord_1</code> plakken	<code>woord_1 + woord_2</code>	<code>'reken' + 'machine'</code>	rekenmachine
de variabele <code>woord</code> <code>n</code> keer herhalen	<code>n * woord</code>	<code>3 * 'bla'</code>	blablabla
het <code>i</code> -de karakter lezen van de variabele <code>woord</code>	<code>woord[i]</code>	<code>woord = 'Python'</code> <code>woord[0]</code>	P

```

1 from math import sqrt
2 getal = input('Geef een natuurlijk getal: ')
3 print(3 * getal)
4 print(sqrt(getal))

```

```

Geef een natuurlijk getal: 42
424242
Traceback (most recent call last):
File "main.py", line 4, in <module>
print(sqrt(getal))
TypeError: must be real number, not str

```

De computer geeft een foutmelding. Je vraagt op lijn 4 om de vierkantswortel te berekenen van de string `'42'`. Dat gaat natuurlijk niet: je kunt geen vierkantswortel berekenen van een stuk tekst.

Dit voorbeeld toont dat je niet meteen kunt gaan rekenen met een getal dat via de functie `input()` ingelezen wordt. Python voorziet daarom de mogelijkheid om een variabele van het ene naar het andere datatype om te zetten.

bewerking	functie	voorbeeld	resultaat
zet een integer <code>a</code> om naar het type string	<code>str(a)</code>	<code>str(7)</code>	<code>'7'</code>
zet een float <code>a</code> om naar een string	<code>str(a)</code>	<code>str(-5.3)</code>	<code>'-5.3'</code>
zet een string <code>a</code> om naar een integer	<code>int(a)</code>	<code>int('-5')</code>	-5
zet een float <code>a</code> om naar een integer	<code>int(a)</code>	<code>int(-5.7)</code>	-5
zet een string <code>a</code> om naar een float	<code>float(a)</code>	<code>float('-5')</code>	-5.0
zet een integer <code>a</code> om naar een float	<code>float(a)</code>	<code>float(-5)</code>	-5.0

Voorbeeld 3:

3. DATATYPES EN INVOER VAN HET TOETSENBORD

```
1 from math import sqrt
2 # gebruik de functie int() om te eisen dat de variabele getal van het
   type integer is
3 getal = int(input('Geef een natuurlijk getal: '))
4 print(3 * getal)
5 print(sqrt(getal))
```

```
Geef een natuurlijk getal: 42
126
6.48074069840786
```

Op lijn 3 lees je een natuurlijk getal in met behulp van de functie `input()`. Het resultaat is van het type string, maar je gebruikt op dezelfde lijn de functie `int()` om de invoer meteen naar het datatype integer om te zetten. De variabele `getal` is dus een integer waarmee je wel kunt rekenen.

3.3 Verwerking

2 Je vindt dit programma in Dodona.

```
1 schooljaar_string = '2021 - 2022' # de waarde van deze variabele staat
   tussen aanhalingstekens. Deze variabele is dus van het type string.
2 schooljaar_integer = 2021 - 2022 # de waarde van deze variabele is
   gelijk aan het verschil van twee gehele getallen. Het verschil is
   dus zelf ook een geheel getal. Deze variabele is dus van het type
   integer.
3 print(schooljaar_string)
4 print(schooljaar_integer)
```

Kopieer en plak het programma in een IDE. Bewaar het programma en voer het uit. Noteer de uitvoer:



3 Je vindt dit programma in Dodona.

```
1 getal_string = '4' # deze variabele is een string
2 getal_integer = 4 # deze variabele is een integer
3 getal_float = 4.0 # deze variabele is een float
4 print(3 * getal_string)
5 print(3 * getal_integer)
6 print(3 * getal_float)
```

Kopieer en plak het programma in een IDE. Bewaar het programma en voer het uit. Noteer de uitvoer:



4 Een voetbalclub wil een aantal nieuwe ballen kopen. Schrijf een programma dat vraagt naar het aantal ballen, de prijs voor een bal, het kortingspercentage en de administratiekost. De verzendingskosten bedragen 0,5 euro per bal voor de eerste 10 ballen. Alle andere ballen worden gratis verzonden. De totale prijs wordt afgerond op 0,01 euro.

- a. Je krijgt een aantal Python-uitdrukkingen waarmee je deze opgave kunt oplossen. Selecteer de juiste uitdrukkingen. Noteer ze in een geschikte volgorde, maar zorg ervoor dat je eerst de waarde vraagt van `aantal_ballen`, daarna die van `eenheidsprijs`, dan het `kortingspercentage` en als laatste de `administratiekost`. Denk na over het datatype dat je wilt toekennen aan deze variabelen.

3. DATATYPES EN INVOER VAN HET TOETSENBORD

- (a) `aantal_ballen = float(input('Geef het gewenste aantal ballen: '))`
- (b) `aantal_ballen = input('Geef het gewenste aantal ballen: ')`
- (c) `aantal_ballen = int(input('Geef het gewenste aantal ballen: '))`
- (d) `eenheidsprijs = input('Geef de eenheidsprijs: ')`
- (e) `eenheidsprijs = float(input('Geef de eenheidsprijs: '))`
- (f) `eenheidsprijs = int(input('Geef de eenheidsprijs: '))`
- (g) `kortingspercentage = float(input('Geef het kortingspercentage: '))`
- (h) `kortingspercentage = int(input('Geef het kortingspercentage: '))`
- (i) `administratiekost = int(input('Geef de administratiekost: '))`
- (j) `administratiekost = float(input('Geef de administratiekost: '))`
- (k) `totaalprijs = aantal_ballen * eenheidsprijs`
- (l) `korting = kortingspercentage / 100 * totaalprijs`
- (m) `verzendingskosten = 0.5*min(10, aantal_ballen)`
- (n) `print(round(totaalprijs - korting + administratiekost + verzendingskosten,2))`

b. Breng je code over naar je IDE. Test je code in Dodona.

3.4 Opdrachten

Reeks 1

- 5** Schrijf een programma dat de lengte (in cm) van de rechthoekszijden vraagt in een rechthoekige driehoek. Het programma berekent en toont de lengte (in cm) van de schuine zijde, afgerond op 1 mm. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je code in Dodona.

- 6** Schrijf een programma dat de oppervlakte van een cirkel vraagt (in cm²). Het programma berekent en toont de straal van deze cirkel (uitgedrukt in cm), afgerond op 1 mm. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je code in Dodona.

- 7** De formule voor het verband tussen een temperatuur C in graden Celsius en een temperatuur F in graden Fahrenheit is:

$$F = \frac{9}{5} \cdot C + 32.$$

Schrijf een programma dat een temperatuur vraagt in °C, deze omzet in °F en het resultaat toont. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je code in Dodona.

- 8** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Schrijf een programma dat eerst de massa (in kg) en dan de lengte (in m) vraagt van een persoon. Bereken en toon de BMI van die persoon, afgerond op één cijfer na de komma. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je code in Dodona.

- 9** Schrijf een programma dat de straal (in cm) van een bol vraagt. Bereken en toon het volume van de bol. Toon het resultaat, uitgedrukt in cm³. Rond af op 1 cm³.

- 10** Schrijf een programma dat de lengte van de grote diagonaal en de kleine diagonaal van een ruit vraagt. Bereken de oppervlakte en de omtrek van deze ruit. Rond de oppervlakte af op 1 mm². Rond de omtrek af op 1 mm. Toon de resultaten op twee lijnen (eerst de oppervlakte, dan de omtrek). Schrijf minstens één lijn commentaar en gebruik minstens vier variabelen. Test je code in Dodona.

Reeks 2

- 11** Schrijf een programma dat een temperatuur vraagt in °F, deze omzet in °C en het resultaat toont. Rond de temperatuur af op 0,1°C en toon het resultaat. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je code in Dodona.
- 12** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.
Schrijf een programma dat de eerst de massa en dan de BMI vraagt van een persoon. Bereken en toon de lengte van die persoon. Druk de lengte uit in cm en rond af op 1 mm. Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je code in Dodona.
- 13** Een rechte met voorschrift $y = ax + b$ gaat door de punten $P(x_1, y_1)$ en $Q(x_2, y_2)$.
Schrijf een programma dat de waarde vraagt van x_1, y_1, x_2, y_2 (met $x_1 \neq x_2$). Bereken de waarde van a en b . Toon deze op één lijn (eerst a , dan b). Schrijf minstens één lijn commentaar en gebruik minstens zes variabelen. Test je code in Dodona.
- 14** Driehoek ABC is rechthoekig in B . Schrijf een programma dat de lengte van de rechthoekszijden $|AB|$ en $|BC|$ vraagt. Bereken $\sin \hat{A}$, $\cos \hat{A}$ en $\tan \hat{A}$. Toon de resultaten in deze volgorde, telkens op een aparte lijn en afgerond op 2 cijfers na de komma. Schrijf minstens één lijn commentaar en gebruik minstens zes variabelen. Test je code in Dodona.

Reeks 3

- 15** Je wilt een bedrag betalen met zo weinig mogelijk biljetten en munten. Je hebt voldoende biljetten van 50, 20, 10 en 5 euro en munten van 2 en 1 euro ter beschikking.
Schrijf een programma dat vraagt naar een bedrag in euro. Dit bedrag is een natuurlijk getal. Het programma berekent hoeveel biljetten van 50, 20, 10 en 5 en hoeveel munten van 2 en 1 je nodig hebt om dit bedrag te betalen. Schrijf minstens één lijn commentaar en gebruik minstens zeven variabelen. Toon de aantallen in deze volgorde op één lijn. Test je code in Dodona.
- 16** Op een bepaald tijdstip op een dag in december stel je een timer in die een aantal uur later zal aflopen.
Schrijf een programma dat vraagt naar de datum (1–31), het tijdstip (0–23) en de duur van de timer (een natuurlijk getal). Bereken de datum en het tijdstip wanneer de timer

afloopt. Je mag hierbij aannemen dat de timer altijd in de loop van de maand zal aflopen. Het programma toont deze twee natuurlijke getallen op één lijn (eerst datum, dan tijdstip). Schrijf minstens één lijn commentaar en gebruik minstens vijf variabelen. Test je code in Dodona.

17 Schrijf een programma dat een tijdstip (0–11) op een analoge klok vraagt en een aantal uren (een natuurlijk getal). De klok wordt nu dit aantal uren doorgedraaid. Bereken de hoek tussen de kleine en de grote wijzer en toon je antwoord. Schrijf minstens één lijn commentaar en gebruik minstens twee variabelen. Test je code in Dodona.

18 In de Angelsaksische wereld worden maten en gewichten uitgedrukt in het Imperial Standard System. Waar wij de lengte van een persoon uitdrukken in (centi)meter, wordt in bijvoorbeeld Groot-Brittannië en de Verenigde Staten foot (voet) en inch (duim) gebruikt. 1 foot komt overeen met 30,48 cm en 1 inch komt overeen met 2,54 cm. In 1 foot passen dus precies 12 inches.

Schrijf een programma dat de lengte van een persoon vraagt (in m). Zet deze lengte om in foot (een natuurlijk getal) en inches (afgerond op 0,1 inch). Toon deze getallen op één lijn (eerst foot, dan inches). Schrijf minstens één lijn commentaar en gebruik minstens drie variabelen. Test je code in Dodona.

19 Sinds 2014 maken alle Europese banken gebruik van het International Bank Account Number (IBAN). Een voorbeeld van een geldig Belgisch IBAN-bankrekeningnummer is BE69 7331 2345 6778. Een Belgisch IBAN-nummer wordt als volgt opgebouwd:

- BE is een internationaal afgesproken code voor Belgische banken.
- 69 is een eerste controlegetal.
- De eerste drie cijfers (733) vormen de bankcode. Dit getal verwijst naar de bank die de rekening beheert.
- De volgende zeven cijfers (1234567) vormen het rekeningnummer van de klant bij deze bank.
- De twee laatste cijfers (78) vormen een tweede controlegetal.

Het eigenlijke rekeningnummer heeft dus maar 10 cijfers, zoals 733-1234567. De controlegetallen worden als volgt berekend:

- Laat het liggend streepje weg. Bereken de rest bij deling van het eigenlijke rekeningnummer door 97. Het getal dat je zo bekomt, is het **tweede** controlegetal.

3. DATATYPES EN INVOER VAN HET TOETSENBORD

- Schrijf twee keer het tweede controlegetal en vul aan met 111400. Bereken van dit getal opnieuw de rest bij deling door 97. Trek deze rest af van 98. Het getal dat je zo bekomt, is het **eerste** controlegetal.

Met de twee controlegetallen kan snel gecontroleerd worden of een ingevoerd rekeningnummer wel geldig is. Als je een cijfer verkeerd ingeeft, zal dat dankzij deze controlegetallen altijd gedetecteerd worden. Zo verkleint de kans dat je per ongeluk geld overschrijft naar een verkeerd rekeningnummer.

- a. Controleer dat de twee controlegetallen voor het rekeningnummer 123-4567890 respectievelijk 32 en 02 zijn.
- b. Schrijf een programma dat als invoer een getal van 10 cijfers inleest (bv. 7331234567) dat overeenkomt met een Belgisch rekeningnummer. Er wordt **geen** liggend streepje ingevoerd tussen de bankcode en het eigenlijke rekeningnummer. Het programma berekent de twee controlegetallen. Als uitvoer geeft het programma het volledige IBAN-nummer in de vorm **BE 69 7331234567 78**. Test je code in Dodona.

- 20** Pasen valt op de eerste zondag na de eerste volle maan van de lente. In zijn boek "Astronomical Algorithms" geeft de Belgische wiskundige en astronoom Jean Meeus (1928) een algoritme om de datum van Pasen in het jaar x ($x > 1582$) te berekenen. Schematisch ziet het er uit als volgt:

Deel	door	(gehele) quotiënt	rest
het jaartal x	19	/	a
het jaartal x	100	b	c
b	4	d	e
$b + 8$	25	f	/
$b - f + 1$	3	g	/
$19a + b - d - g + 15$	30	/	h
c	4	i	k
$32 + 2e + 2i - h - k$	7	/	l
$a + 11h + 22l$	451	m	/
$h + l - 7m + 114$	31	n	p

De waarde van n die je zo bekomt, is gelijk aan de maand waarin Pasen valt in het jaar x (3 = maart, 4 = april). De dag waarop Pasen valt, is gelijk aan $p + 1$.

Implementeer dit algoritme in Python. Vraag de gebruiker naar een jaartal. Als uitvoer geeft het programma de paasdatum in dat jaar. De uitvoer wordt gegeven in de vorm **12 / 4 / 2020**. Test je code in Dodona.

4.1 Instap

- 1 Je vindt dit programma in Dodona. Kopieer en plak het programma in een IDE.

```
1 # je gebruikt het functievoorschrift  $f(x) = 3x+5$ 
2 # deze functie vraagt een x-waarde x en berekent  $y=f(x)$ 
3 def f(x):
4     y = 3*x + 5
5     return(y)
6
7 # na de definitie van de functie f begint hier het hoofdprogramma
8 x = float(input('Geef een x-waarde: '))
9 y = f(x)
10 print(y)
```

Voer het programma uit. Geef x de waarde -2. Noteer de uitvoer:

```
Geef een x-waarde: -2
```

- 2 Je vindt dit programma in Dodona. Kopieer en plak het programma in een IDE.

```
1 from math import sqrt
2 # definieer de functie schuine_zijde
3 # deze leest de waarde in van de twee rechthoekszijden en berekent de
  lengte van de schuine zijde in een rechthoekige driehoek
4 def schuine_zijde(a, b):
5     c = sqrt(a**2 + b**2)
```

```
6     return(c)
7
8 # na de definitie van de functie schuine_zijde begint hier het
   hoofdprogramma
9 x = float(input('Geef de lengte (in cm) van de eerste rechthoekszijde: '
   ))
10 y = float(input('Geef de lengte (in cm) van de tweede rechthoekszijde: '
   ))
11 z = schuine_zijde(x, y)
12 print('De lengte van de schuine zijde is gelijk aan', z, 'cm.')
```

Voer het programma uit. Geef **x** de waarde 3. Geef **y** de waarde 4. Noteer de uitvoer:

```
Geef de lengte (in cm) van de eerste rechthoekszijde: 3
Geef de lengte (in cm) van de tweede rechthoekszijde: 4
```

4.2 Theorie

Opgave 1 maakt duidelijk dat het gedrag van een zelf gedefinieerde functie in Python heel erg lijkt op dat van een wiskundige functie. De Python-functie leest een getal in, zet dat om naar een ander getal en voert de berekende waarde terug uit naar het hoofdprogramma via de **return()**-functie.

Door functies te gebruiken, haal je een deel van de code weg uit je hoofdprogramma. In het hoofdprogramma roep je de functie dan op, net zoals je dat zou doen met een standaard functie (zoals **abs()**) of een geïmporteerde functie (zoals **sqrt()**).

Je hoeft in het hoofdprogramma niet te weten *hoe* de uitvoer van **f()** precies berekend wordt. Je moet alleen weten *wat* de functie **f()** doet. De functie **f()** verwacht precies één getal als invoer. Je zegt dat de functie één *parameter* heeft.

De functie **f()** berekent op basis van deze parameter één enkel getal **y**. Dit berekende getal wordt via **return(y)** teruggestuurd naar het hoofdprogramma.

Decompositie, abstractie en functies

Decompositie

Met *decompositie* wordt het opsplitsen bedoeld van een groot, ingewikkeld probleem in kleinere, behapbare deelproblemen die je apart kunt oplossen. In plaats van meteen te proberen het hele probleem in één keer op te lossen, breek je het op in stukken die elk op zich veel eenvoudiger zijn.

Decompositie is geen concept dat specifiek is voor programmeren, maar een algemene denkstrategie om complexe problemen aan te pakken. Als je een driegangenmenu wilt klaarmaken, splits je die taak ook op in “maak het voorgerecht”, “maak het hoofdgerecht” en “maak het dessert”. Als je hulp krijgt in de keuken, kan je de taken perfect verdelen. De persoon die het voorgerecht maakt, kan zich alleen daarop focussen en moet in principe niet weten hoe het hoofdgerecht gemaakt wordt.

Ook in de context van programmeren is het altijd een goed idee om een ingewikkeld probleem op te splitsen in deelproblemen. Voor elk van deze deelproblemen kan je een functie schrijven. Op die manier is elke oplossing voor een deelprobleem op zichzelf klein en gemakkelijk te begrijpen. Je kunt je code gemakkelijk testen en aanpassen. Om het volledige probleem op te lossen, maak je in het hoofdprogramma gebruik van de deelfuncties die je gedefinieerd hebt.

Abstractie

Dankzij functies kan je *abstractie* inbouwen in je code. “Abstractie maken van iets” betekent dat je je focust op *wat* iets doet, zonder je bezig te houden met *hoe* het precies gebeurt. Je verbergt details, zodat je met het grotere geheel kunt blijven werken zonder overweldigd te raken.

Als je bijvoorbeeld een functie `schuine_zijde(a, b)` definieert, dan hoeft je op het niveau van je hoofdprogramma niet te weten dat daar intern een formule met een vierkantswortel in zit. Het hoofdprogramma roept gewoon `schuine_zijde(a, b)` op en krijgt een antwoord terug. Je hebt de complexiteit van de berekening weggeabstraheerd achter een simpele naam.

Voordelen van functies

Het gebruik van functies biedt een aantal duidelijke voordelen:

- Functies volgen vaak vanzelf uit het opsplitsen van een ingewikkeld probleem in kleinere deelproblemen (decompositie).
- Je kunt met verschillende mensen samenwerken aan één groot project. Je verdeelt de taken, en je daarbij hoeft niet in detail te weten hoe elk deelprobleem opgelost wordt.
- Functies laten toe om abstractie in te bouwen: je kunt je in je hoofdprogramma focussen op de grote lijnen. De details zitten in de aparte functies.
- Je code wordt leesbaarder en gemakkelijker te onderhouden.
- Stukken code kunnen op een elegante manier hergebruikt worden.

Syntax

Python voert je code lijn per lijn uit, van boven naar beneden. Wanneer je in het hoofdprogramma een functie oproept, moet die eerst gedefinieerd zijn. Je kunt je functie dus best bovenaan definiëren, voor je aan het hoofdprogramma begint.

In Python definieer je een functie als volgt:

- Je begint met het commando **def**, gevolgd door een spatie. Daarna komt de naam van de functie. Deze naam kies je zelf. Je moet rekening houden met dezelfde beperkingen als bij de naam van een variabele. Na de naam van de functie komt tussen haakjes de parameter(s) van de functie. Als je meer dan één parameter gebruikt, worden de parameters van elkaar gescheiden door een komma. Op het einde van de regel komt een dubbele punt. **Deze dubbele punt is belangrijk. Als je geen dubbele punt typt, beschouwt Python dit als een syntax error. Het programma zal niet uitgevoerd kunnen worden.**
- Op de volgende regels komt dan de implementatie van je functie. **Het is belangrijk dat dit stuk code ingesprongen is. Zo zie je visueel meteen welke code deel uitmaakt van de definitie van de functie. Als je code niet inspringt, beschouwt Python dit als een syntax error. Het programma zal niet uitgevoerd kunnen worden.**
- De berekende waarde wordt teruggestuurd naar het hoofdprogramma via de functie **return()**.

In pseudocode ziet het er zo uit:

```
1 # het is altijd goed om het gedrag van je functie te documenteren:
2 # hoeveel parameters verwacht je functie? welk datatype moet(en) deze
  hebben?
3 # wat is de uitvoer? welk datatype heeft de uitvoer?
4
5 def functie_naam(parameter_1, parameter_2, ..., parameter_n):
6     # merk op dat deze regel ingesprongen is!
7     # in dit blokje bereken je de uitvoer van je functie
8     # met de return-functie stuur je de berekende waarde terug naar het
  hoofdprogramma
9     # de definitie van de functie stopt dan ook hier
10    return(uitvoer)
11
12 # op deze lijn stopt ook het inspringen: hier begint dus het
  hoofdprogramma
13 # in het hoofdprogramma roep je de functie op als volgt:
14 resultaat = functie_naam(x_1, x_2, ..., x_n)
```

Meteen nadat de uitvoer is teruggestuurd naar het hoofdprogramma, stopt ook de uitvoering van de functie. Python herneemt dan het hoofdprogramma op de plaats waar de functie opgeroepen werd.

4.3 Overzicht

In dit hoofdstuk heb je kennigemaakt met functies. Hieronder vind je een overzicht van de belangrijkste begrippen en constructies.

Wat is een functie?

Een zelf gedefinieerde functie in Python gedraagt zich net als een wiskundige functie: ze leest een of meerdere waarden in (de *parameters*), berekent daarmee een resultaat en stuurt dat resultaat terug naar het hoofdprogramma via **return()**.

In het hoofdprogramma hoef je niet te weten *hoe* een functie haar uitvoer precies berekent. Je moet alleen weten *wat* de functie doet.

Syntax

Python voert je code lijn per lijn uit, van boven naar beneden. Een functie moet dus gedefinieerd zijn voor je ze in het hoofdprogramma oproept. Definieer je functies daarom best bovenaan je programma.

- Je begint de definitie met het commando **def**, gevolgd door de naam van de functie (die je zelf kiest, met dezelfde beperkingen als bij een variabele) en tussen haakjes de parameter(s) van de functie. Gebruik je meer dan één parameter, dan scheid je die van elkaar met een komma. De regel eindigt met een dubbele punt. **Vergeet die dubbele punt niet, anders beschouwt Python dit als een syntax error.**
- Op de volgende regels komt de implementatie van je functie. **Dit stuk code moet ingesprongen zijn.** Zo is meteen duidelijk welke code bij de functie hoort. Ontbreekt het inspringen, dan beschouwt Python dit ook als een syntax error.
- Met **return()** stuur je de berekende waarde terug naar het hoofdprogramma. Zodra deze regel uitgevoerd is, stopt de functie. Python gaat dan verder met het hoofdprogramma, op de plaats waar de functie werd opgeroepen.

In pseudocode ziet dat er zo uit:

```
1 def functie_naam(parameter_1, parameter_2, ..., parameter_n):
2     # dit blokje code is ingesprongen
3     # hier bereken je de uitvoer van je functie
4     return(uitvoer)
5
6 # hier stopt het inspringen: vanaf hier begint het hoofdprogramma
7 resultaat = functie_naam(x_1, x_2, ..., x_n)
```

4.4 Verwerking

3 Je vindt in Dodona een stuk ongedocumenteerde code:

```
1 def C_naar_F(C):
2     F = 1.8*C + 32
3     return(F)
4
5 print(C_naar_F(20))
```

- Waarvoor zou je de functie `C_naar_F()` kunnen gebruiken?
- Bereken zelf de waarde van `C_naar_F(20)`.
- Kopieer en plak de code in je IDE. Voer het programma uit. Noteer de uitvoer.

4 Schrijf een programma dat de straal van een cirkel vraagt. De uitvoer moet gelijk zijn aan de oppervlakte van de cirkel, afgerond op 1 mm².

- Je krijgt een aantal Python-uitdrukkingen om deze opgave op te lossen. Rangschik ze in een juiste volgorde. Let op het correct inspringen van sommige stukken code.

- `straal = float(input('Geef de straal van de cirkel: '))`
- `print(oppervlakte_cirkel(straal))`
- `def oppervlakte_cirkel(r):`
- `return(round(pi * r**2, 2))`
- `from math import pi`

- Test je programma in Dodona.

5 Je vindt in Dodona een programma.

```
1 from math import sqrt
2
3 # de functie schuine_zijde() leest de waarde (type float) in van de twee
   rechthoekszijden in een rechthoekige driehoek
```

4. FUNCTIES

```
4 # de uitvoer is de lengte van de schuine zijde (type float)
5 def schuine_zijde(a, b):
6     c = sqrt(a**2 + b**2)
7     return(c)
8
9 # de functie bereken_sinus() leest de waarde (type float) in van de
    overstaande en de aanliggende rechthoekszijde in een rechthoekige
    driehoek
10 # de uitvoer is de sinus van de hoek (type float)
11 def bereken_sinus(aanliggende, overstaande):
12     schuine = schuine_zijde(aanliggende, overstaande)
13     sinus = overstaande / schuine
14     return(sinus)
15
16 # na de definitie van de twee functies begint hier het hoofdprogramma
17 a = float(input('Geef de lengte (in cm) van de aanliggende
    rechthoekszijde: '))
18 b = float(input('Geef de lengte (in cm) van de overstaande
    rechthoekszijde: '))
19 sinus = bereken_sinus(a, b)
20 print(sinus)
```

- a. Kopieer en plak de code in je IDE. Voer het programma uit in je IDE. De aanliggende zijde AB heeft een lengte van 4 cm. De overstaande zijde BC heeft een lengte van 3 cm. Noteer de uitvoer:

```
Geef de lengte (in cm) van de aanliggende rechthoekszijde:
Geef de lengte (in cm) van de overstaande rechthoekszijde:
```

- b. Omschrijf de uitvoer van dit programma.
- c. Voeg aan dit programma een functie `bereken_cosinus()` toe. Deze functie leest (in deze volgorde) de lengte van de aanliggende en de overstaande rechthoekszijde in. De uitvoer van de functie is de cosinus van de hoek $\hat{A}$. Verwijder het hoofdprogramma. Test je code in Dodona.

4.5 Opdrachten

Reeks 1

- 6** De Body Mass Index (BMI) van een persoon met massa m (uitgedrukt in kg) en lengte l (uitgedrukt in m) wordt gegeven door $\frac{m}{l^2}$.

Schrijf een functie `bereken_BMI()` met twee parameters: de lengte (uitgedrukt in cm) en de massa (uitgedrukt in kg) van een persoon. De functie berekent de BMI van die persoon, afgerond op één cijfer na de komma, en stuurt deze waarde terug. Schrijf minstens één lijn commentaar. Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.

- 7** Gegeven zijn de punten $P(x_1, y_1)$ en $Q(x_2, y_2)$.
- Schrijf een functie `x_middelpunt()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de x-coördinaat van het middelpunt van lijnstuk $[PQ]$ en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Schrijf een functie `y_middelpunt()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de y-coördinaat van het middelpunt van lijnstuk $[PQ]$ en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Schrijf een functie `lengte_PQ()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de lengte van het lijnstuk $[PQ]$ en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.

Reeks 2

- 8** Schrijf een functie `A_naar_r()` met één parameter: de oppervlakte van een cirkel (in cm^2). De functie berekent de straal van deze cirkel (uitgedrukt in cm) op 1 mm nauwkeurig en stuurt deze terug. Schrijf minstens één lijn commentaar. Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.
- 9** Schrijf een functie `F_naar_C()` met één parameter: een temperatuur in $^{\circ}\text{F}$. De functie berekent de temperatuur in $^{\circ}\text{C}$ en stuurt deze terug, afgerond op $0,1^{\circ}\text{C}$. Schrijf minstens één lijn commentaar. Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.

Reeks 3

- 10** Een schuine rechte met voorschrift $y = ax + b$ gaat door de punten $P(x_1, y_1)$ en $Q(x_2, y_2)$.
- Schrijf een functie `rico()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de rico a van rechte PQ en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Schrijf een functie `snijpunt_y()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de y -coördinaat b van het snijpunt van rechte PQ met de y -as en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Schrijf een functie `snijpunt_x()` met vier parameters, x_1, y_1, x_2, y_2 . De functie berekent de x -coördinaat van het snijpunt van rechte PQ met de x -as en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.
- 11** In de Angelsaksische wereld worden maten en gewichten uitgedrukt in het Imperial Standard System. Waar wij de lengte van een persoon uitdrukken in (centi)meter, wordt in bijvoorbeeld Groot-Brittannië en de Verenigde Staten foot (voet) en inch (duim) gebruikt. 1 foot komt overeen met 30,48 cm en 1 inch komt overeen met 2,54 cm. In 1 foot passen dus precies 12 inches.
- Schrijf een functie `cm_naar_ft()` met één parameter: de lengte van een persoon (uitgedrukt in cm). De functie berekent het aantal foot (een natuurlijk getal) en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Schrijf een functie `cm_naar_in()` met één parameter: de lengte van een persoon (uitgedrukt in cm). De functie berekent het aantal inch (een natuurlijk getal kleiner dan 12) en stuurt deze waarde terug. Schrijf minstens één lijn commentaar.
 - Test je code in Dodona, maar let daarbij op dat je geen hoofdprogramma ingeeft.

5.1 Instap

1

- a. Welke zijn de mogelijke waarden van de rest als je een natuurlijk getal `n` deelt door 2?
- b. Hoe noem je een natuurlijk getal `n` waarvan de rest bij deling door 2 gelijk is aan nul?
- c. Hoe noem je een natuurlijk getal `n` waarvan de rest bij deling door 2 niet gelijk is aan nul?
- d. In Hoofdstuk 1 heb je kennis gemaakt met een operator om in Python de rest te berekenen bij de staartdeling van twee getallen. Hoe bereken je in Python de rest bij deling van `deeltal` door `deler`?
- e. Je vindt dit programma in Dodona.

```
1 getal = 24
2 # bereken de rest bij deling van 24 door 2:
3 rest = getal % 2
4 # controleer of de rest gelijk is aan nul:
5 if rest == 0:
6     # als de rest gelijk is aan nul, zeg je dat n een even getal is
7     print(getal, 'is een even getal.')
```

- (a) Kopieer en plak het programma in een IDE. Bewaar het programma en voer het uit. Noteer de uitvoer:

- (b) Pas lijn 1 van het programma aan. Geef de variabele `getal` de waarde 35. Bewaar het programma en voer het uit. Noteer hier de uitvoer die de computer geeft:

2 Je vindt dit programma in Dodona.

```
1 letter = input('Typ een hoofdletter: ')
2 # controleer of de variabele letter een klinker is:
3 if letter == 'A' or letter == 'E' or letter == 'I' or letter == 'O' or
   letter == 'U':
4     # als letter een klinker is, print dat dan ook:
5     print(letter, 'is een klinker.')
6 else:
7     # als letter geen klinker is, is het dus een medeklinker. Print dat
   dan ook:
8     print(letter, 'is een medeklinker.')
```

Kopieer en plak het programma in een IDE. Bewaar het programma en voer het uit. Test het een paar keer door verschillende letters in te voeren. Probeer het ook eens met een kleine letter a. Noteer telkens de uitvoer die de computer geeft:

Typ een hoofdletter:

Typ een hoofdletter:

Typ een hoofdletter:

5.2 Theorie

Een nieuw datatype: Booleans

Zowel in Opgave 1 (lijn 5) als in Opgave 2 (lijn 3) wordt er gecontroleerd of de waarde van een variabele gelijk is aan een andere waarde. Er zijn maar twee mogelijke uitkomsten voor deze test.

- Ofwel zijn beide waarden gelijk. In dit geval zeg je dat het resultaat van de test **True** (waar) is.
- Ofwel zijn de waarden niet gelijk. In dit geval zeg je dat het resultaat van de test **False** (onwaar) is.

De waarden **True** en **False** komen in de computerwetenschappen zo vaak voor dat ze samen het datatype *boolean** vormen. De naam boolean is misschien nieuw, maar je bent zeker vertrouwd met het concept. Wat een computerwetenschapper **True** noemt, heb je bij Techniek een logische 1 genoemd. De boolean **False** komt overeen met een logische 0 bij Techniek.

De tabel geeft een overzicht van de verschillende mogelijkheden om gegevens met elkaar te vergelijken.

bewerking	operator	voorbeeld	resultaat
gelijk zijn aan	<code>==</code>	<code>3 == 4</code>	False
niet gelijk zijn aan	<code>!=</code>	<code>-2 != 5</code>	True
groter zijn dan	<code>></code>	<code>4 > 1</code>	True
groter zijn dan of gelijk zijn aan	<code>>=</code>	<code>-1 >= 3</code>	False
kleiner zijn dan	<code><</code>	<code>7 < -12</code>	False
kleiner zijn dan of gelijk zijn aan	<code><=</code>	<code>6 <= 6</code>	True

Let op: een gewoon gelijkheidsteken (`=`) wordt in Python uitsluitend gebruikt om een waarde toe te kennen aan een variabele. Om te testen of twee waarden gelijk zijn aan elkaar, heb je een dubbel gelijkheidsteken nodig (`==`).

Zoals je bij Techniek geleerd hebt, kan je Booleaanse waarden met elkaar combineren via de logische bewerkingen **and**, **or** en **not**. De volgende tabellen vatten de regels van deze bewerkingen samen.

and	True	False
True	True	False
False	False	False

or	True	False
True	True	True
False	True	False

not	
True	False
False	True

Er bestaan nog andere manieren om een Boolean te bekomen. Met de operator **in** kan je bijvoorbeeld testen of een stukje tekst (van het type string) voorkomt in een ander stuk tekst.

*George Boole (1815 – 1864) was een Brits wiskundige. Hij was de bedenker van wat tegenwoordig booleaanse logica wordt genoemd, die aan de basis ligt van de moderne digitale computerlogica. Daardoor wordt Boole beschouwd als één van de grondleggers van de computerwetenschap. Ook de operatoren AND, OR, NOT, waar onder andere zoekmachines gebruik van maken voor het specificeren van zoekopdrachten, worden nu nog booleaanse operatoren genoemd.

5. VOORWAARDELIJKE UITVOERING

Voorbeeld 1:

```
1 if 'f' in 'taart':
2     print('De letter f komt voor in het woord taart.')
3 else:
4     print('De letter f komt niet voor in het woord taart.')
```

```
De letter f komt niet voor in het woord taart.
```

Voorbeeld 2:

```
1 if 'art' in 'taart':
2     print('De letters art komen voor in het woord taart.')
3 else:
4     print('De letters art komen niet voor in het woord taart.')
```

```
De letters art komen voor in het woord taart.
```

Let op: de operator **in** kan alleen overweg met gegevens van het type string.

Voorbeeld 3:

```
1 tekst = 'In 2009 liep Usain Bolt de 100 m in 9,58s.'
2 print(100 in tekst)
```

```
TypeError: 'in <string>' requires string as left operand, not int
```

Voorbeeld 4:

```
1 getal = 24
2 print(3 in getal)
```

```
Traceback (most recent call last):
  File "main.py", line 14, in <module>
    print(3 in getal)
  TypeError: argument of type 'int' is not iterable
```

Als je wilt testen of een cijfer of getal voorkomt in een ander getal of in een stuk tekst, moet je dit eerst omzetten naar het datatype string.

Voorbeeld 5:

```
1 tekst = 'In 2009 liep Usain Bolt de 100 m in 9,58s.'  
2 print('100' in tekst)
```

```
True
```

Voorbeeld 6:

```
1 getal = '24'  
2 print('3' in getal)
```

```
False
```

if ...

Opgave 1 illustreerde de meest eenvoudige manier om een voorwaardelijke lus te maken in Python. In pseudocode zou je dit als volgt noteren:

```
1 # controleer of voldaan is aan de Booleaanse variabele voorwaarde  
2 if voorwaarde:  
3     # als voldaan is aan voorwaarde, dan wordt dit blokje code  
    uitgevoerd  
4     # ...  
5 # vanaf hier komt het verdere verloop van het programma
```

Syntactisch zijn er in Python twee zaken waar je op moet letten:

- Je ziet dat op het einde van het **if**-statement (lijn 2 in de pseudocode) een dubbele punt staat.
- Verder valt het op dat de code van de voorwaardelijke lus (lijnen 3-4) ingesprongen is.

Als er niet voldaan is aan deze beide eisen, beschouwt Python dit als een syntax error. Gelukkig helpt je IDE je hierbij. Wanneer je een dubbele punt typt en de regel afsluit met enter, zorgt de IDE ervoor dat de volgende regel automatisch inspringt.

Op lijn 2 evalueert de computer of er voldaan is aan **voorwaarde**. Als deze **True** is, wordt het blokje code op lijnen 3-4 uitgevoerd. Afhankelijk van **voorwaarde** wordt de code op deze lijnen soms wel en soms niet uitgevoerd. Een dergelijk stukje code noem je een **voorwaardelijke lus**.

Nadat de voorwaardelijke lus op lijnen 3-4 uitgevoerd is, gaat het programma verder op lijn 5. Ook als er niet voldaan is aan **voorwaarde** (als deze **False** is), gaat het programma meteen verder op lijn 5.

if ... else

Opgave 2 en Voorbeelden 1-2 illustreerden een variant op het **if**-statement. Als **letter** gelijk was aan **A**, **E**, **I**, **O** of **U**, werd er geprint dat de letter een klinker was. In alle andere gevallen kwam de mededeling dat de letter een medeklinker was – ook als de letter een kleine letter **a** was.

In pseudocode zou je deze variant als volgt kunnen weergeven:

```
1 # controleer of voldaan is aan de Booleaanse variabele voorwaarde
2 if voorwaarde:
3     # als voldaan is aan voorwaarde, dan wordt dit eerste blokje code
    uitgevoerd
4     # ...
5 else:
6     # als er niet voldaan is aan voorwaarde, dan wordt dit tweede blokje
    code uitgevoerd
7     # ...
8 # vanaf hier komt het verdere verloop van het programma
```

Op lijn 2 evalueert de computer of er voldaan is aan **voorwaarde**. Als deze **True** is, wordt de code op lijnen 3-4 uitgevoerd. Als **voorwaarde** op lijn 2 **False** is, zorgt de **else:** op lijn 5 ervoor dat lijnen 6-7 worden uitgevoerd.

Er zal dus altijd precies één van de twee voorwaardelijke lussen uitgevoerd worden. Na het uitvoeren van de voorwaardelijke lus herneemt de computer het programma vanaf lijn 8.

Je merkt dat er een dubbele punt staat na zowel het **if**- als het **else**-statement. De voorwaardelijke blokjes code op lijnen 3-4 en lijnen 6-7 staan opnieuw ingesprongen.

if ... elif ... else

Een derde variant bekom je door nog één of meerdere **elif**-blokken toe te voegen. **elif** is een afkorting voor het Engelse **else if**. In pseudocode ziet de constructie er zo uit:

```
1 # controleer of voldaan is aan de Booleaanse variabele voorwaarde1
2 if voorwaarde1:
3     # als voldaan is aan voorwaarde1, dan wordt dit eerste blokje code
    uitgevoerd
4     # ...
5 elif voorwaarde2:
6     # als er niet voldaan is aan voorwaarde1, maar wel aan voorwaarde2,
    dan wordt dit tweede blokje code uitgevoerd
7     # ...
8 else:
```



```
9      # als er niet voldaan is aan een van beide voorwaarden, dan wordt
      dit derde blokje code uitgevoerd
10     # ...
11 # vanaf hier komt het verdere verloop van het programma
```

Op lijn 2 wordt **voorwaarde1** gecontroleerd. Als deze **True** is, wordt het bijhorende blokje code (lijnen 3-4) uitgevoerd. Zodra dit gebeurd is, gaat het programma verder op lijn 11.

Als er niet voldaan is aan **voorwaarde1**, gaat het programma verder op lijn 5. Daar wordt **voorwaarde2** gecontroleerd. Als deze **True** is, wordt het bijhorende blokje code (lijnen 6-7) uitgevoerd. Wanneer dit gebeurd is, gaat het programma verder op lijn 11.

Als er aan geen van beide voorwaarden voldaan is, gaat het programma verder op lijn 8. Daar staat een **else:**. Als beide voorwaarden **False** zijn, wordt het blokje code op lijnen 9-10 uitgevoerd.

Je ziet dat er een dubbele punt moet staan na zowel het **if**-, het **elif**- en het **else**-statement. De bijhorende blokjes code moeten ook inspringen.

In de pseudocode wordt het **if**-statement uitgebreid met één enkel **elif**-statement. Je mag een **if**-constructie uitbreiden met meerdere **elif**-statements. Het **else**-gedeelte dat in de pseudocode wordt getoond, hoeft ook niet altijd toegevoegd worden.

5.3 Overzicht

In dit hoofdstuk heb je kennism gemaakt met voorwaardelijke uitvoering. Hieronder vind je een overzicht van de belangrijkste begrippen en constructies.

Booleans

Naast integer, float en string heb je een vierde datatype leren kennen: de *boolean*. Een boolean kan slechts twee waarden aannemen: **True** (waar) of **False** (onwaar). Je krijgt een boolean onder meer door twee waarden met elkaar te vergelijken.

bewerking	operator	voorbeeld	resultaat
gelijk zijn aan	<code>==</code>	<code>3 == 4</code>	False
niet gelijk zijn aan	<code>!=</code>	<code>-2 != 5</code>	True
groter zijn dan	<code>></code>	<code>4 > 1</code>	True
groter zijn dan of gelijk zijn aan	<code>>=</code>	<code>-1 >= 3</code>	False
kleiner zijn dan	<code><</code>	<code>7 < -12</code>	False
kleiner zijn dan of gelijk zijn aan	<code><=</code>	<code>6 <= 6</code>	True

Let op: gebruik altijd een dubbel gelijkheidsteken (`==`) om twee waarden te vergelijken. Een enkel gelijkheidsteken (`=`) gebruik je uitsluitend om een waarde toe te kennen aan een variabele.

Je kunt booleans ook met elkaar combineren via de logische bewerkingen **and**, **or** en **not**. Met de operator **in** kan je dan weer testen of een stukje tekst voorkomt in een ander stuk tekst. Let wel op: deze operator werkt alleen met gegevens van het type string. Wil je testen of een getal voorkomt in een ander getal of in een tekst, dan moet je dat getal eerst omzetten naar een string.

De drie vormen van het **if**-statement

Een stukje code dat voorwaardelijk uitgevoerd wordt, noem je een **voorwaardelijke lus**. In Python bestaan daar drie varianten van.

- Met **if ...** laat je een blokje code enkel uitvoeren als er voldaan is aan een voorwaarde. Is de voorwaarde **False**, dan slaat het programma dat blokje gewoon over.

```
1 if voorwaarde:
2     # wordt enkel uitgevoerd als voorwaarde True is
3     # ...
```

- Met **if** ... **else** zorg je ervoor dat er altijd precies één van beide blokjes code uitgevoerd wordt: het eerste als de voorwaarde **True** is, het tweede als ze **False** is.

```
1 if voorwaarde:
2     # wordt uitgevoerd als voorwaarde True is
3     # ...
4 else:
5     # wordt uitgevoerd als voorwaarde False is
6     # ...
```

- Met **if** ... **elif** ... **else** kan je meerdere voorwaarden na elkaar controleren. Zodra een voorwaarde **True** is, wordt het bijhorende blokje code uitgevoerd en slaat het programma de rest van de constructie over. Is geen enkele voorwaarde **True**, dan wordt het **else**-blokje uitgevoerd (als dat aanwezig is).

```
1 if voorwaarde1:
2     # wordt uitgevoerd als voorwaarde1 True is
3     # ...
4 elif voorwaarde2:
5     # wordt uitgevoerd als voorwaarde1 False is, maar voorwaarde2
    True
6     # ...
7 else:
8     # wordt uitgevoerd als geen van beide voorwaarden True is
9     # ...
```

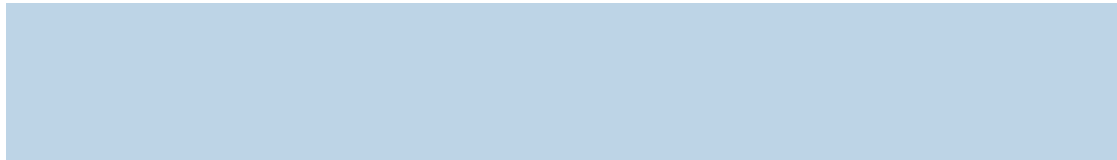
Onthoud dat elk **if**-, **elif**- en **else**-statement altijd afgesloten wordt met een dubbele punt, en dat de bijhorende code telkens ingesprongen moet worden. Zonder deze twee zaken beschouwt Python je code als een syntax error.

5.4 Verwerking

- 3** Je vindt dit programma in Dodona. Analyseer de code, maar voer ze nog niet uit in een IDE.

```
1 a = 3
2 b = 2
3 print(a < b)
4 print(a == b)
5 print(a != b)
6 print(a >= b)
```

Noteer de uitvoer die de computer volgens jou zal geven. Controleer door de code te plakken in een IDE en het programma uit te voeren.



- 4** Een programma vraagt de waarde van een eerste geheel getal. Wanneer de gebruiker dit heeft ingegeven, vraagt het de waarde van een tweede geheel getal. Als het eerste getal gelijk is aan het tweede getal, toont het programma als uitvoer **De getallen zijn gelijk aan elkaar..** Als het eerste getal groter is dan het tweede getal, toont het programma als uitvoer **Het eerste getal is groter dan het tweede getal..** Als het eerste getal kleiner is dan het tweede getal, toont het programma als uitvoer **Het eerste getal is kleiner dan het tweede getal..**

- a. Je krijgt een aantal Python-uitdrukkingen. Rangschik ze in de juiste volgorde. Besteed aandacht aan het inspringen van sommige stukken code.

- (a) `print('De getallen zijn gelijk aan elkaar.')`
- (b) `else:`
- (c) `print('Het eerste getal is kleiner dan het tweede getal.')`
- (d) `elif getal_1 > getal_2:`
- (e) `if getal_1 == getal_2:`

(f) `getal_1 = int(input('Geef een eerste geheel getal: '))`

(g) `print('Het eerste getal is groter dan het tweede getal.')`

(h) `getal_2 = int(input('Geef een tweede geheel getal: '))`

b. Breng je code over naar je IDE. Test je code in Dodona.

5

a. Je vindt dit ongedocumenteerde programma in Dodona. Analyseer de code, maar voer ze nog niet uit in een IDE.

```
1 getal = 21
2 if getal % 3 == 0:
3     print(getal, 'is deelbaar door 3.')
4 elif getal % 7 == 0:
5     print(getal, 'is deelbaar door 7.')
```

Noteer de uitvoer die de computer volgens jou zal geven. Controleer door de code te plakken in een IDE en het programma uit te voeren.

b. Analyseer nu het volgende programma. Zie je het subtiele verschil op lijn 4? Voer het niet uit in je IDE.

```
1 getal = 21
2 if getal % 3 == 0:
3     print(getal, 'is deelbaar door 3.')
4 if getal % 7 == 0:
5     print(getal, 'is deelbaar door 7.')
```

Noteer de uitvoer die de computer volgens jou zal geven. Controleer door de code aan te passen in een IDE en het programma uit te voeren.

In de vorige opgaves werd een `if`-statement altijd gevolgd door een test (`if getal_1 > getal_2, if getal % 3 == 0, ...`). Dat hoeft niet altijd het geval te zijn. Je kunt ook het resultaat van de test opslaan in een variabele met het datatype boolean. In de `if ...`-constructie kan je dan op een elegante manier verwijzen naar deze variabele.

6

Je vindt dit programma in Dodona. Analyseer de code.

5. VOORWAARDELIJKE UITVOERING

```
1 # lees een natuurlijk getal in
2 getal = int(input('Geef een natuurlijk getal: '))
3 # controleer de deelbaarheid van dit getal door 3 en door 7
4 getal_deelbaar_door_3 = getal % 3 == 0
5 getal_deelbaar_door_7 = getal % 7 == 0
6 # als een getal deelbaar is door 3 en door 7, is het deelbaar door 21
7 if getal_deelbaar_door_3 and getal_deelbaar_door_7:
8     print(getal, 'is deelbaar door 21.')
9 elif getal_deelbaar_door_3:
10    print(getal, 'is deelbaar door 3.')
11 elif getal_deelbaar_door_7:
12    print(getal, 'is deelbaar door 7.')
13 else:
14    print(getal, 'is niet deelbaar door 3 en 7.')
```

Noteer de uitvoer die de computer volgens jou zal geven. Controleer door de code te plakken in een IDE en het programma uit te voeren.

```
Geef een natuurlijk getal: 18
```

```
Geef een natuurlijk getal: 147
```

```
Geef een natuurlijk getal: 34
```

5.5 Opdrachten

Reeks 1

- 7 Schrijf een programma dat achtereenvolgens de lengte (in m) en de massa (in kg) van de gebruiker vraagt. Het berekent de BMI van de gebruiker en interpreteert deze waarde aan de hand van de tabel. Het programma toont op één lijn de BMI (afgerond op 1 cijfer na de komma) en de interpretatie.

BMI	interpretatie
$BMI < 18.5$	ondergewicht
$18.5 \leq BMI < 25$	gezond gewicht
$25 \leq BMI < 30$	overgewicht
$BMI \geq 30$	obesitas

- 8 Op het einde van de opleiding aan een universiteit of hogeschool wordt aan elke student een graad toegekend. Die graad wordt bepaald in functie van het gemiddelde van alle resultaten die de student behaald heeft tijdens de opleiding. De tabel geeft aan hoe het gemiddelde omgezet zou kunnen worden naar een graad.

gemiddelde	graad	afkorting
minder dan 68%	voldoening	V
minstens 68%	onderscheiding	O
minstens 77%	grote onderscheiding	GO
minstens 85%	grootste onderscheiding	GGO
minstens 90%	grootste onderscheiding met felicitaties	F

- a. Je krijgt een programma om een gemiddelde in te lezen en om te zetten naar de afkorting van een graad. Het programma geeft niet altijd het juiste resultaat. Zoek de redeneringsfout in het programma.

```

1 # lees de gemiddelde score in
2 gemiddelde = float(input('Geef de gemiddelde score: '))
3
4 # bepaal de graad
5 if gemiddelde < 68:
6     graad = 'V'
7 elif gemiddelde >= 68:
8     graad = 'O'

```

5. VOORWAARDELIJKE UITVOERING

```
9 elif gemiddelde >= 77:
10     graad = 'GO'
11 elif gemiddelde >= 85:
12     graad = 'GGO'
13 elif gemiddelde >= 90:
14     graad = 'F'
15
16 # schrijf de graad uit
17 print(graad)
```

b. Verbeter het programma. Test je code in Dodona.

9 Schrijf een programma dat vraagt om twee natuurlijke getallen in te voeren. Het programma berekent en toont of het grootste getal een veelvoud van het kleinste getal is. Schrijf de uitvoer in de vorm **15 is een veelvoud van 3** en **17 is geen veelvoud van 4**.

10 Een vervoersmaatschappij biedt aan jonge reizigers twee soorten tickets aan. Een jongerenticket kost altijd 7 euro. De prijs van een standaard ticket hangt af van de afstand volgens de formule

$$\text{prijs} = \text{€}1,50 + \text{€}0,15/\text{km}.$$

Schrijf een programma dat een afstand (in km) vraagt. Het programma berekent en toont het voordeligste ticket (**jongerenticket** of **standaard ticket**) voor deze afstand. Op een tweede lijn toont het de prijs van het ticket. Test je programma in Dodona.

Reeks 2

11 Als je in het buitenland geld wilt afhalen aan een bankautomaat, rekent de bank soms een kost aan die bovenop het bedrag komt dat je afhaalt. Je hebt drie bankkaarten op zak.

- Met bankkaart A rekent de bank een vaste kost van €2 aan plus 2% van het afgehaalde bedrag.
- Met bankkaart B rekent de bank een kost aan van 5% van het afgehaalde bedrag.
- Met bankkaart C rekent de bank een vaste kost van €5 aan.

Schrijf een programma dat vraagt om een bedrag in te voeren. Dit bedrag is een natuurlijk getal. Het programma berekent en toont de voordeligste bankkaart (**A**, **B** of **C**) om dit bedrag af te halen. Op een tweede lijn toont het het totale bedrag dat van je rekening zal afgehouden worden. Test je programma in Dodona.

12 Schrijf een programma dat twee keer vraagt om een getal in te voeren. Daarna vraagt het om een bewerking in te voeren (+, −, *, /, **). Het programma berekent en toont het resultaat van de bewerking.[†] Test je programma in Dodona.

13 Een nieuwe supermarkt probeert klanten te winnen door grote kortingen aan te bieden:

- Klanten die voor minstens 50 euro kopen, krijgen 2% korting op het volledige aankoopbedrag.
- Klanten die voor minstens 100 euro kopen, krijgen 3% korting op het volledige aankoopbedrag.
- Klanten die voor minstens 150 euro kopen, krijgen 4% korting op het volledige aankoopbedrag.
- Klanten die voor minstens 200 euro kopen, krijgen 5% korting op het volledige aankoopbedrag.

Schrijf een programma dat vraagt voor hoeveel euro de klant gekocht heeft. Het programma berekent de korting en het te betalen bedrag. Rond beide bedragen af op €0,01. Toon op één lijn het te betalen bedrag en de korting. Test je programma in Dodona.

14 Een wachtwoord wordt veilig genoemd als het aan de volgende criteria voldoet:

- Het wachtwoord telt minstens 12 karakters.[‡]
- Het wachtwoord bevat minstens één cijfer.
- Het wachtwoord bevat minstens één van de volgende drie karakters: %, !, ?.

Schrijf een programma dat aan de gebruiker een wachtwoord vraagt. Het programma geeft als uitvoer **veilig** als het aan elk van deze drie voorwaarden voldoet. Als er aan minstens één voorwaarde niet voldaan is, geeft het programma als uitvoer **onveilig**. Test je code in Dodona.

[†]Deze methode lijkt misschien ver gezocht, maar je kunt verbazend efficiënt werken op deze manier. Er bestaan dan ook wetenschappelijke rekenmachines die deze zogenaamde Reverse Polish Notation gebruiken.

[‡]Je kunt hiervoor de functie **len()** gebruiken die in paragraaf 1.3 werd geïntroduceerd.

5. VOORWAARDELIJKE UITVOERING

15 Schrijf een programma dat aan de gebruiker een stuk tekst vraagt. Het programma geeft als uitvoer het aantal **verschillende** klinkers in de tekst. Test je code in Dodona.

16 De Aarde draait rond de Zon in 365 dagen, 5 uren, 48 minuten en 45 seconden. Onze kalender telt 365 dagen. Om rekening te houden met de bijkomende 5 uren, 48 minuten en 45 seconden, werd het systeem van schrikkeljaren bedacht. In een schrikkeljaar heeft de maand februari geen 28 maar 29 dagen. Een schrikkeljaar telt dus 366 dagen.

Men spreekt van een schrikkeljaar als het jaartal wel deelbaar is door 4, maar niet door 100. Als het jaartal deelbaar is door 400, is het wel een schrikkeljaar.

a. Gebruik de definitie om te bepalen of het gegeven jaartal een schrikkeljaar is.

- 1900
- 2000
- 2020
- 2021

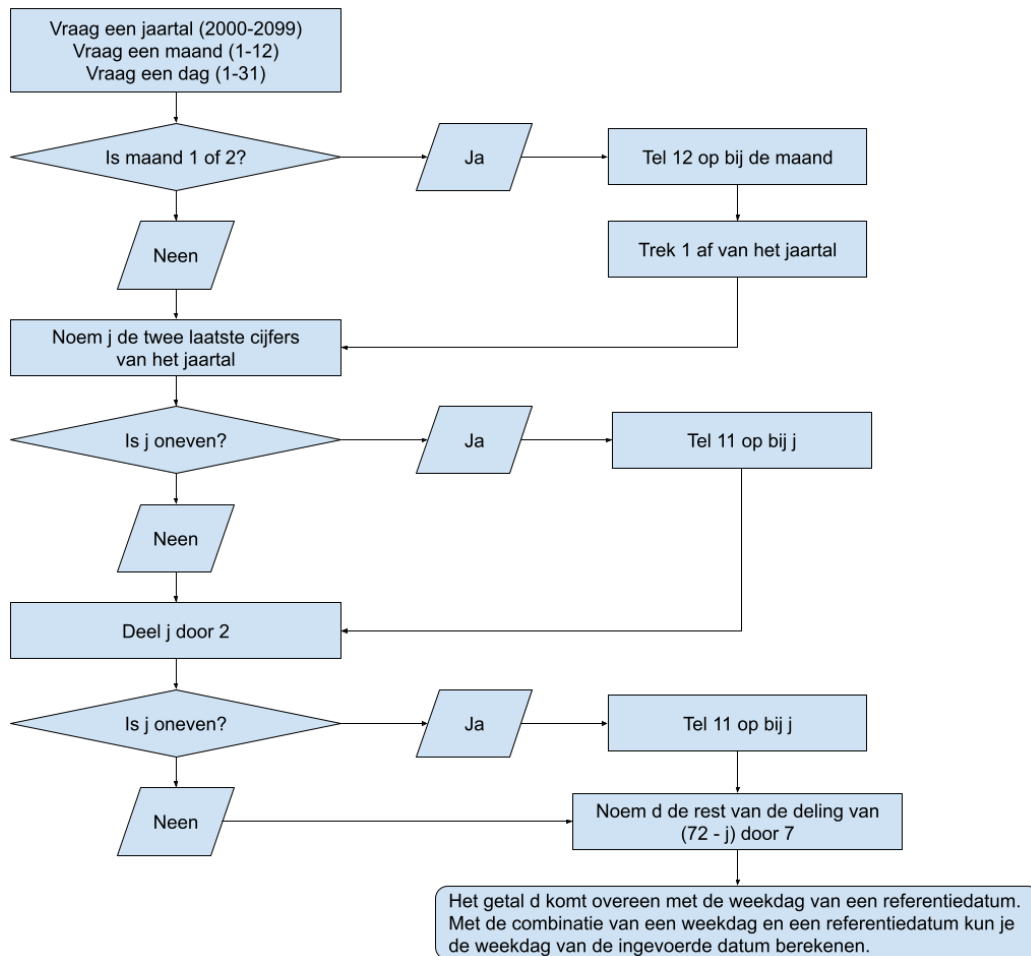
b. Schrijf een programma dat aan de gebruiker een jaartal vraagt. Het programma toont of het ingevoerde jaartal een schrikkeljaar is of niet. De uitvoer van het programma is in het formaat **1600 is een schrikkeljaar.** of **1789 is geen schrikkeljaar..** Test je code in Dodona.

Reeks 3

17 Schrijf een programma dat aan de gebruiker eerst de naam van een maand en dan een jaartal vraagt. Je programma toont het aantal dagen van die maand in dat jaar. De uitvoer is van de vorm **februari 2020 heeft 29 dagen..** Test je code in Dodona.

18 De Britse wiskundige John Conway (1937 – 2020) heeft een methode bedacht om uit het hoofd te berekenen met welke weekdag een willekeurige datum overeenkomt. In deze opgave wordt gewerkt met een vereenvoudigd algoritme dat geldig is van 1 maart 2000 tot en met 28 februari 2100. Het algoritme bestaat uit twee stappen.

- In een eerste stap bereken je een zogenaamde **referentieweekdag**. Het algoritme hiervoor staat schematisch weergegeven in een beslissingsdiagram.



Het resultaat van dit algoritme is een getal d tussen 0 en 6. $d=0$ wil zeggen dat de referentieweekdag in het ingevoerde jaar een zondag is. Bij $d=1$ valt de referentieweekdag in dat jaar op een maandag, ... $d=6$ wil zeggen dat de referentieweekdag in dat jaar op een zaterdag valt.

- De referentieweekdag die je in het eerste deel hebt berekend, is meestal niet de weekdag van de ingevoerde datum. Voor een willekeurig jaar komt de referentieweekdag wel overeen met de **referentiedatums** die in de tabel gegeven zijn.

Voorbeeld 1: Voor het jaar 2002 vind je $d=4$. Controleer dit. De referentieweekdag in 2002 is dus een donderdag. De referentiedatums 7 maart 2002, 4 april 2002, 2 mei 2002, ..., 7 november 2002 en 5 december 2002 vallen daarom allemaal op een donderdag. Bovendien zullen ook 2 januari 2003 en 6 februari 2003 op een donderdag vallen.

Op basis van deze tabel kan je dan in een tweede stap de weekdag van de ingegeven

5. VOORWAARDELIJKE UITVOERING

maand	referentiedag
maart	7
april	4
mei	2
juni	6
juli	4
augustus	1
september	5
oktober	3
november	7
december	5
januari	2
februari	6

datum bepalen. Stel dat je wilt weten op welke weekdag 23 maart 2002 viel. Je weet op basis van het voorgaande dat de referentiedatum 7 maart 2002 een donderdag was. Ook 14 en 21 maart 2002 waren dus donderdagen. 22 maart 2002 was een vrijdag, en 23 maart 2002 ten slotte was een zaterdag.

- Gebruik dit algoritme om handmatig te bepalen op welke dag je geboren bent.
- Implementeer dit algoritme in Python. Het programma vraagt eerst het jaartal (een natuurlijk getal tussen 2000 en 2099). Daarna vraagt het de maand (een natuurlijk getal tussen 1 en 12). Ten slotte vraagt het de dag (een natuurlijk getal tussen 1 en 31). Het programma berekent de weekdag voor de ingevoerde datum. De uitvoer van het programma is in het formaat **dinsdag 9 1 2007**. Test je code in Dodona.

6.1 Instap

- 1 a. Schrijf een programma dat drie keer **Python** toont, telkens op een nieuwe lijn. Test je code in Dodona.
- b. Je vindt dit programma in Dodona.

```
1 n = int(input('Geef het aantal herhalingen: '))
2 for i in range(n):
3     print('Python')
```

Kopieer en plak het programma in je IDE. Voer het een paar keer uit met verschillende waarden van **n**. Noteer hier wat het programma precies doet:

- 2 Je vindt dit programma in Dodona.

```
1 n = int(input('Geef een natuurlijk getal: '))
2 for i in range(n):
3     print(i)
```

- a. Kopieer en plak het programma in je IDE. Voer het een paar keer uit met verschillende waarden van **n**. Noteer hier wat het programma precies doet:
- b. Op lijn 2 wordt de variabele **i** geïntroduceerd. **i** wordt gedefinieerd op een manier die je nog niet eerder hebt gezien. Welke waarden neemt deze variabele aan?

6.2 Theorie

`range()`

Met de functie `range()` genereer je een rij getallen. Je kunt `range()` op meer dan één manier gebruiken:

- `range(n)` genereert een rij van `n` getallen: `0, 1, 2, ..., n-1`. Daarbij moet `n` een natuurlijk getal zijn.

Je zou misschien verwachten dat het eerste element in de rij `range(n)` gelijk is aan 1 en dat het laatste element gelijk is aan `n`. Let op: dit is niet het geval. **Het eerste element in de rij is gelijk aan 0 en het laatste element in `range(n)` is gelijk aan `n-1`.** Misschien vind je dat wat vreemd, maar gevorderde computerwetenschappers vinden dit heel normaal.

Voorbeeld 1: `range(5)` komt overeen met de rij `0, 1, 2, 3, 4`.

- `range(start, stop)` genereert een rij getallen die begint bij `start` en eindigt **net vóór** `stop`. De stapgrootte is gelijk aan 1. `range(start, stop)` komt dus overeen met de rij `start, start + 1, start + 2, ..., stop - 1`.

Python verwacht dat zowel `start` als `stop` van het datatype `integer` zijn. `start` en `stop` moeten dus gehele getallen zijn.

Voorbeeld 2: `range(5, 10)` komt overeen met de rij `5, 6, 7, 8, 9`.

- `range(start, stop, stap)` genereert een rij getallen die begint bij `start` en eindigt **net vóór** `stop`. De parameter `stap` legt de stapgrootte vast.

Python verwacht dat zowel `start`, `stop` als `stap` van het datatype `integer` zijn.

Voorbeeld 3: `range(0, 10, 2)` komt overeen met de rij `0, 2, 4, 6, 8`.

Voorbeeld 4: `range(99, 0, -1)` komt overeen met de rij `99, 98, ..., 3, 2, 1`.

`for ... in ...`

Als je precies weet hoe vaak je een bepaald stukje code wilt herhalen, kan je gebruik maken van een `for ... in ...`-lus. Het herhaaldelijk uitvoeren van een stuk code wordt **itereren** genoemd.

In pseudocode ziet de algemene syntax van een `for ... in ...`-lus er als volgt uit:

```
1 for item in collectie:
2     # dit blokje code wordt uitgevoerd voor elk item in collectie
3     # ...
4 # vanaf hier komt het verdere verloop van het programma
```

Wanneer `item` geen echte betekenis heeft, wordt traditioneel vaak de variabele `i` gebruikt. Voor Python maakt de naam van `item` geen enkel verschil. Wanneer `item` wel een betekenis heeft voor degene die het programma schrijft, kan het nuttig zijn om hier een zinvolle naam voor te kiezen.

Merk op dat er een dubbele punt staat op het einde van lijn 1 en dat lijnen 2-3 ingesprongen zijn. Je herkent deze syntax van bij voorwaardelijke lussen. De ingesprongen lijnen worden uitgevoerd voor elke waarde van `item` in `collectie`. Wanneer dat gebeurd is, gaat het programma verder op lijn 4.

De begrippen `item` en `collectie` zijn nogal vaag. Je kunt ze op verschillende manieren concreet invullen:

- `collectie` kan een rij getallen zijn die gegenereerd wordt door een `range()`-functie. `item` neemt dan, getal per getal, alle waarden aan van die rij. Als je een stukje code precies `n` keer wil herhalen, kan je dat gemakkelijk doen met de constructie `for i in range(n):`.

Voorbeeld 5:

```
1 for i in range(3):  
2     print('Python')
```

```
Python  
Python  
Python
```

- `collectie` kan ook een rij getallen zijn die je zelf definieert:

Voorbeeld 6:

```
1 for getal in (10, 100, 1000):  
2     print(getal)
```

```
10  
100  
1000
```

- `collectie` kan ook van het type `string` zijn. `item` neemt achtereenvolgens alle karakters aan van die `string`:

Voorbeeld 7:

```
1 for karakter in 'A7? Mis!':  
2     print(karakter)
```

```
A  
7  
?  
  
M  
i  
s  
!
```

- **collectie** kan ook een variabele zijn. Die variabele kan op zijn beurt een rij getallen of een **string** zijn:

```
1 woord = 'hond'  
2 for letter in woord:  
3     print(letter)
```

```
h  
o  
n  
d
```


6.3 Overzicht

In dit hoofdstuk heb je kennism gemaakt met begrensde herhaling. Hieronder vind je een overzicht van de belangrijkste begrippen en constructies.

`range()`

Met de functie `range()` genereer je een rij getallen. Je kunt `range()` op drie manieren gebruiken.

- `range(n)` genereert een rij van `n` getallen: `0, 1, 2, ..., n-1`. Let op: het eerste element is gelijk aan `0`, niet aan `1`, en het laatste element is gelijk aan `n-1`, niet aan `n`.

Voorbeeld 1: `range(5)` komt overeen met de rij `0, 1, 2, 3, 4`.

- `range(start, stop)` genereert een rij getallen die begint bij `start` en eindigt net vóór `stop`, met een stapgrootte van `1`.

Voorbeeld 2: `range(5, 10)` komt overeen met de rij `5, 6, 7, 8, 9`.

- `range(start, stop, stap)` genereert een rij getallen die begint bij `start` en eindigt net vóór `stop`, met een stapgrootte gelijk aan `stap`.

Voorbeeld 3: `range(0, 10, 2)` komt overeen met de rij `0, 2, 4, 6, 8`.

Voorbeeld 4: `range(99, 0, -1)` komt overeen met de rij `99, 98, ..., 3, 2, 1`.

Bij alle drie de vormen verwacht Python dat `n`, `start`, `stop` en `stap` gehele getallen zijn.

`for ... in ...`

Weet je precies hoe vaak je een stukje code wilt herhalen, dan gebruik je een `for ... in ...`-lus. Het herhaaldelijk uitvoeren van een stuk code wordt **itereren** genoemd.

In pseudocode ziet de algemene syntax er zo uit:

```
1 for item in collectie:
2     # dit blokje code wordt uitgevoerd voor elk item in collectie
3     # ...
4 # vanaf hier komt het verdere verloop van het programma
```

6. BEGRENSEDE HERHALING

Net als bij voorwaardelijke lussen staat er een dubbele punt op het einde van de eerste lijn, en is de bijhorende code ingesprongen. Voor elke waarde van `item` in `collectie` wordt de ingesprongen code uitgevoerd. Zodra alle waarden van `collectie` doorlopen zijn, gaat het programma verder na de lus.

`collectie` kan op verschillende manieren ingevuld worden:

- een rij getallen, gegenereerd door `range()`. Wil je een stukje code precies `n` keer herhalen, dan gebruik je `for i in range(n):`.
- een rij getallen die je zelf definieert, bijvoorbeeld `for getal in (10, 100, 1000):`.
- een string. `item` neemt dan achtereenvolgens elk karakter van die string aan.
- een variabele, die op haar beurt een rij getallen of een string kan bevatten.

Wanneer `item` geen echte betekenis heeft, wordt traditioneel de variabele `i` gebruikt. Voor Python maakt de naam van `item` verder geen enkel verschil, maar een zinvolle naam kan je code wel leesbaarder maken.

6.4 Verwerking

- 3** Een programma vraagt een natuurlijk getal `n`. De uitvoer is de som van alle getallen strikt kleiner dan `n` die deelbaar zijn door 3 of 5.

a. Je krijgt een aantal Python-uitdrukkingen. Rangschik ze in een juiste volgorde. Besteed aandacht aan het inspringen van sommige stukken code.

- (a) `for i in range(n):`
- (b) `som = som + i`
- (c) `som = 0`
- (d) `print(som)`
- (e) `if i % 3 == 0 or i % 5 == 0:`
- (f) `n = int(input('Geef een natuurlijk getal: '))`

b. Breng je code over naar je IDE. Test je code in Dodona.

- 4** Je vindt dit programma in Dodona. Analyseer de code. Je mag ze kopiëren en plakken in een IDE en daar uitvoeren.

```

1 # laat i toenemen van 1 t.e.m. 10
2 for i in range(1, 11):
3     # definieer uitvoer als een lege string
4     uitvoer = ''
5     # laat ook j toenemen van 1 t.e.m. 10
6     for j in range(1, 11):
7         # bereken het product van i en j
8         product = i*j
9         # plak het berekende product en een spatie aan de bestaande
        uitvoer
10     uitvoer = uitvoer + str(product) + ' '
11     # toon de variabele uitvoer
12     print(uitvoer)
```

a. Hoe vaak wordt lijn 4 uitgevoerd?

b. Hoe vaak wordt lijn 8 uitgevoerd?

c. Hoe vaak wordt lijn 12 uitgevoerd?

6.5 Opdrachten

Reeks 1

- 5** Schrijf een programma dat de waarde van een natuurlijk getal `n` vraagt. Het programma toont de eerste 10 lijnen van de tafel van vermenigvuldiging van `n`. Elke lijn is van de vorm `7 * 12 = 84`. Test je code in Dodona.
- 6** Schrijf een programma dat een natuurlijk getal `n` vraagt aan de gebruiker. Het programma berekent de eerste `n` oneven getallen en toont deze telkens op een nieuwe lijn. Test je code in Dodona.
- 7** Schrijf een programma dat aan de gebruiker een natuurlijk getal vraagt. Je programma genereert als uitvoer de som van alle natuurlijke getallen kleiner dan of gelijk aan dit getal. Test je code in Dodona.
- 8** Schrijf een programma dat aan de gebruiker een woord vraagt met uitsluitend kleine letters. Je programma toont vervolgens het aantal klinkers en het aantal medeklinkers in de vorm `zwemmen heeft 2 klinkers en 5 medeklinkers`. Test je code in Dodona.
- 9** Schrijf een programma dat vraagt naar de waarde van een natuurlijk getal. Het programma gaat na of dit getal een priemgetal is. Schrijf de uitvoer in de vorm `11 is een priemgetal.` en `12 is geen priemgetal..` Test je code in Dodona.

Reeks 2

- 10** Schrijf een programma dat 10 keer vraagt naar de waarde van een natuurlijk getal. Als uitvoer toont het programma het grootste van deze 10 getallen, het kleinste van de 10 getallen en het aantal veelvouden van 3. Test je code in Dodona.
- 11** De rij van Fibonacci is een rij getallen die als volgt wordt opgebouwd.
- De eerste en de tweede term van de rij zijn gelijk aan 1.
 - Elke volgende term is gelijk aan de som van de twee voorgaande termen.

Schrijf een programma dat aan de gebruiker een natuurlijk getal n vraagt (met $n \geq 2$). Je programma genereert als uitvoer de n eerste termen uit de rij van Fibonacci, telkens op een nieuwe lijn. Test je code in Dodona.

- 12** Een heel langzame manier om π te benaderen, is door de som te nemen van een groot aantal termen:

$$\pi \approx 4 \cdot \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots\right)$$

Vraag aan de gebruiker een natuurlijk getal n . Kies voor n een natuurlijk getal tussen 100 000 en 10 000 000. De uitvoer is de benadering van π die je bekomt door de eerste n termen van deze som op te tellen. Test je code in Dodona.

- 13** Schrijf een programma dat vraagt om twee woorden. Als er gemeenschappelijke letters zijn in deze twee woorden, toon je uitvoer in de vorm **De woorden algoritme en Python hebben volgende letter(s) gemeenschappelijk: ot**. Als er geen enkele letter gemeenschappelijk is, toon je uitvoer in de vorm **De woorden boek en plant hebben geen enkele letter gemeenschappelijk..**

Je mag hoofdletters beschouwen als verschillend van kleine letters, maar iedere letter mag niet meer dan één keer gerapporteerd worden. Zo hebben de woorden **peer** en **een** bijvoorbeeld slechts één letter gemeen, namelijk de letter **e**. Test je code in Dodona.

Reeks 3

- 14** Schrijf een programma dat aan de gebruiker een natuurlijk getal vraagt. Je programma genereert als uitvoer de som van de cijfers van dat getal. Test je code in Dodona.
- 15** Vraag aan de gebruiker een natuurlijk getal n . Beschouw dan de n eerste termen van de rij van Fibonacci. De uitvoer is gelijk aan de som van alle termen die deelbaar zijn door 2. Test je code in Dodona.
- 16** Gebruik het algoritme van Conway (Hoofdstuk 5) om te bepalen hoe vaak Kerstmis op een maandag, dinsdag, ..., zondag valt tussen 2000 en 2099. Geef de uitvoer in de vorm **Aantal keer Kerstmis op maandag: 14**. Test je code in Dodona.

7.1 Instap

- 1 Je vindt dit programma in Dodona.

```
1 # lees een getal in
2 getal = int(input('Typ een oneven getal: '))
3 # voer een blokje code uit zolang het ingevoerde getal even is
4 while getal % 2 == 0:
5     print(getal, 'is geen oneven getal. Probeer opnieuw.')
6     getal = int(input('Typ een oneven getal: '))
7 # het ingevoerde getal is niet even
8 print(getal, 'is oneven.')
```

Kopieer en plak het programma in je IDE. Voer het programma uit. Noteer wat het programma precies doet. Wat gebeurt er wanneer je toch een even getal intypt?

7.2 Theorie

while ...

Wanneer je op voorhand niet weet hoe vaak je een bepaald stukje code wilt herhalen, is het geen goed idee om een **for ... in ...**-constructie te gebruiken. Je kunt dan beter gebruik maken van een lus die zichzelf herhaalt **zolang aan een voorwaarde voldaan is**. Dat is precies wat de **while ...**-lus doet in Python. In pseudocode ziet die er zo uit:

```
1 # controleer of voldaan is aan de Booleaanse variabele voorwaarde
2 while voorwaarde:
3     # zolang voldaan is aan voorwaarde, wordt dit blokje code
    uitgevoerd
4     # ...
5 # zodra er niet voldaan is aan voorwaarde, gaat het programma hier
    verder
```

break

Bij een **while** ...-lus loop je het risico dat de lus nooit beëindigd wordt. Als **voorwaarde** op lijn 2 in de pseudocode nooit **False** wordt, blijft de computer de lus uitvoeren. De enige manier om de lus te beëindigen, is door het hele programma te beëindigen.

Om dat te vermijden, kan je een **break** introduceren. Het **break**-statement zorgt ervoor dat de lus waartoe de **break** behoort, onmiddellijk afgebroken wordt. Het programma gaat dan verder op de eerste lijn na de lus. Kijk naar de pseudocode voor meer uitleg:

```
1 while voorwaarde_1:
2     # zolang voldaan is aan voorwaarde_1, wordt dit blokje code
    uitgevoerd
3     # ...
4     # controleer of voldaan is aan voorwaarde_2
5     if voorwaarde_2:
6         # als voldaan is aan voorwaarde_2, zorgt de break ervoor dat de
        while-lus meteen afgebroken wordt
7         break
8     else:
9         # deze regels worden uitgevoerd zolang voorwaarde_1 True en
        voorwaarde_2 False is
10        # ...
11 # zodra voorwaarde_1 False is of zodra voorwaarde_2 True is, gaat het
    programma verder vanaf hier
```


7.3 Overzicht

In dit hoofdstuk heb je kennisgemaakt met voorwaardelijke herhaling. Hieronder vind je een overzicht van de belangrijkste begrippen en constructies.

while ...

Weet je op voorhand niet hoe vaak je een stukje code wilt herhalen, dan is een **for ... in ...**-constructie geen goed idee. Je gebruikt dan beter een lus die zichzelf herhaalt **zolang aan een voorwaarde voldaan is**: de **while ...**-lus.

In pseudocode ziet die er zo uit:

```
1 # controleer of voldaan is aan de Booleaanse variabele voorwaarde
2 while voorwaarde:
3     # zolang voldaan is aan voorwaarde, wordt dit blokje code
    uitgevoerd
4     # ...
5 # zodra er niet voldaan is aan voorwaarde, gaat het programma hier
    verder
```

Zolang **voorwaarde True** is, blijft het ingesprongen blokje code uitgevoerd worden. Zodra **voorwaarde False** wordt, gaat het programma verder na de lus.

break

Bij een **while ...**-lus loop je het risico dat **voorwaarde** nooit **False** wordt, waardoor de lus nooit eindigt. De enige manier om zo'n lus dan nog te stoppen, is door het hele programma af te breken.

Om dat te vermijden, kan je een **break** inbouwen. Het **break**-statement zorgt ervoor dat de lus waartoe het behoort, onmiddellijk afgebroken wordt. Het programma gaat dan meteen verder op de eerste lijn na de lus.

In pseudocode ziet dat er zo uit:

```
1 while voorwaarde_1:
2     # zolang voldaan is aan voorwaarde_1, wordt dit blokje code
    uitgevoerd
3     # ...
4     # controleer of voldaan is aan voorwaarde_2
5     if voorwaarde_2:
6         # als voldaan is aan voorwaarde_2, zorgt de break ervoor dat de
        while-lus meteen afgebroken wordt
7         break
```

```
8     else:
9         # deze regels worden uitgevoerd zolang voorwaarde_1 True en
          voorwaarde_2 False is
10        # ...
11 # zodra voorwaarde_1 False is of zodra voorwaarde_2 True is, gaat het
    programma verder vanaf hier
```

Een **break** zit dus typisch verscholen in een **if**-statement binnen een **while**-lus: pas wanneer aan een bepaalde voorwaarde voldaan is, moet de lus effectief afgebroken worden.

for ... in ... versus while ...

Je kent nu twee manieren om een stukje code te herhalen. De keuze tussen beide hangt af van één simpele vraag: weet je op voorhand hoe vaak de lus moet uitgevoerd worden?

- Weet je dat op voorhand wel, dan gebruik je een **for ... in ...**-lus. Dat is bijvoorbeeld het geval wanneer je een stukje code precies **n** keer wilt herhalen, of wanneer je elk karakter van een string of elk element van een rij getallen wilt overlopen.
- Weet je dat op voorhand niet, dan gebruik je een **while ...**-lus. Dat is bijvoorbeeld het geval wanneer je blijft vragen om een geldige invoer, zolang de gebruiker geen geldig getal intypt: je weet vooraf niet hoe vaak de gebruiker een fout zal intypen.

Merk op dat je een **for ... in ...**-lus in principe altijd zou kunnen herschrijven als een **while ...**-lus, maar niet omgekeerd. Een **while ...**-lus is dus algemener inzetbaar, maar een **for ... in ...**-lus is vaak korter en duidelijker te lezen wanneer je op voorhand weet hoe vaak je moet herhalen. Analyseer daarom de opgave altijd goed, en kies de constructie die het best aansluit bij wat je precies wilt bereiken.

7.4 Verwerking

- 2 Je vindt dit programma in Dodona.

```

1 # definieer twee numerieke variabelen
2 som = 0
3 aantal = 0
4 # herhaal voor altijd
5 while True:
6     # lees de invoer in, die ofwel een getal kan zijn, ofwel de string
    STOP
7     invoer = input('Geef een getal of typ STOP: ')
8     if invoer == 'STOP':
9         # als de gebruiker STOP heeft getypt, onderbreek je
    onmiddellijk de while-lus
10        # het programma gaat verder op lijn 18
11        break
12    else:
13        # de gebruiker heeft een getal getypt
14        # verhoog de waarde van de variabele som met het ingevoerde
    getal
15        som = som + float(invoer)
16        # verhoog de waarde van de variabele som met 1
17        aantal = aantal + 1
18 # bereken en toon het gemiddelde van de ingevoerde getallen
19 gemiddelde = som / aantal
20 print('Het gemiddelde van deze getallen is', gemiddelde)

```

Kopieer en plak het programma in je IDE. Voer het programma uit. Noteer wat het programma precies doet.

- 3 Een programma vraagt een natuurlijk getal aan de gebruiker. De bedoeling is om dit getal te reduceren naar één cijfer. Dat doe je door de som te nemen van alle cijfers van dit getal. Als die som op zijn beurt bestaat uit meer dan één cijfer, blijf je verder werken tot je uiteindelijk één enkel cijfer bekomt. Test je code in Dodona.

Voorbeeld 1: 87 wordt gereduceerd naar 6, want $8 + 7 = 15$ en $1 + 5 = 6$.

- Je krijgt een aantal Python-uitdrukkingen. Rangschik ze in de juiste volgorde. Besteed aandacht aan het inspringen van sommige stukken code.

7. VOORWAARDELIJKE HERHALING

- (a) `som = som + int(getal[i])`
- (b) `while int(getal) > 9:`
- (c) `for i in range(len(getal)):`
- (d) `getal = str(som)`
- (e) `getal = input('Geef een natuurlijk getal: ')`
- (f) `print(som)`
- (g) `som = 0`

b. Breng je code over naar je IDE. Test je code in Dodona.

7.5 Opdrachten

Reeks 1

4 In de eerste graad heb je een algoritme geleerd om een getal te ontbinden in priemfactoren.

- a. Pas dit algoritme toe om 1320 te ontbinden in factoren.
- b. Schrijf een programma dat aan de gebruiker een natuurlijk getal vraagt. Het programma berekent en toont alle priemfactoren van dit getal, telkens op een nieuwe lijn. Als een factor n keer voorkomt in de ontbinding van het getal, wordt deze factor ook n keer getoond. Als het ingevoerde getal een priemgetal is, zegt het programma dit in de vorm **13 is een priemgetal..** Test je code in Dodona.

5 Schrijf een programma dat vraagt om natuurlijke getallen in te voeren. De invoer eindigt wanneer de gebruiker STOP typt. Het programma toont op één lijn het grootste ingevoerde getal, het kleinste ingevoerde getal en het aantal ingevoerde getallen. Test je code in Dodona.

Reeks 2

6 De oud-Griekse wiskundige Euclides heeft rond 300 v.C. een algoritme bedacht om de grootste gemene deler (ggd) van twee getallen te berekenen. Het algoritme steunt op het herhaaldelijk toepassen van de eigenschap:

De grootste gemene deler van **grootste_getal** en **kleinste_getal** is gelijk aan de grootste gemene deler van **kleinste_getal** en de rest bij deling van **grootste_getal** door **kleinste_getal**.

Voorbeeld 1:

$\text{ggd}(752, 372) = \text{ggd}(372, 8)$	want $752 = 2 \cdot 372 + 8$
$= \text{ggd}(8, 4)$	want $372 = 46 \cdot 8 + 4$
$= \text{ggd}(4, 0)$	want $8 = 2 \cdot 4 + 0$
$= 4$	want elk getal is een deler van nul

7. VOORWAARDELIJKE HERHALING

- a. Gebruik dit algoritme om de grootste gemene deler te bepalen van 1071 en 462.
- b. Schrijf een programma dat vraagt naar twee natuurlijke getallen. Het programma berekent en toont de grootste gemene deler van deze getallen. Test je code in Dodona.

- 7** Schrijf een programma dat een willekeurig getal genereert tussen 0 en 10. Laat de gebruiker een getal raden. Bij elke poging zegt het programma **Hoger** als het geheime getal groter is dan het ingevoerde getal. Het programma zegt **Lager** als het geheime getal kleiner is dan het ingevoerde getal. Wanneer het ingevoerde getal juist is, wordt ook het aantal pogingen getoond om het getal te raden.

Voor deze opgave kan je je programma niet testen met Dodona.

Om een willekeurig geheel getal te genereren in het interval `[a, b]`, kan je gebruik maken van deze code:

```
1 from random import randint
2 getal = randint(a,b)           # randint = random integer
```

Tip: bij het testen van je programma is het handig als je het getal kent dat de computer gegenereerd heeft. Zo verlies je niet te veel tijd: je kunt altijd zelf beslissen of je het juiste of een fout antwoord invoert. In de testfase kan je het geheime getal daarom best printen. Zet deze lijn in commentaar zodra je programma klaar is.

Reeks 3

- 8** Schrijf een programma dat de omtrek berekent van een willekeurige veelhoek* in het vlak. De x- en y-coördinaten van de opeenvolgende hoekpunten van de veelhoek worden één voor één ingelezen. De invoer wordt beëindigd zodra er STOP wordt ingevoerd in plaats van een x-coördinaat. Daarna wordt de omtrek berekend en getoond. Test je code in Dodona.

Let op: het gaat om een gesloten veelhoek. Het laatst ingevoerde hoekpunt wordt dus opnieuw verbonden met het eerste hoekpunt.

*Het zou kunnen dat sommige zijden van de veelhoek snijden met andere zijden. In dat geval wordt de veelhoek complex genoemd, maar deze blijft wel voldoen aan de definitie van een veelhoek.

- 9** Bij het ontwerpen van zijn Analytical Engine dacht Charles Babbage al na over mogelijke toepassingen van zijn machine. In zijn boek *The Ninth Bridgewater Treatise* (1837) stelde hij zich de volgende vraag:

Wat is het kleinste natuurlijke getal waarvan het kwadraat eindigt met de cijfers 269 696?

Schrijf een programma dat dit specifieke probleem van Babbage in een algemene vorm oplost. Vraag een natuurlijk getal aan de gebruiker. Het programma berekent en toont het kleinste getal waarvan het kwadraat eindigt op het gegeven getal. Toon de uitvoer in de vorm $25264 * 25264 = 638269696$. Test je code in Dodona.

Tot nu toe kwam met één variabele altijd één enkele waarde overeen. In heel wat situaties wil je meerdere waarden tegelijk opslaan:

- de namen van alle leerlingen in een klas;
- het resultaat van de laatste evaluatie van alle leerlingen die informatica volgen;
- een boodschappenlijst;
- de tien best verkochte boeken van het afgelopen jaar;
- ...

Strikt genomen zou je al deze gegevens in een string kunnen bewaren. Je zou bv. kunnen definiëren dat `klas_6A = 'Benjamin Anna Charles'`. Maar als Benjamin de klas verlaat, kan je dit element niet eenvoudig verwijderen uit de variabele `klas_6A`. Je zou de klaslijst ook niet alfabetisch kunnen sorteren, als dat nodig zou zijn. Met andere woorden: de datatypes die we tot nu toe kennen, volstaan niet voor een aantal situaties.

Om aan één enkele variabele meerdere waarden toe te kennen, hebben we een *datastructuur* nodig. Python kent verschillende datastructuren. In de volgende hoofdstukken van deze cursus bekijken we de *tuple*, de *verzameling* (Engels: *set*) en de *lijst* (Engels: *list*). Daarnaast is ook de *dictionary* een veelgebruikte datastructuur, maar daar gaan we in deze cursus niet dieper op in. Het is ten slotte ook mogelijk om je eigen datastructuur te definiëren, maar ook daar zullen we geen aandacht aan besteden.

Elke datastructuur heeft zijn eigen kenmerken en toepassingen. De keuze van de juiste datastructuur is in het algemeen erg belangrijk. Met name wanneer de efficiëntie van uitvoering van belang is, is het belangrijk om de juiste datastructuur te kiezen.

8.1 Instap

Kijk naar onderstaande code en de bijhorende uitvoer.

```
1 # definieer een variabele die de coördinaten van een punt in de ruimte
   vastlegt
2 # deze coördinaten horen samen
3 # we willen ze dan ook opslaan in een enkele variabele
4 # dat kan met een tuple:
5 P = (-4, 8, 1)
6
7 def afstand_ruimte_OP(punt):
8     # P is een tuple met drie elementen
9     # we kunnen de tuple als volgt "uitpakken":
10    x, y, z = punt
11    return (x**2 + y**2 + z**2) ** 0.5
12
13 print(afstand_ruimte_OP(P))
```

```
9.0
```

8.2 Theorie

Wat is een tuple?

In het Nederlands noemen we de coördinaatgetallen van een punt in het vlak (bv. $(-3, 2)$) een *koppel*. De coördinaatgetallen van een punt in de ruimte (bv. $(1, 2, -3)$) noemen we een *drietal*. $(1, 2, -3, 4)$ zouden we een *viertal* noemen. De coördinaten van een punt in een n -dimensionale ruimte noemen we in het algemeen een *n-tal*.

In het Engels noemen we deze structuren respectievelijk een *couple*, een *triple* en een *quadruple*. Je ziet dat deze termen eindigen op *-ple*. In het Engels noemen we een geordend n -tal daarom een *tu-ple*. In Python noteer je een tuple met ronde haakjes.

Een tuple is in het algemeen een geordende, onveranderlijke reeks van waarden. De elementen van een tuple hoeven geen getallen te zijn. `T = (1, 'A', (-5, 3))` is dus een tuple met drie elementen: een integer, een string en een tuple. Een waarde mag meer dan één keer voorkomen in een tuple. `P = (1, 1, 1)` is dus een geldige tuple.

Aantal elementen

Net zoals bij strings, geeft de functie `len()` het aantal elementen in een tuple.

```

1 regenboog = ('rood', 'oranje', 'geel', 'groen', 'blauw', 'indigo',
               'violet')
2 # bepaal het aantal elementen van de kleuren van de regenboog
3 print(len(regenboog))

```

```
7
```

Elementen selecteren via slicing

De ordening van de waarden in een tuple is belangrijk. Daardoor is het ook mogelijk om, net zoals bij een string, via slicing te verwijzen naar de afzonderlijke waarden:

```

1 print(regenboog[0])           # het "eerste" element van de tuple
2 print(regenboog[-1])          # het laatste element van de tuple
3 print(regenboog[2:4])          # de waarde met een index 2 tot vlak
                                voor index 4

```

```

rood
violet
('geel', 'groen')

```

Unpacking

Je kan de verschillende elementen van een tuple met n elementen gemakkelijk en elegant opslaan in n verschillende variabelen. Dit wordt *unpacking* genoemd. Kijk naar onderstaand voorbeeldje.

```

1 RGB = (26, 110, 161)
2 rood, blauw, groen = RGB
3 print(rood)

```

```
26
```

Een tuple is onveranderlijk

Eenmaal een tuple aangemaakt is, kan de inhoud nooit meer aangepast worden. Je kan geen elementen toevoegen of verwijderen. Je kan de waarde van een element in een tuple ook niet veranderen. Dit rigide gedrag lijkt misschien vervelend, maar deze onveranderlijkheid drukt uit dat de waarden bij elkaar horen, en dat ook moeten blijven doen. Als je de

8. TUPLES

mogelijkheid wil openhouden om de inhoud van een datastructuur te kunnen aanpassen, is een tuple dus geen goede keuze.

```
1 regenboog = ('rood', 'oranje', 'geel', 'groen', 'blauw', 'indigo', 'violet')
2 regenboog[0] = 'Rood'
```

```
TypeError: 'tuple' object does not support item assignment
```

Wanneer gebruik je een tuple?

Je gebruikt een tuple wanneer je een aantal waarden hebt die samen één logisch geheel vormen, waarvan de volgorde vastligt en die niet mogen veranderen. Tuples worden vaak gebruikt om data simpelweg door te geven, zonder ze te bewerken. Een concrete toepassing is wanneer een functie meerdere waarden berekent die je wilt uitvoeren. Je kan in je functie maar één **return** schrijven, maar je kan alle waarden wel verpakken in een tuple. Onderstaand stukje code illustreert dit:

```
1 def rechthoek_info(breedte, hoogte):
2     omtrek = 2 * (breedte + hoogte)
3     oppervlakte = breedte * hoogte
4     return (omtrek, oppervlakte) # we returnen een tuple met twee
    waarden
5
6 # gebruik unpacking om de omtrek en de oppervlakte te berekenen van een
    rechthoek met breedte 5 en hoogte 3
7 o, A = rechthoek_info(5, 3)
8
9 # print de resultaten
10 print('Omtrek:', o)
11 print('Oppervlakte:', A)
```

```
Omtrek: 16
Oppervlakte: 15
```

Wanneer gebruik je geen tuple?

Gebruik geen tuple wanneer:

- je elementen later nog wilt kunnen aanpassen;

- je elementen wilt kunnen toevoegen of verwijderen;
- de volgorde van de elementen betekenisloos is.

Je bent al lang vertrouwd met verzamelingen. Je kent de verzamelingen $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ en $\mathbb{C}$. Het lijkt misschien alsof de elementen van een verzameling altijd getallen moeten zijn, maar dat is niet het geval. Je zou perfect een verzameling kunnen definiëren met de namen van je broers en zussen. Een boodschappenlijstje zou je ook kunnen voorstellen in een verzameling.

9.1 Instap

Kijk naar onderstaande code en de bijhorende uitvoer.

```
1 # definieer een variabele die een boodschappenlijst voorstelt
2 # de volgorde van de elementen is onbelangrijk
3 # we kiezen dus voor een variabele van het type verzameling (set)
4 boodschappen = {'melk', 'brood', 'eieren', 'pasta', 'kaas'}
5 print(boodschappen)
```

Als je dit programma uitvoert, krijg je volgende uitvoer:

```
{'brood', 'kaas', 'pasta', 'eieren', 'melk'}
```

Je merkt dat de volgorde waarin de boodschappenlijst wordt weergegeven, niet dezelfde is als die waarin de boodschappen werden ingevoerd. Als je dit programma een tweede keer uitvoert, krijg je volgende uitvoer:

```
{'eieren', 'pasta', 'kaas', 'brood', 'melk'}
```

Vreemd genoeg geeft dit programma niet elke keer dezelfde uitvoer. De volgorde waarin de elementen worden weergegeven, is niet altijd dezelfde. Het illustreert dat de volgorde van de elementen totaal onbelangrijk is bij een verzameling.

9.2 Theorie

Wat is een verzameling?

Een verzameling is een datastructuur met *ongeordende* en *unieke* elementen.

- *Ongeordend* betekent dat er geen vaste volgorde is. Dat heeft als direct gevolg dat je elementen in een verzameling niet kunt opvragen via een index. In tegenstelling tot variabelen van het type string of tuple, kan je dus geen slicing gebruiken om een element te selecteren.
- *Uniek* betekent dat elk element maar één keer voorkomt. Waarden die meer dan één keer zouden voorkomen, worden automatisch verwijderd.

In Python worden verzamelingen genoteerd met accolades (`{}`). Je zou dan ook denken dat je een lege verzameling kunt definiëren als `lege_verzameling = {}`, maar dat is niet waar:

```
1 lege_verzameling = {}
2 print(type(lege_verzameling))
```

```
<class 'dict'>
```

Een `{}` maakt dus een object van het type dictionary, geen verzameling. Een lege verzameling maak je als volgt aan:

```
1 lege_verzameling = set()
2 print(type(lege_verzameling))
```

```
<class 'set'>
```

Elementen toevoegen en verwijderen

Je kunt eenvoudig elementen toevoegen aan een verzameling, of elementen verwijderen uit een verzameling.

- Om een element toe te voegen aan een bestaande verzameling, gebruik je `add()`. Onderstaande code illustreert het gebruik van `add()`.


```

1 # definieer een variabele die een boodschappenlijst voorstelt
2 # de volgorde van de elementen is onbelangrijk
3 # we kiezen dus voor een variabele van het type verzameling (set)
4 boodschappen = {'melk', 'brood', 'eieren', 'pasta', 'kaas'}
5 # ik wil ook appels kopen
6 boodschappen.add('appels')
7 print(boodschappen)

```

```
{'brood', 'kaas', 'pasta', 'eieren', 'appels', 'melk'}
```

- Om een element te verwijderen uit een bestaande verzameling, gebruik je `remove()`. Onderstaande code illustreert het gebruik van `remove()`.

```

1 # bij nader inzien is er nog genoeg pasta
2 # ik wil de pasta dus verwijderen uit mijn boodschappenlijst
3 boodschappen.remove('pasta')
4 print(boodschappen)

```

```
{'brood', 'kaas', 'eieren', 'appels', 'melk'}
```

Om een element toe te voegen aan een verzameling, gebruik je `add()`. De notatie `boodschappen.add('appels')` is echter nieuw. Hetzelfde geldt voor `remove()`.

`add()` en `remove()` zijn *geen functies*. Het zijn *methodes* die toegepast kunnen worden op objecten van het type set.

Om het verschil tussen functies en methodes te begrijpen, moet je weten dat Python een *objectgeoriënteerde* programmeertaal is. Alles in Python (integers, floats, strings, tuples, verzamelingen, lijsten, ...) wordt intern in feite bekeken als een *object*.

Een object is een stukje data dat ook *methodes* heeft: acties die specifiek bij dat type horen. Een *methode* is dus een soort functie die vastzit aan een object. Je roept een methode op *via* dat object, omdat ze iets doet *met* dat object. Een methode heeft dus geen uitvoer, omdat een methode rechtstreeks inwerkt *op* een object, en dat object dus kan beïnvloeden.

Een *functie* daarentegen staat los van een specifiek object. Je roept een functie op met haar naam, en je geeft zelf de parameters door waarmee ze moet werken. De functie heeft duidelijk wel een uitvoer.

Voorbeelden:

- `len()` is een functie: ze werkt op allerlei soorten objecten (strings, tuples, sets, lijsten, ...).

- `add()` is een methode van een set: je kunt ze alleen oproepen via een set-object, zoals `boodschappen.add('appels')`. Je kunt de methode `add()` dus niet toepassen op een lijst.

Het onderscheid tussen functies en methodes is misschien abstract en technisch, maar het volstaat dat je weet hoe je ze kunt gebruiken.

Aantal elementen

Net zoals bij strings en tuples, geeft de functie `len()` het aantal elementen in een verzameling.

```
1 # bepaal het aantal elementen van de boodschappenlijst
2 print(len(boodschappen))
```

```
5
```

Lidmaatschap controleren

Net zoals bij strings kan je de operator `in` gebruiken om te testen of een bepaald element al dan niet voorkomt in een verzameling:

```
1 # controleer of aardbeien voorkomen in deze verzameling
2 print('aardbeien' in boodschappen)
```

```
False
```

```
1 # controleer of eieren voorkomen in deze verzameling
2 print('eieren' in boodschappen)
```

```
True
```

Unie, doorsnede, verschil

Je kent de wiskundige begrippen unie, doorsnede en verschil van verzamelingen. Python kent operatoren om deze bewerkingen uit te voeren op verzamelingen:

- de operator `|` wordt gebruikt om de unie van verzamelingen ($A \cup B$) te bepalen;
- de operator `&` wordt gebruikt om de doorsnede van verzamelingen ($A \cap B$) te bepalen;

- de operator $-$ wordt gebruikt om het verschil van verzamelingen ($A \setminus B$) te bepalen.

Onderstaande code maakt duidelijk hoe je deze operatoren kunt gebruiken, en wat ze precies doen.

```
1 # we maken twee afzonderlijke boodschappenlijsten
2 boodschappen_1 = {'melk', 'brood', 'eieren', 'pasta', 'kaas'}
3 boodschappen_2 = {'wc-papier', 'melk', 'tandpasta', 'tomaten'}
4 # bepaal de unie van beide verzamelingen
5 print(boodschappen_1 | boodschappen_2)
```

```
{'eieren', 'tandpasta', 'wc-papier', 'melk', 'kaas', 'pasta',
'brood', 'tomaten'}
```

```
1 # we maken twee afzonderlijke boodschappenlijsten
2 boodschappen_1 = {'melk', 'brood', 'eieren', 'pasta', 'kaas'}
3 boodschappen_2 = {'wc-papier', 'melk', 'tandpasta', 'tomaten'}
4 # bepaal de doorsnede van beide verzamelingen
5 print(boodschappen_1 & boodschappen_2)
```

```
{'melk'}
```

```
1 # we maken twee afzonderlijke boodschappenlijsten
2 boodschappen_1 = {'melk', 'brood', 'eieren', 'pasta', 'kaas'}
3 boodschappen_2 = {'wc-papier', 'melk', 'tandpasta', 'tomaten'}
4 # bepaal het verschil van beide verzamelingen
5 print(boodschappen_1 - boodschappen_2)
```

```
{'brood', 'kaas', 'pasta', 'eieren'}
```

Wanneer gebruik je een verzameling?

Een verzameling is de beste datastructuur wanneer:

- je enkel unieke elementen nodig hebt. Dubbels worden automatisch verwijderd.
- de volgorde niet belangrijk is. Verzamelingen bewaren geen volgorde, wat ideaal is voor ongeordende informatie.
- het niet nodig of zelfs niet wenselijk is om elementen aan te passen.
- je snel en efficiënt wilt testen of een element voorkomt.

- je wiskundige verzamelingenoperaties wilt uitvoeren, zoals unie (`|`), doorsnede (`&`) en verschil (`-`).

Wanneer gebruik je geen verzameling?

Gebruik geen verzameling wanneer:

- de volgorde belangrijk is;
- je elementen wilt selecteren of aanpassen via een index;
- dubbele waarden betekenis hebben;
- je resultaten reproduceerbaar wilt afdrukken in vaste volgorde.

Uitgewerkt voorbeeld

Onderstaand stukje code illustreert dit met een praktisch voorbeeld.

```
1 # je organiseert een feestje voor je verjaardag
2 # je wilt graag je vrienden uit de klas uitnodigen, plus je vrienden van
   de scouts en van de sportclub
3 # probleem: er is overlap tussen deze groepen
4 # sommige klasgenoten zitten ook in de scouts
5 # andere klasgenoten zitten ook in de sportclub
6 # iemand van de scouts zit ook in de sportclub, maar niet in je klas
7 # je hebt zelfs een klasgenoot die zowel in de scouts als in de
   sportclub zit
8 # het wordt ingewikkeld om het overzicht te behouden: hoeveel mensen wil
   je nu eigenlijk uitnodigen?
9 klasgenoten = {'Emma', 'Noah', 'Liam', 'Alice'}
10 scouts = {'Liam', 'Alice', 'Mila', 'Arthur'}
11 sportclub = {'Noah', 'Arthur', 'Alice', 'Lucas'}
12
13 # je wilt iedereen uitnodigen
14 # je hebt dus de unie nodig van deze drie verzamelingen
15 # dubbele waarden worden automatisch verwijderd
16 uitnodigingen = klasgenoten | scouts | sportclub
17
18 print(uitnodigingen)
```

```
{'Mila', 'Noah', 'Liam', 'Lucas', 'Alice', 'Arthur', 'Emma'}
```

10

Lijsten

Er zijn duidelijk situaties waarin het gebruik van tuples of verzamelingen aangewezen is, maar door hun specifieke eigenschappen zijn ze niet heel breed inzetbaar. Vaak wil je een datastructuur waarvan de volgorde van de elementen belangrijk is (zoals bij een tuple), maar wil je de mogelijkheid hebben om elementen toe te voegen of te verwijderen (zoals bij verzamelingen). Soms wil je elementen van positie kunnen veranderen (wat helemaal niet mogelijk is bij tuples of verzamelingen). Het sorteren van een aantal elementen is dus niet mogelijk bij tuples of verzamelingen.

Lijsten zijn de meest flexibele datastructuur. Ze combineren eigenschappen van tuples (volgorde) en verzamelingen (aanpasbaarheid). Dankzij hun veelzijdigheid worden lijsten heel vaak gebruikt, in de meest uiteenlopende situaties. We gaan dan ook veel dieper in op lijsten en hun toepassingen dan we gedaan hebben voor tuples en verzamelingen.

10.1 Instap

Kijk naar onderstaande code en de bijhorende uitvoer.

```
1 # we starten met deze initiele playlist
2 playlist = ['Imagine', 'Billie Jean', 'Smells Like Teen Spirit']
3 print(playlist)
```

```
['Imagine', 'Billie Jean', 'Smells Like Teen Spirit']
```

```
1 # je ontdekt een nieuw nummer
2 # voeg dit nummer toe achteraan de playlist
3 playlist.append('Bohemian Rhapsody')
4 print(playlist)
```

```
['Imagine', 'Billie Jean', 'Smells Like Teen Spirit', 'Bohemian Rhapsody']
```

```
1 # Je bent 'Smells Like Teen Spirit' even beu gehoord
2 playlist.remove('Smells Like Teen Spirit')
3 print(playlist)
```

```
['Imagine', 'Billie Jean', 'Bohemian Rhapsody']
```

```
1 # verplaats 'Billie Jean' naar het begin van je playlist
2 nummer = playlist.pop(1)      # verwijder het nummer op index 1
3 playlist.insert(0, nummer)    # plaats dit nummer vooraan in de lijst
4 print(playlist)
```

```
['Billie Jean', 'Imagine', 'Bohemian Rhapsody']
```

```
1 # sorteer de playlist alfabetisch
2 playlist.sort()
3 print(playlist)
```

```
['Billie Jean', 'Bohemian Rhapsody', 'Imagine']
```

10.2 Theorie

Wat is een lijst?

Een lijst (Engels: list) is een datastructuur van *geordende, veranderlijke* elementen. De elementen hoeven *niet uniek* te zijn. Dat wil zeggen:

- De volgorde van de elementen blijft behouden. Daardoor kan je elementen selecteren via hun *index* (hun positie in de lijst).
- Je kunt elementen toevoegen, verwijderen en aanpassen.
- Eenzelfde waarde mag meerdere keren voorkomen.

In Python worden lijsten genoteerd met rechte haakjes (`[]`). Je kunt een lege lijst definiëren als `lege_lijs` = `[]`.

De elementen van een lijst mogen verschillende datatypes hebben. Zo is `[-1, 'To be or not to be', 3.14, {1, 'SDV'}, (1, (-3,5)), [1, -2, 3]]` een perfect geldige lijst.

Minimum, maximum, aantal elementen van een lijst

Net zoals bij strings, tuples en verzamelingen, geeft de functie `len()` het aantal elementen in een lijst. De functies `min()` en `max()` geven respectievelijk het kleinste en het grootste element van de lijst. Met de functie `sum()` kan je snel de som berekenen van de elementen van een lijst – in de veronderstelling dat de elementen getallen voorstellen.

```
1 lijst = [120, 340, 560, 220, 410]
2 # toon het kleinste element van de lijst
3 print(min(lijst))
```

```
120
```

```
1 # toon het grootste element van de lijst
2 print(max(lijst))
```

```
560
```

```
1 # toon het aantal elementen van de lijst
2 print(len(lijst))
```

```
5
```

```
1 # toon de som van alle elementen van de lijst
2 print(sum(lijst))
```

```
1650
```

```
1 # je kunt nu gemakkelijk het gemiddelde berekenen van alle elementen
2 print(sum(lijst) / len(lijst))
```

```
330.0
```

Elementen selecteren via slicing

De ordening van de elementen in een lijst is belangrijk. Daardoor is het ook mogelijk om, net zoals bij strings en tuples, via slicing te verwijzen naar de afzonderlijke waarden.

- `lijst[i]` geeft het element met index `i` van `lijst`. Merk op dat het eerste element van de lijst overeenkomt met index `0`.

10. LIJSTEN

- `lijst[start:stop]` geeft het element vanaf index `start` tot vlak voor het element met index `stop` van `lijst`. Het verschil `stop - start` is gelijk aan het aantal geselecteerde elementen. Je hoeft niet noodzakelijk een waarde toe te kennen aan `start` en/of `stop`. Als je geen waarde toekent, kent Python zelf de waarde `0` toe.
- `lijst[start:stop:stap]` geeft het element vanaf index `start` tot vlak voor het element met index `stop` met een stapgrootte `stap`.
- Je kunt ook twee lijsten samenvoegen (concateneren) met de operator `+`.

Onderstaand stukje code illustreert hoe dit alles in zijn werk zou kunnen gaan.

```
1 playlist = [  
2     'Bohemian Rhapsody',      # index 0  
3     'Imagine',               # index 1  
4     'Wonderwall',           # index 2  
5     'Billie Jean',          # index 3  
6     'Smells Like Teen Spirit', # index 4  
7     'Like a Rolling Stone',  # index 5  
8     'Hey Jude'              # index 6  
9 ]  
10 print(playlist[0])          # toon het "eerste" element van de lijst
```

```
Bohemian Rhapsody
```

```
1 print(playlist[-1])         # toon het laatste element van de lijst
```

```
Hey Jude
```

```
1 print(playlist[1:4])        # selecteer en toon de elementen met index 1 tot  
                              vlak voor index 4
```

```
['Imagine', 'Wonderwall', 'Billie Jean']
```

```
1 print(playlist[::2])        # toon de elementen met index 0, 2, 4 en 6
```

```
['Bohemian Rhapsody', 'Wonderwall', 'Smells Like Teen Spirit', 'Hey  
Jude']
```

```
1 print(playlist[:2] + playlist[4:])    # dit is een alternatieve manier  
om de elementen met index 2 en 3 te verwijderen uit de lijst
```



```
['Bohemian Rhapsody', 'Imagine', 'Smells Like Teen Spirit', 'Like a Rolling Stone', 'Hey Jude']
```

Elementen aanpassen, toevoegen en verwijderen

Lijsten zijn erg veelzijdig. Dankzij die flexibiliteit kan je eenvoudig elementen van een lijst aanpassen. Je kunt ook elementen toevoegen, verwijderen of van positie veranderen.

- Je kunt eenvoudig een nieuwe waarde toekennen aan een bestaand element van een lijst:

```
1 # wijzig de naam van het element met index 4
2 playlist[4] = 'Smells Like Teen Spirit (Live)'
3 print(playlist)
```

```
['Bohemian Rhapsody', 'Imagine', 'Wonderwall', 'Billie Jean',
 'Smells Like Teen Spirit (Live)', 'Like a Rolling Stone', 'Hey
 Jude']
```

- Met de methode `append(x)` kan je een nieuw element `x` toevoegen achteraan de lijst:

```
1 # voeg 'Wish You Were Here' toe achteraan de lijst
2 playlist.append('Wish You Were Here')
3 print(playlist)
```

```
['Bohemian Rhapsody', 'Imagine', 'Wonderwall', 'Billie Jean',
 'Smells Like Teen Spirit (Live)', 'Like a Rolling Stone', 'Hey
 Jude', 'Wish You Were Here']
```

- Met de methode `insert(i, x)` kan je een nieuw element `x` toevoegen dat de index `i` krijgt. Alle andere elementen schuiven door.

```
1 # voeg 'Creep' toe op index 2
2 playlist.insert(2, 'Creep')
3 print(playlist)
```

```
['Bohemian Rhapsody', 'Imagine', 'Creep', 'Wonderwall', 'Billie
 Jean', 'Smells Like Teen Spirit (Live)', 'Like a Rolling Stone',
 'Hey Jude', 'Wish You Were Here']
```

- Met de methode `remove(x)` kan je een element `x` verwijderen uit de lijst. Als `x` meer dan één keer zou voorkomen in de lijst, dan wordt het eerste voorkomen van `x` verwijderd.

10. LIJSTEN

```
1 # verwijder het element 'Wonderwall'
2 playlist.remove('Wonderwall')
3 print(playlist)
```

```
['Bohemian Rhapsody', 'Imagine', 'Creep', 'Billie Jean', 'Smells Like Teen Spirit (Live)', 'Like a Rolling Stone', 'Hey Jude', 'Wish You Were Here']
```

- Met de methode `pop(i)` kan je het element met index `i` verwijderen uit de lijst. De waarde van dit element wordt teruggestuurd, dus je kunt deze eventueel opslaan in een variabele.

```
1 # verwijder het element op index 3
2 verwijderd = playlist.pop(3)
3 print(verwijderd)
4 print(playlist)
```

```
Billie Jean
['Bohemian Rhapsody', 'Imagine', 'Creep', 'Smells Like Teen Spirit (Live)', 'Like a Rolling Stone', 'Hey Jude', 'Wish You Were Here']
```

- Met de methode `index(x)` kan je de index van het element `x` opvragen. Als `x` meer dan één keer zou voorkomen, geeft deze methode de index van het eerste voorkomen.

```
1 # bepaal de index van het element 'Imagine'
2 i = playlist.index('Imagine')
3 print(i)
```

```
1
```

Lidmaatschap controleren

Net zoals bij verzamelingen kan je met de operator `in` eenvoudig controleren of een element voorkomt in een lijst:

```
1 # komt 'Hotel California' voor in de lijst?
2 print('Hotel California' in playlist)
```

```
False
```

```
1 # komt 'Hey Jude' voor in de lijst?
2 print('Hey Jude' in playlist)
```

```
True
```

Sorteren

Met de methode `sort()` kan je een lijst alfabetisch sorteren.

```
1 # sorteer de lijst alfabetisch
2 playlist.sort()
3 print(playlist)
```

```
['Bohemian Rhapsody', 'Creep', 'Hey Jude', 'Imagine', 'Like a Rolling
Stone', 'Smells Like Teen Spirit (Live)', 'Wish You Were Here']
```

Als je omgekeerd alfabetisch wilt sorteren, kan dat met `sort(reverse=True)`.

```
1 # sorteer de lijst omgekeerd alfabetisch
2 playlist.sort(reverse=True)
3 print(playlist)
```

```
['Wish You Were Here', 'Smells Like Teen Spirit (Live)', 'Like a
Rolling Stone', 'Imagine', 'Hey Jude', 'Creep', 'Bohemian Rhapsody']
```

Itereren over een lijst

Heel vaak wil je alle elementen in een lijst afzonderlijk overlopen. Dat kan op twee manieren.

- Je kunt *itereren over de waarden* van de elementen in de lijst.

```
1 # itereer over alle waarden in de lijst
2 for titel in playlist:
3     print(titel)
```

```
Wish You Were Here
Smells Like Teen Spirit (Live)
Like a Rolling Stone
Imagine
Hey Jude
Creep
Bohemian Rhapsody
```

- Je kunt *itereren over de indices* van de elementen in de lijst.

```
1 # itereer over de indices van alle elementen in de lijst
2 for i in range(len(playlist)):
3     # voor onze playlist neemt i achtereenvolgens de waarden 0, 1,
      2, ..., 6 aan
4     # toon alleen de titels met een even index
5     if i % 2 == 0:
6         print(playlist[i])
```

```
Wish You Were Here
Like a Rolling Stone
Hey Jude
Bohemian Rhapsody
```

Het is erg belangrijk dat je bij een opgave goed nadenkt over hoe je precies wilt itereren. **In geval van twijfel, kies je beter om te itereren over de indices.** Als je de index van een element kent, kan je altijd eenvoudig de waarde oproepen via `lijst[i]`. Mogelijk leidt deze methode niet in alle gevallen tot de elegantste oplossing, maar door te itereren over de indices, zal je nooit voor een onaangename verrassing komen te staan.

Omgekeerd zou je de index van een bepaalde waarde kunnen oproepen via `lijst.index(waarde)`. Dit gaat goed zolang alle waarden in de lijst uniek zijn, maar je mag problemen verwachten als `waarde` meer dan één keer zou voorkomen in de lijst. Je moet dus het potentiële voordeel van elegante code in sommige gevallen afwegen tegen het risico dat je code niet altijd zal werken.

Gevaarlijke valkuilen

Er zijn twee fouten die vaak gemaakt worden door beginners. Laat je niet vangen door volgende valkuilen.

- Omdat `sort()` een methode is die rechtstreeks inwerkt op de lijst zelf, wordt er geen waarde geretourneerd. Dat klinkt misschien abstract, maar onderstaand voorbeeld verduidelijkt dit hopelijk.

```
1 # sort() is een methode, geen functie
2 # het onderscheid is subtiel, en het is misschien verwarrend
3 # de volgende lijn is syntactisch correct, maar doet niet wat je
      misschien denkt dat het doet
4 playlist = playlist.sort()
5 print(playlist)
```

Je denkt misschien dat de lijst opnieuw alfabetisch gesorteerd is, maar dat is niet geval. De waarde van de variabele `playlist` is nu

```
None
```

Met andere woorden: je bent de inhoud van de lijst `playlist` volledig kwijtgespeeld door deze goedbedoelde constructie. Oeps, dat was waarschijnlijk niet de bedoeling.

- Kijk even naar volgend stukje code.

```
1 a = 3
2 b = a
3 a = a + 1
4 print(a, b)
```

Je begrijpt ongetwijfeld de uitvoer van dit programma:

```
4 3
```

De situatie is verschillend wanneer je werkt met lijsten.

```
1 lijst_1 = [1, 2]
2 lijst_2 = lijst_1
3 lijst_1.append(3)
4 print(lijst_1, lijst_2)
```

```
[1, 2, 3] [1, 2, 3]
```

Wanneer je `lijst2 = lijst1` schrijft, maak je dus géén kopie. Beide variabelen verwijzen in werkelijkheid naar dezelfde lijst in het geheugen van de computer. Verander je de ene lijst, dan verandert de andere ook – ook al zou je dat niet meteen verwachten.

Als je een “echte” kopie van een lijst zou willen maken, doe je dat met de methode `copy()`.

```
1 lijst_1 = [1, 2]
2 lijst_2 = lijst_1.copy()
3 lijst_1.append(3)
4 print(lijst_1, lijst_2)
```

```
[1, 2, 3] [1, 2]
```

Wanneer gebruik je een lijst?

Gebruik een lijst wanneer:

- het van belang is in welke volgorde de elementen voorkomen;
- de waarde van element moet kunnen veranderen;
- elementen moeten kunnen verschuiven binnen de datastructuur;
- je wilt itereren, filteren, sorteren.

Wanneer gebruik je geen lijst?

Gebruik geen lijst wanneer:

- elementen uniek moeten blijven;
- je wiskundige verzamelingsbewerkingen wilt uitvoeren;
- structuur en betekenis vastliggen.

Omwille van hun flexibiliteit is de verleiding groot om lijsten te gebruiken in situaties waarvoor ze eigenlijk niet bedoeld zijn. Dat hoeft niet noodzakelijk problematisch te zijn, maar het kan wel leiden tot onnodig complexe code en/of een minder efficiënte uitvoering van je programma. De juiste datastructuur kiezen is dus een belangrijke conceptuele stap, waar je grondig over moet nadenken.

10.3 Overzicht

Je gebruikt een datastructuur wanneer je meerdere waarden wil toekennen aan één enkele variabele. In dit hoofdstuk hebben we **tuples**, **verzamelingen** en **lijsten** behandeld als voorbeeld van datastructuren.

Eigenschappen van tuples, verzamelingen en lijsten

Eigenschap	Tuple	Verzameling	Lijst
Geordend	ja	nee	ja
Veranderlijk	nee	ja	ja
Elementen uniek	nee	ja	nee
Indexering mogelijk	ja	nee	ja
Slicing mogelijk	ja	nee	ja
Mag duplicaten bevatten	ja	nee	ja

- Gebruik een tuple voor vaste gegevens.
- Gebruik een verzameling voor unieke elementen zonder volgorde.
- Gebruik een lijst voor geordende gegevens die mogen veranderen.

Algemene functies

- De functie `len(x)` geeft het aantal elementen van een tuple, verzameling of lijst.
- De functie `type(x)` geeft het datatype van `x`.

Tuples

Aanmaken

- `t = (1, 2, 3)`

Indexering en slicing

- `t[i]` verwijst naar het element met index `i` in tuple `t`.
- `t[start: stop]` geeft een tuple terug die begint bij het element met index `start` van tuple `t` en die stopt *vlak voor* het element met index `stop`.

Unpacking

- Voor een tuple `t` met drie elementen kan je de afzonderlijke elementen individueel uitpakken en bewaren met `x, y, z = t`

Verzamelingen

Aanmaken

- `s = {1, 2, 3}`

Lege verzameling

- `s = set()`

Elementen toevoegen en verwijderen

- De methode `s.add(x)` voegt `x` toe aan de verzameling `s`.
- De methode `s.remove(x)` verwijdert `x` uit de verzameling `s`.

Lidmaatschap testen

- De operator `x in s` geeft `True` terug als het element `x` voorkomt in de verzameling `s`, en `False` als dat niet het geval is.

Verzameloingsoperatoren

- `s1 | s2` geeft een verzameling terug die de **unie** is van verzamelingen `s1` en `s2`.
- `s1 & s2` geeft een verzameling terug die de **doorsnede** is van verzamelingen `s1` en `s2`.
- `s1 - s2` geeft een verzameling terug die het **verschil** is van verzamelingen `s1` en `s2`.

Lijsten

Aanmaken

- `lijst = [1, 2, 3]`

Indexering en slicing

- `lijst[i]` verwijst naar het element op index `i` van `lijst`.
- `lijst[start:stop]` geeft een lijst terug die begint bij het element met index `start` van `lijst` en die stopt vlak voor het element met index `stop`.

- `lijst[start:stop:stap]` geeft een lijst terug die begint bij het element met index `start` van `lijst` en die stopt vlak voor het element met index `stop`, met stapgrootte `stap`.

Elementen toevoegen

- De methode `lijst.append(x)` voegt element `x` toe **achteraan** `lijst`.
- De methode `lijst.insert(i, x)` voegt element `x` toe op index `i` in `lijst`. Alle volgende elementen schuiven één plaats op.

Elementen verwijderen

- De methode `lijst.remove(x)` verwijdert het **eerste voorkomen** van `x` uit `lijst`. Er wordt een foutmelding gegeven als `x` niet voorkomt.
- De methode `lijst.pop(i)` verwijdert het element op index `i` in `lijst` én **retourneert** dit element.

Zoeken en testen

- De methode `lijst.index(x)` geeft de index van het eerste voorkomen van `x` in `lijst`. Er wordt een foutmelding gegeven als `x` niet voorkomt.
- De operator `x in lijst` geeft `True` terug als het element `x` voorkomt in `lijst`, en `False` als dat niet het geval is.

Sorteren

- De methode `lijst.sort()` sorteert de lijst oplopend (alfabetisch of numeriek). De originele lijst wordt aangepast.
- De methode `lijst.sort(reverse=True)` sorteert de lijst in omgekeerde volgorde.

Kopiëren

- De methode `lijst.copy()` maakt een **nieuwe lijst** met dezelfde elementen. Wijzigingen aan de kopie hebben geen invloed op de originele lijst, en omgekeerd.

Itereren

- Met de constructie `for element in lijst:` itereer je over alle **waarden** van de lijst.
- Met de constructie `for i in range(len(lijst)):` itereer je over alle **indices** van de lijst.
- In geval van twijfel kan je beter kiezen om te itereren over de indices.

Veelgemaakte fouten

- De methode `lijst.sort()` retourneert **geen** nieuwe lijst.
- `lijst2 = lijst1` maakt **geen kopie**.

Funcities en methodes

In Python bestaan er twee manieren om bewerkingen uit te voeren op datastructuren: **funcities** en **methodes**.

Funcities

- Funcities staan **los van een object**.
- Ze werken op een waarde die je als argument meegeeft.
- Ze kunnen toegepast worden op verschillende datatypes.
- Algemene vorm: `functie(argument)`
- Voorbeelden: `len()`, `print()`, `abs()`, ...

Methodes

- Methodes horen bij een **specifiek datatype**.
- Ze worden opgeroepen via het object waarop ze werken.
- Ze wijzigen vaak het object zelf.
- Algemene vorm: `object.methode(argument)`
- Voorbeelden: `lijst.append(x)`, `verzameling.add(x)`, `lijst.sort()`, ...

Belangrijk verschil

- Funcities geven (meestal) een resultaat terug.
- Methodes werken (meestal) rechtstreeks op het object.

Voorbeeld

- De functie `len(lijst)` geeft uitvoer, maar verandert de lijst niet.
- De methode `lijst.sort()` geeft geen uitvoer, maar werkt in op `lijst` en past deze dus zelf aan.

10.4 Verwerking

- 1 Je krijgt een aantal situaties waarin je meerdere waarden wilt bewaren in één enkele variabele. Welke datastructuur zou je telkens gebruiken? Soms is er interpretatie mogelijk. Verklaar dus telkens je keuze.
 - a. Je wilt de geboortedatum van een leerling opslaan: dag, maand en jaar.
 - b. De studiemeesters willen de namen bijhouden van alle leerlingen die vandaag afwezig zijn.
 - c. Je wilt een lijst bijhouden van high scores in een game.
 - d. Je wilt bijhouden welke vakken een leerling volgt. Een vak mag maar één keer voorkomen.
 - e. Je wilt bijhouden welke leerkrachten lesgeven in het zesde jaar.
 - f. Je wilt een overzicht van alle verschillende automerken op de parking van een supermarkt.
 - g. Je wilt bijhouden welke spelers momenteel op het veld staan in een sportwedstrijd.
 - h. Je wilt de discografie opslaan van The Beatles.
 - i. Je wilt het verzameld werk opslaan van een auteur die nog in leven is.
 - j. Je wilt een overzicht bewaren van de tien best verkochte boeken van vorig jaar.
- 2 Je gebruikt een tuple voor een boodschappenlijst. Wat gaat er fout zodra de inhoud moet veranderen?
- 3 Een ijssalon wil een datastructuur met alle unieke smaken in volgorde van populariteit. Welke datastructuur stel je voor?
- 4 De school wilt een overzicht maken van klassen, waarbij elke klas een lijst van leerlingen bevat. Wat is het meest geschikte datatype voor het hoogste niveau? Waarom?
- 5 Kijk naar onderstaand stukje code en de bijhorende uitvoer.


```
1 # maak een verzameling van lijsten
2 verzameling = {[1, 2], [3, 4]}
3 print(verzameling)
```

```
TypeError: cannot use 'list' as a set element (unhashable type:
'list')
```

Waarom zou een verzameling van lijsten niet toegestaan zijn in Python?

10.5 Opdrachten

Reeks 1

6 Je vindt in Dodona volgende playlist:

```
1 playlist = [  
2     'Bohemian Rhapsody',  
3     'Black',  
4     'Go Your Own Way',  
5     'Billie Jean',  
6     'Smells Like Teen Spirit',  
7     'Like a Rolling Stone',  
8     'Hey Jude',  
9     'Nothing Compares 2 U',  
10    'Wonderwall',  
11    'Purple Rain',  
12    'Back To Black',  
13    'Like a Prayer'  
14 ]
```

- a. Druk de volledige playlist af.
- b. Toon het eerste nummer van de playlist.
- c. Toon het laatste nummer van de playlist.
- d. Toon de nummers met index 2 tot en met 5.
- e. Je realiseert je dat het gaat om een live-versie van 'Bohemian Rhapsody'. Pas de naam van dit nummer aan naar 'Bohemian Rhapsody (Live)'. Druk de aangepaste playlist opnieuw af.
- f. Voeg het nummer 'Stairway to Heaven' toe achteraan de playlist. Voeg het nummer 'Let It Be' toe op index 1. Druk de playlist opnieuw af.
- g. Verwijder het nummer 'Wonderwall' uit de playlist. Zorg dat 'Billie Jean' helemaal vooraan komt te staan. Druk de aangepaste playlist af.
- h. Maak een nieuwe lijst met alle nummers waarvan de titel meer dan 15 tekens bevat. Toon deze nieuwe lijst.
- i. Toon hoeveel nummers er in totaal in de huidige playlist staan.
- j. Toon hoeveel nummers het woord 'Like' in de titel bevatten.

- k. Maak een kopie van de playlist. Voeg 'God Only Knows' toe vooraan in de kopie. Zorg dat de oorspronkelijke playlist niet bewerkt wordt. Toon deze nieuwe lijst.
- l. Maak een nieuwe playlist met alleen de nummers op even index. Sorteer deze nieuwe lijst alfabetisch. Toon het resultaat.

Test je code in Dodona.

- 7** Schrijf een programma dat aan de gebruiker vraagt om een lijst met getallen in te voeren. Deze getallen stellen de maximumtemperatuur voor die in een bepaalde periode gemeten is door het KMI in Ukkel. Je programma berekent en toont vervolgens:

- de lijst die gesorteerd is van groot naar klein;
- de hoogste waarde in de lijst;
- het aantal dagen waarop de maximumtemperatuur groter was dan 25°C;
- de gemiddelde temperatuur in deze periode, afgerond op 0,1°C;
- het aantal dagen waarop de temperatuur hoger was dan de gemiddelde temperatuur.

Test je code in Dodona.

- 8** Schrijf een programma dat aan de gebruiker vraagt om een lijst met namen in te voeren. Sommige namen komen meer dan één keer voor in de lijst. De uitvoer van je programma is een nieuwe lijst waarin de namen in dezelfde volgorde staan, maar waarbij elke naam precies één keer voorkomt. Test je code in Dodona.

- 9** Schrijf een programma dat aan de gebruiker vraagt om een lijst met getallen in te voeren. De uitvoer van je programma is een nieuwe lijst waarin elk getal verhoogd werd met 1. Test je code in Dodona.

- 10** Schrijf een functie `verwijder_leerling(naam, lijst)`. `naam` is een string. `lijst` is een lijst met namen. Als `naam` voorkomt in `lijst`, geeft je functie de lijst met namen terug, maar `naam` mag hier niet meer in voorkomen. Als `naam` niet voorkomt in `lijst`, geeft je functie de melding `<naam> komt niet voor in <lijst>`. Test je code in Dodona.

- 11** Schrijf een functie `langste_titel(playlist)`. `playlist` is een lijst van strings. Je functie geeft de langste titel terug die voorkomt in `playlist`. Test je code in Dodona.

Reeks 2

- 12** Schrijf een programma dat aan de gebruiker vraagt om een lijst met getallen in te voeren. De uitvoer van je programma toont op twee verschillende lijnen

- een lijst met de elementen op een even index;
- een lijst met de elementen op een oneven index.

Test je code in Dodona.

- 13** Schrijf een programma dat aan de gebruiker vraagt om een lijst met voornamen in te voeren. De uitvoer van je programma is een lijst van alle namen die beginnen met een klinker. In deze nieuwe lijst mag elke naam niet meer dan één keer voorkomen. Test je code in Dodona.

- 14** Onze genetische code wordt bewaard in het DNA. Een DNA-molecule bestaat uit twee ketens die als een dubbele helix met elkaar vervlochten zijn. Sterk vereenvoudigd komt het er op neer dat een DNA-molecule bestaat uit een opeenvolging van vier stikstofbasen: adenine (A), guanine (G), thymine (T) of cytosine (C). De volgorde van deze basen in het DNA vormt een code waarmee het lichaam eiwitten kan maken.

In de dubbele helix zijn de twee afzonderlijke DNA-strengen elkaars complement. Dat wil zeggen dat tegenover elke A een T staat, en vice versa. Tegenover elke C staat een G, en vice versa.

Schrijf een functie `complement(DNA)`. De lijst `DNA` bestaat uitsluitend uit de strings `'A'`, `'C'`, `'G'` en `'T'`. Deze lijst stelt een DNA-streng voor. Je functie geeft een nieuwe lijst terug die de complementaire streng voorstelt. De oorspronkelijke lijst mag niet gewijzigd worden.

- 15** Een man komt uit het café en is te dronken om nog de weg naar huis te vinden. Hij wandelt dus doelloos door de stad. Merkwaardig genoeg is hij wel nog in staat om zich te oriënteren volgens het magnetisch veld van de aarde, want elke stap is ofwel pal naar het noorden, het oosten, het zuiden of het westen.

De positie van het café komt overeen met de oorsprong van een orthonormaal assenstelsel. De wandeling van de man wordt voorgesteld door een lijst met als mogelijke elementen:

- `'N'`: de man zet een stap volgens de positieve oriëntatie van de y -as;
- `'O'`: de man zet een stap naar rechts, volgens de positieve oriëntatie van de x -as;
- `'Z'`: de man zet een stap omlaag, volgens de negatieve oriëntatie van de y -as;
- `'W'`: de man zet een stap naar links, volgens de negatieve oriëntatie van de x -as.

Schrijf een functie `dronken_wandeling(lijt)` die een tuple teruggeeft. Het eerste element van de tuple is opnieuw een tuple met daarin de x - en y - coördinaat van de man op het einde van zijn wandeling. Het tweede element van de tuple is het aantal stappen dat de man in totaal gezet heeft. Test je code in Dodona.

- 16** Tijdens de examenperiode zitten de leerlingen van een klas, gesorteerd volgens klasnummer, op één lange lijn. Spieken van de leerling links of rechts van je is dus niet heel erg moeilijk. Je mag uitgaan van de veronderstelling dat spieken bij leerlingen die niet je burens zijn, onmogelijk is.

Om mogelijke fraude gemakkelijk te detecteren, wil de school de resultaten automatisch analyseren. Wanneer een mogelijk geval van fraude automatisch gedetecteerd wordt, kan de leerkracht manueel nakijken of er al dan niet gespiekt werd.

De resultaten van alle leerlingen in een klas worden in een lijst voorgesteld. Deze resultaten zijn percentages, natuurlijke getallen tussen 0 en 100. We noemen een situatie *verdacht* wanneer een leerling een resultaat haalt dat hoogstens 5 punten afwijkt van een leerling die naast hem/haar zat. We noemen een situatie *erg verdacht* wanneer twee naast zittende leerlingen exact hetzelfde resultaat behaald hebben.

Schrijf een functie `gespiekt(lijt)` die alle verdachte en erg verdachte situaties toont. Als leerlingen met klasnummer n en $n + 1$ een verdachte situatie vormen, geeft je programma als uitvoer **`n en n+1 zijn verdacht..`** Als leerlingen met klasnummer n en $n + 1$ een erg verdachte situatie vormen, geeft je programma als uitvoer **`n en n+1 zijn erg verdacht..`** Als er geen verdachte situaties waren, geeft je programma als uitvoer **`Er waren geen verdachte situaties..`** Test je code in Dodona.

- 17** We noemen een getal perfect wanneer het gelijk is aan de som van de echte delers van dat getal. Een echte deler van een getal x is daarbij gedefinieerd als een positieve deler van x , die niet gelijk is aan x zelf. Zo is 6 een perfect getal, want het heeft 1, 2 en 3 als echte delers, en $1 + 2 + 3 = 6$.

Schrijf een programma dat aan de gebruiker een natuurlijk getal vraagt. Je programma toont dan alle perfecte getallen die kleiner zijn dan of gelijk aan dat getal. Je programma toont daarbij ook de som waarmee geverifieerd kan worden dat de berekening klopt. Test je code in Dodona.

- 18** Er zijn verschillende manieren om het centrum van een aantal getallen te karakteriseren. De bekendste zijn wellicht het gemiddelde en de mediaan. Je berekent het gemiddelde van de getallen als volgt:

$$\bar{x} = \frac{1}{n} \cdot \sum_{i=1}^n x_i.$$

De mediaan van een aantal getallen vind je door ze te sorteren van klein naar groot. Bij een oneven aantal getallen is de mediaan dan gelijk aan het middelste getallen. Voor een even aantal getallen is de mediaan gelijk aan het gemiddelde van de twee middelste getallen.

10. LIJSTEN

Er zijn ook verschillende manieren om de spreiding van een aantal getallen te karakteriseren. Je kent misschien de interkwartielafstand. Deze is het verschil van het derde met het eerste kwartiel. Het eerste kwartiel (Q_1) is de mediaan van de kleinste helft waarnemingsgetallen. Het derde kwartiel (Q_3) is de mediaan van de grootste helft waarnemingsgetallen. Daarnaast ken je misschien ook de standaardafwijking als maat voor de spreiding van een aantal getallen. Je berekent deze als volgt:

$$s = \sqrt{\frac{1}{n-1} \cdot \sum_{i=1}^n (x_i - \bar{x})^2}.$$

- Schrijf een functie `gemiddelde(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft het gemiddelde van al deze getallen terug, afgerond op 1 cijfer na de komma.
- Schrijf een functie `mediaan(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft de mediaan van al deze getallen terug, afgerond op 1 cijfer na de komma.
- Schrijf een functie `Q1(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft het eerste kwartiel van al deze getallen terug, afgerond op 1 cijfer na de komma.
- Schrijf een functie `Q3(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft het derde kwartiel van al deze getallen terug, afgerond op 1 cijfer na de komma.
- Schrijf een functie `IKA(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft de interkwartielafstand van al deze getallen terug, afgerond op 1 cijfer na de komma.
- Schrijf een functie `standaardafwijking(lijt)`. De variabele `lijst` bevat een aantal getallen. Je functie geeft de standaardafwijking van al deze getallen terug, afgerond op 1 cijfer na de komma.

Reeks 3

19 Je wilt een nieuwe klascompetitie organiseren. Deelnemers aan deze competitie moeten een bal zo lang mogelijk hooghouden, d.w.z. de bal raken zonder dat deze de grond raakt. Je telt voor elke kandidaat het aantal baltoetsen, en op basis daarvan bereken je een score:

- De speler start in niveau 1; elke baltoets is 1 punt waard.
- Vanaf de 11de baltoets zit de speler in niveau 2; de baltoetsen zijn nu elk 2 punten waard.

- Vanaf de 21ste baltoets zit de speler in niveau 3; de baltoetsen zijn nu elk 3 punten waard.
- ...

Een speler die de bal 22 keer hoog kan houden, behaalt bv. een individuele score van $10 \cdot 1 + 10 \cdot 2 + 2 \cdot 3 = 36$ punten.

Per klas mogen verschillende leerlingen deelnemen. Elke leerling krijgt 1 kans. De totale score van een klas wordt berekend als het gemiddelde van de scores van alle deelnemende leerlingen. Dit gemiddelde wordt vervolgens afgerond naar beneden.

- Schrijf een functie `niveau(n)`. De variabele `n` stelt de n -de baltoets voor. De functie geeft het niveau terug waarin de speler zit bij de n -de baltoets. Dit getal is gelijk aan de score die de kandidaat verdient met deze ene baltoets.
- Schrijf een functie `score(aantal)`. De variabele `aantal` stelt het aantal baltoetsen voor dat een kandidaat heeft behaald. De functie geeft de totale score terug van deze kandidaat.
- Schrijf een functie `klasresultaat(lijt)`. In de variabele `lijst` zitten het aantal baltoetsen van alle deelnemers in een klas. Elk van deze aantallen is een natuurlijk getal. De uitvoer van je functie is de gemiddelde score van alle deelnemers in deze klas.

20 Onze genetische code wordt bewaard in het DNA. Een DNA-molecule bestaat uit twee ketens die als een dubbele helix met elkaar vervlochten zijn. Sterk vereenvoudigd komt het er op neer dat een DNA-molecule bestaat uit een opeenvolging van vier stikstofbasen: adenine (A), guanine (G), thymine (T) of cytosine (C). De volgorde van deze basen in het DNA vormt een code waarmee het lichaam eiwitten kan maken.

Een stuk DNA dat voor een bepaalde eigenschap codeert, noemen we een gen. Meestal gaat bestaat een gen uit een opeenvolging van duizenden stikstofbasen. Van een aantal ziektes en aandoeningen weten we precies welk gen daarvoor verantwoordelijk is. Het is dus mogelijk om het DNA van een patiënt uit te lezen en na te gaan of een bepaald gen al dan niet voorkomt. Precies dat is wat we in deze opgave gaan doen.

Schrijf een functie `DNA(uitgelezen, gen)`. `uitgelezen` is een opeenvolging van de strings 'A', 'C', 'G' en 'T'. `uitgelezen` staat symbool voor het uitgelezen DNA. Ook `gen` is een opeenvolging van de strings 'A', 'C', 'G' en 'T'. Deze lijst staat symbool voor een bepaald gen.

Als de precieze opeenvolging van `gen` voorkomt in `uitgelezen`, is de patiënt erfelijk belast met dit gen. Je programma geeft dan een tuple (`True`, `begin`) terug, waarbij `begin` de index is vanaf waar het gen (`gen`) voorkomt in het DNA (`uitgelezen`). Als de precieze opeenvolging van `gen` niet voorkomt in `uitgelezen`, geeft je functie `False` terug.

- 21** Eendjes vissen is een gekend spel van op de kermis. De eendjes komen één per één voorbij, en de speler kan er een aantal uit vissen. Elk eendje heeft als waarde een natuurlijk getal tussen 1 en 10.

In deze opgave wordt een variant op het klassieke eendjes vissen beschouwd. De speler ziet op voorhand namelijk de waarden van alle eendjes die voorbij komen. Deze waarden zitten in een lijst. De bedoeling is nu om een welbepaald (maar variabel) aantal opeenvolgende eendjes te selecteren om een zo hoog mogelijke score te behalen.

Schrijf een functie `eendjes_vissen(lijst, aantal)`. De variabele `lijst` stelt de eendjes voor die de speler voorbij ziet komen. De variabele `aantal` is het aantal eendjes dat de speler mag vissen. De functie geeft een tuple (`positie`, `score`) terug. Hierbij is `positie` de index van het eerste eendje dat de speler moet vissen, en `score` is de maximale score die hij/zij zo kan behalen.

11

Geneste lijsten

Je weet ondertussen dat de elementen van een lijst om het even welk datatype mogen hebben. Dat betekent ook dat een element van een lijst zelf een lijst mag zijn. Zo'n lijst noemen we een *geneste lijst* (Engels: *nested list*), of ook wel een lijst van lijsten. Geneste lijsten laten je toe om complexere, gestructureerde gegevens overzichtelijk voor te stellen.

11.1 Instap

Kijk naar onderstaande code en de bijhorende uitvoer. De gegevens zijn de echte uitslag van de finale van de 800 meter bij de vrouwen op het EK atletiek 2026 in Birmingham.

```
1 # de lijst bevat de uitslag van de finale 800m vrouwen op het EK
   atletiek 2026
2 # elke sublijst bevat de naam, het land en de tijd van een atlete
3 uitslag = [
4     ['Audrey Werro', 'SUI', '1:54.81'],
5     ['Keely Hodgkinson', 'GBR', '1:55.01'],
6     ['Femke Broeders-Bol', 'NED', '1:55.54'],
7     ['Gabiya Galvydyte', 'LTU', '1:59.12'],
8     ['Jemma Reekie', 'GBR', '1:59.15'],
9     ['Eloisa Coiro', 'ITA', '1:59.35'],
10    ['Valentina Rosamilia', 'SUI', '1:59.42'],
11    ['Veronica Vancardo', 'SUI', '2:00.17'],
12    ['Anais Bourgoïn', 'FRA', '2:00.65'],
13    ['Clara Liberman', 'FRA', '2:01.84']
14 ]
15 print(uitslag)
```

11. GENESTE LIJSTEN

```
[[ 'Audrey Werro', 'SUI', '1:54.81'], [ 'Keely Hodgkinson', 'GBR',  
  '1:55.01'], [ 'Femke Broeders-Bol', 'NED', '1:55.54'], [ 'Gabija  
Galvydyte', 'LTU', '1:59.12'], [ 'Jemma Reekie', 'GBR', '1:59.15'],  
[ 'Eloisa Coiro', 'ITA', '1:59.35'], [ 'Valentina Rosamilia', 'SUI',  
  '1:59.42'], [ 'Veronica Vancardo', 'SUI', '2:00.17'], [ 'Anais  
Bourgoin', 'FRA', '2:00.65'], [ 'Clara Liberman', 'FRA', '2:01.84']]
```

```
1 # toon het resultaat van de bronzen medaillewinnares (index 2)  
2 print(uitslag[2])
```

```
[ 'Femke Broeders-Bol', 'NED', '1:55.54']
```

```
1 # toon enkel de naam van de bronzen medaillewinnares  
2 print(uitslag[2][0])
```

```
Femke Broeders-Bol
```

```
1 # toon enkel de naam en de tijd van de laatst geklasseerde finaliste  
2 print(uitslag[-1][0], uitslag[-1][2])
```

```
Clara Liberman 2:01.84
```

11.2 Theorie

Wat is een geneste lijst?

Een *geneste lijst* is een lijst waarvan één of meerdere elementen zelf een lijst zijn. Op die manier kan je gegevens in verschillende niveaus structureren. In het voorbeeld hierboven is `uitslag` een lijst met tien elementen, en elk van die elementen is op zijn beurt een lijst met de naam, het land en de tijd van een atlete.

Er is geen enkele beperking op het aantal niveaus dat je kunt nesten. Je zou dus perfect een lijst van lijsten van lijsten kunnen maken, al wordt zo'n structuur al snel moeilijk leesbaar. In de praktijk beperk je het aantal niveaus dan ook meestal tot twee, zoals in het voorbeeld hierboven.

Matrices

Een bijzonder geval van een geneste lijst is een matrix: een rechthoekige structuur van rijen en kolommen, waarbij elke rij een lijst van getallen is.

```
1 # een matrix van 3 rijen en 3 kolommen
2 matrix = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7 print(matrix)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Elementen selecteren

Om een element van een geneste lijst te selecteren, gebruik je *twee keer een index na elkaar*. De eerste index verwijst naar de positie in de buitenste lijst, de tweede index naar de positie in de binnenste lijst.

- `geneste_lijst[i]` geeft de sublijst op positie `i`.
- `geneste_lijst[i][j]` geeft het element op positie `j` van de sublijst op positie `i`.

```
1 # toon de volledige tweede rij van de matrix
2 # deze komt overeen met het element met index 1 in de geneste lijst
3 # de uitvoer is dus zelf opnieuw een lijst
4 print(matrix[1])
```

```
[4, 5, 6]
```

```
1 # toon het element op de tweede rij, derde kolom
2 print(matrix[1][2])
```

```
6
```

Slicing

Ook slicing blijft mogelijk, zowel op het niveau van de sublijsten als op het niveau van de elementen van een sublijst.

```
1 # toon de eerste twee rijen van de matrix
2 print(matrix[0:2])
```

```
[[1, 2, 3], [4, 5, 6]]
```

```
1 # toon de eerste twee elementen van de tweede rij
2 print(matrix[1][0:2])
```

```
[4, 5]
```

Elementen aanpassen, toevoegen en verwijderen

Je past een element van een geneste lijst aan door de gepaste combinatie van indices te gebruiken. Ook de methodes die je al kende van gewone lijsten (`append()`, `insert()`, `remove()`, `pop()`) kan je toepassen op een sublijst.

```
1 matrix = [
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ]
6 # wijzig het element op de eerste rij, tweede kolom
7 matrix[0][1] = 20
8 print(matrix)
```

```
[[1, 20, 3], [4, 5, 6], [7, 8, 9]]
```

```
1 # voeg een nieuw getal toe aan de derde rij
2 # wiskundig kunnen we deze geneste lijst uiteraard geen matrix meer
   noemen
3 matrix[2].append(10)
4 print(matrix)
```

```
[[1, 20, 3], [4, 5, 6], [7, 8, 9, 10]]
```

```
1 # voeg een volledig nieuwe rij toe aan de matrix
2 matrix.append([11, 12, 13])
3 print(matrix)
```

```
[[1, 20, 3], [4, 5, 6], [7, 8, 9, 10], [11, 12, 13]]
```

Itereren over geneste lijsten

Om alle elementen van een geneste lijst te overlopen, heb je telkens *twee* **for**-lussen nodig: een eerste om te itereren over de sublijsten, en een tweede om binnen elke sublijst over de afzonderlijke elementen te itereren. Men noemt dit een *geneste lus* (Engels: *nested loop*).

```
1 matrix = [
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ]
6 # toon elk element van de matrix afzonderlijk
7 for rij in matrix:
8     for element in rij:
9         print(element)
```

```
1
2
3
4
5
6
7
8
9
```

Net zoals bij een gewone lijst kan je er ook voor kiezen om over de indices te itereren in plaats van over de waarden. Dat is vooral handig wanneer je de positie van een element nodig hebt, bijvoorbeeld om het element nadien aan te passen.

```
1 # tel bij elk element van de matrix het getal 1 op
2 for i in range(len(matrix)):
3     for j in range(len(matrix[i])):
4         # verhoog de waarde van het element op rij i en kolom j met 1
5         matrix[i][j] = matrix[i][j] + 1
6 print(matrix)
```

```
[[2, 3, 4], [5, 6, 7], [8, 9, 10]]
```

Wanneer gebruik je een geneste lijst?

Gebruik een geneste lijst wanneer:

11. GENESTE LIJSTEN

- je gegevens in verschillende niveaus wilt structureren;
- je gegevens in de vorm van een matrix, een rooster of een tabel wilt voorstellen;
- de sublijsten op zich al zinvolle, samenhangende groepen elementen vormen.

Wanneer gebruik je geen geneste lijst?

Gebruik geen geneste lijst wanneer één niveau van structuur voldoende is. In dat geval houd je beter een gewone lijst aan.

Zoals steeds geldt: hoe meer niveaus je nest, hoe moeilijker je code leesbaar en onderhoudbaar wordt. Denk dus goed na over de structuur van je gegevens, en nest niet dieper dan strikt noodzakelijk.

11.3 Verwerking

- 1** Je krijgt in Dodona de lijst met de gegevens van de finale van de 800 meter bij de vrouwen op het EK atletiek 2026 in Birmingham.
- Schrijf een programma dat de naam van de winnares toont.
 - Schrijf een programma dat de nationaliteit van de atlete op de vijfde plaats toont.
 - Schrijf een programma dat controleert of er een Belgische atlete in de finale zat.
 - Schrijf een programma dat de tijd toont van de laatste finaliste.
 - Schrijf een programma dat telt hoeveel Italiaanse deelneemsters ('ITA') er in de finale zaten.

Test je code in Dodona.

- 2** Maak een matrix van 4 rijen en 4 kolommen, gevuld met de getallen 1 tot en met 16 (rij per rij). Schrijf een programma dat de som berekent van alle getallen op de hoofddiagonaal van de matrix. Test je code in Dodona.
- 3** Schrijf een programma dat een geneste lijst met de resultaten van een klein voetbaltoernooi voorstelt: elke sublijst bevat de naam van een ploeg en het aantal gescoorde doelpunten, bijvoorbeeld ['Rood', 3]. Bepaal met een lus welke ploeg de meeste doelpunten scoorde. Test je code in Dodona.
- 4** Stel een tic-tac-toe-bord voor als een geneste lijst van 3 rijen en 3 kolommen, waarbij elk vakje de waarde 'X', 'O' of ' ' (leeg) heeft. Schrijf een programma dat het bord overzichtelijk toont, rij per rij. Test je code in Dodona.

11.4 Opdrachten

Reeks 1

5 Je krijgt in Dodona de lijst met de gegevens van de finale van de 800 meter bij de vrouwen op het EK atletiek 2026 in Birmingham.

- Overloop de volledige lijst met een lus en toon voor elke deelnemster een nette regel in de vorm **1. Audrey Werro (SUI) 1:54.81**.
- Maak een nieuwe lijst met enkel de namen van de deelnemers, in de volgorde waarin ze gefinisht zijn, maar zonder land en tijd.
- Bepaal welk land het vaakst voorkomt in de uitslag.

Test je code in Dodona.

6 We werken met `klaslijst`, een geneste lijst. Elk element van `klaslijst` ziet er uit als `[naam, leeftijd]`, waarbij `naam` en `leeftijd` respectievelijk een string en een integer zijn. Een mogelijk voorbeeld van een `klaslijst` zou `[['Ella', 17], ['Noah', 18], ['Max', 16]]` kunnen zijn.

- Schrijf een functie `toon_namen(klaslijst)`. Je functie geeft een lijst terug van alle namen in `klaslijst`, alfabetisch gesorteerd.
- Schrijf een functie `toon_leeftijd(naam, klaslijst)`. Je functie geeft de leeftijd terug van `naam`.

Test je code in Dodona.

Reeks 2

7 Je krijgt in Dodona de lijst met de gegevens van de finale van de 800 meter bij de vrouwen op het EK atletiek 2026 in Birmingham. Groepeer de deelnemers per land in een nieuwe geneste structuur: een lijst met daarin voor elk land een lijst van de atletes uit dat land. Test je code in Dodona.

Reeks 3

- 8 Je krijgt in Dodona de lijst met de gegevens van de finale van de 800 meter bij de vrouwen op het EK atletiek 2026 in Birmingham. Maak een nieuwe lijst met daarin de 9 tijdsverschillen tussen de 10 deelnemende atletes. Test je code in Dodona.
- 9 We werken met `rapport`, een geneste lijst. Elk element van `rapport` ziet er uit als `[vak, resultaat, klasgemiddelde]`, waarbij `vak` een string is. `resultaat` en `klasgemiddelde` zijn allebei een integer. Een mogelijk voorbeeld van een `rapport` zou `[['wiskunde', 67, 58], ['fysica', 82, 63], ['informatica', 100, 52], ['Frans', 48, 63]]` kunnen zijn.
- Schrijf een functie `bereken_gemiddelde(rapport)`. Je functie geeft het gemiddelde terug van je eigen resultaten op je rapport.
 - Schrijf een functie `beter_dan_gemiddelde(rapport)`. Je functie geeft een lijst terug met alle vakken waarvan je resultaat beter is dan het gemiddelde van je eigen resultaten.
 - Schrijf een functie `beter_dan_klasgemiddelde(rapport)`. Je functie geeft een lijst terug met alle vakken waarvan je resultaat beter is dan het klasgemiddelde.
 - Schrijf een functie `beste_vak(rapport)`. Je functie geeft het beste vak terug op je rapport. Als je je beste resultaat hebt gehaald op meer dan één vak, geeft je functie een lijst terug met de vakken waarop je je beste resultaat hebt gehaald.

Test je code in Dodona.

- 10 We werken met `puntenboek`, een geneste lijst. Elk element van `puntenboek` ziet er uit als `[naam, lijst]`, waarbij `naam` een string is. `lijst` is een lijst van getallen. De getallen stellen de resultaten (op 10) voor van leerlingen op een aantal evaluaties in een bepaalde periode. Een mogelijk voorbeeld van een `puntenboek` zou `[['Ella', [7.5, 8, 3, 10]], ['Noah', [6, 7, 8.5, 5]], ['Max', [3, 2.5, 6.5, 4]]]` kunnen zijn.
- Schrijf een functie `bereken_gemiddelde(puntenboek)`. Je functie geeft een geneste lijst terug. Elk element van de lijst bevat de naam en het gemiddelde (afgerond op 1 decimaal) dat elke leerling op het rapport zal zien. De volgorde van namen is die van de gegeven lijst `puntenboek`.
 - Schrijf een functie `geslaagd(puntenboek)`. Je functie geeft een lijst terug met de namen van de leerlingen die geslaagd zijn op hun rapport. De volgorde van namen is die van de gegeven lijst `puntenboek`.

11. GENESTE LIJSTEN

- Schrijf een functie `sorteer(rapport)`. Je functie geeft een geneste lijst terug. Elk element van deze lijst bevat de naam en het gemiddelde dat elke leerling op het rapport zal zien. De namen staan nu gesorteerd op gemiddelde: de leerling met het hoogste gemiddelde staat vooraan in de geneste lijst.

Test je code in Dodona.

11 We werken met `bioscoop`, een geneste lijst. Elk element van `bioscoop` ziet er uit als een lijst van strings. De strings kunnen alleen `'vrij'` of `'bezet'` zijn. `bioscoop` stelt dus de bezetting voor in een bioscoopzaal. Een voorbeeld zou `[['vrij', 'bezet', 'vrij'], ['bezet', 'vrij', 'bezet']]` kunnen zijn.

- Schrijf een functie `aantal_rijen(bioscoop)` die het aantal rijen in de bioscoop teruggeeft.
- Schrijf een functie `aantal_zetels_per_rij(bioscoop)` die het aantal zetels op elke rij in de bioscoop teruggeeft.
- Schrijf een functie `aantal_vrije_zetels(bioscoop)` die het totale aantal vrije zetels in de bioscoop teruggeeft.
- Schrijf een functie `vrije_rij(bioscoop)` die `True` of `False` teruggeeft, naargelang er een volledige rij vrij is of niet.
- Schrijf een functie `vrije_rij(bioscoop)` die het grootste aantal vrije zetels op eenzelfde rij teruggeeft.

In het vorige hoofdstuk heb je geleerd dat datastructuren, zoals lijsten en verzamelingen, erg geschikt zijn om bewerkingen uit te voeren op grote hoeveelheden gegevens. Ze zijn daarentegen niet geschikt om die gegevens te bewaren. Om gegevens te bewaren, gebruik je een *bestand*.

In dit laatste hoofdstuk komen volgende vragen aan bod:

- Wat is het verschil tussen het geheugen en de harde schijf van een computer?
- Wat is een bestand precies?
- Welke soorten bestanden bestaan er?
- Hoe kan je in een Python-programma gegevens inlezen vanuit een tekstbestand?
- Hoe kan je in een Python-programma gegevens inlezen vanuit een **csv**-bestand?
- Hoe kan je in een Python-programma gegevens wegschrijven naar een tekstbestand?

12.1 Inleiding

Beknopte inleiding tot de architectuur van een computer

Om precies te begrijpen *waarom* dit laatste hoofdstuk belangrijk is, moet je begrijpen hoe een computer van binnen in elkaar zit. We gebruiken de algemene term *computer*, maar ook tablets en smartphones zijn op precies dezelfde manier opgebouwd. Het is absoluut niet de bedoeling om al te diep in te gaan op allerlei technische details.

Een computer bestaat uit verschillende onderdelen die elk een specifieke taak hebben. De volgende drie onderdelen vormen het geraamte van elk computersysteem:

- a. De **processor** (Central Processing Unit, of ook wel CPU genoemd) is het onderdeel dat effectief *rekent* en *beslist*. Elke instructie die een programma uitvoert (twee getallen optellen, een voorwaarde controleren, een lus herhalen, ...) wordt door de processor verwerkt. Een belangrijke parameter van de processor is de *kloksnelheid*. Deze wordt uitgedrukt in GHz, en geeft aan hoeveel instructies er per seconde verwerkt kunnen worden door de processor.

- b. Het **werkgeheugen** (Random Access Memory, of ook wel RAM-geheugen of kortweg *geheugen* genoemd) bevat de gegevens die de processor nodig heeft om je programma uit te voeren. Als je in je programma een variabele aanmaakt, komt deze dus in het geheugen van de computer te staan.

Het RAM-geheugen van een computer is *vluchtig*. Gegevens worden maar *tijdelijk* bewaard in het geheugen. Als je de computer opnieuw opstart, of als je deze uitschakelt, gaat alle inhoud van het geheugen verloren.

Het RAM-geheugen werkt erg snel, en is daardoor vrij duur. Het RAM-geheugen is bijgevolg relatief beperkt in omvang. Anno 2026 heeft een computer of laptop typisch een RAM-geheugen van 16 GB.

- c. Om gegevens permanent te bewaren, is een computer voorzien van opslagruimte. In het verleden was dit een **harde schijf** (Hard Disc Drive, of HDD). Hoewel de permanente opslagruimte van een computer tegenwoordig bestaat uit een Solid State Drive (SSD), wordt de term *harde schijf* nog vaak gebruikt.

Een harde schijf is *niet vluchtig*. Gegevens blijven bewaard, ook wanneer de computer uitgeschakeld is.

Een harde schijf werkt veel trager dan het RAM-geheugen, maar is veel goedkoper. Anno 2026 heeft een computer of laptop typisch een opslagcapaciteit van 500 GB of meer.

Tegenwoordig worden gegevens steeds vaker bewaard *in the cloud*. De kans is groot dat je zelf gegevens bewaart bij diensten als Google Drive, Microsoft OneDrive of Apple iCloud. Die gegevens staan dan niet fysiek op je eigen harde schijf, maar worden bewaard op een server. Dankzij een snelle internetverbinding kan je werken met deze bestanden alsof ze op je eigen computer staan, terwijl ze in werkelijkheid op een server staan die honderden of duizenden kilometer van je verwijderd is.

RAM-geheugen en opslagcapaciteit worden erg vaak door elkaar gehaald, maar ze zijn fundamenteel verschillend. Vergelijk het met de manier waarop je studeert. Je bureau (RAM-geheugen) is relatief klein, maar je werkt er snel en efficiënt op. Alles wat je nodig hebt om te studeren, leg je erop. Je boekenkast (harde schijf) is veel groter en bevat al je boeken en notities, maar je kan er niet rechtstreeks op schrijven of rekenen. Om te studeren, moet je een document eerst naar je bureau halen.

Belangrijk om te onthouden:

- De processor kan alleen rechtstreeks werken met gegevens die in het werkgeheugen (RAM) staan.
- De processor kan geen gegevens rechtstreeks van een harde schijf lezen.

1 Zoek op hoeveel RAM-geheugen en welke opslagcapaciteit je laptop of smartphone heeft.

Bits en bytes

Menselijke communicatie bestaat uit zinnen. Elke zin bestaat uit woorden en leestekens. Elk woord bestaat dan weer uit afzonderlijke letters. Je zou kunnen zeggen dat letters de kleinste bouwsteen zijn van onze communicatie. De hele tekst die je voor je hebt, is in essentie opgebouwd uit niet meer dan 26 hoofdletters, 26 kleine letters, 10 cijfers en een tiental leestekens.

Digitale informatie is op een andere manier opgebouwd. De kleinste bouwsteen van digitale informatie noemen we een *byte*. Elke byte bestaat uit 8 *bits*. Het woord bit is afkomstig van *binary digit*. Vrij vertaald is een bit dus een binair (tweetallig) cijfer. In een tweetallig rekenstelsel zijn er maar twee cijfers: 0 en 1. Een bit kan dus slechts de waarden 0 en 1 aannemen.

Met 8 bits kan je dus exact $2^8 = 256$ verschillende bytes maken: van `00000000` tot en met `11111111`. In een decimaal (tientallig) systeem komen deze getallen respectievelijk overeen met 0 en 255.

Elke byte komt overeen met één enkel karakter. Dat kan een letter, een cijfer, een leesteken of een spatie zijn. De (uitgebreide) ASCII-code * koppelt (Latijnse) letters, cijfers en leestekens aan een byte. Zo heeft men afgesproken dat de byte `00110000` overeenkomt met het getal 0. De bytes `01000001`, `01100001` en `11100000` stellen respectievelijk de letters A, a en à voor. De byte `00101011` stelt het symbool + voor.

Bestanden

Een datastructuur is in feite een manier waarop gegevens in het RAM-geheugen van een computer voorgesteld worden. Dankzij die voorstelling kan de processor de gegevens in de datastructuur snel en efficiënt verwerken.

Een datastructuur is daarentegen niet geschikt om gegevens op te slaan op de harde schijf van een computer. Gegevens worden op de harde schijf opgeslagen onder de vorm van *bestanden*.

Een bestand is een collectie digitale informatie, voorgesteld door bytes. We maken onderscheid tussen twee soorten bestanden:

*American Standard Code for Information Interchange

- a. Een *tekstbestand* bevat enkel platte tekst, zonder enige lay-out. Een tekstbestand herken je vaak aan de extensie (bv. `.txt` of `.csv`) en kan geopend worden met een eenvoudig programma zoals Kladblok of TextEdit. De inhoud van het bestand kan rechtstreeks gelezen worden door een mens.
- b. Een *binair bestand* is op zichzelf niet leesbaar door een mens, maar kan wel door een computer geïnterpreteerd worden. Je kent zeker bestanden met de extensies `.pdf`, `.jpg`, `.docx` of `.pptx`.

In de rest van dit hoofdstuk zullen we uitsluitend werken met tekstbestanden.

12.2 Gegevens inlezen vanuit een tekstbestand

Instap

- 2** Veronderstel dat er een tekstbestand `Romeo.txt` in dezelfde map staat als het Python-programma dat je schrijft. In dit bestand vind je de beroemde speech die Juliet uitspreekt in Shakespeare's *Romeo and Juliet*:

O Romeo, Romeo, wherefore art thou Romeo?
Deny thy father and refuse thy name.
Or if thou wilt not, be but sworn my love
And I'll no longer be a Capulet.
'Tis but thy name that is my enemy:
Thou art thyself, though not a Montague.
What's Montague? It is nor hand nor foot
Nor arm nor face nor any other part
Belonging to a man. O be some other name.
What's in a name? That which we call a rose
By any other name would smell as sweet;
So Romeo would, were he not Romeo call'd,
Retain that dear perfection which he owes
Without that title. Romeo, doff thy name,
And for that name, which is no part of thee,
Take all myself.

Schrijf een programma dat telt hoe vaak Juliet de naam *Romeo* uitspreekt in deze speech, en dat telt hoeveel lijnen haar speech telt.

Je vindt een mogelijk programma in Dodona.


```
1 # initialiseer de variabelen
2 aantal_lijnen = 0
3 aantal_keer_Romeo = 0
4
5 # lees het bestand 'Romeo.txt' in
6 # de parameter 'r' geeft aan dat we uitsluitend willen lezen in dit
  bestand
7 # lezen = read = 'r'
8 with open('Romeo.txt', 'r') as speech:
9
10     # overloop elke lijn tekst in het bestand
11     for lijn in speech:
12         # verhoog de waarde van aantal_lijnen
13         aantal_lijnen += 1
14
15         # controleer of de string 'Romeo' voorkomt op deze lijn
16         if 'Romeo' in lijn:
17             # verhoog de waarde van aantal_keer_Romeo
18             aantal_keer_Romeo += 1
19
20     # de for-lus is afgewerkt
21     # we hebben de volledige speech van Juliet geanalyseerd
22
23 # toon de resultaten
24 print('Aantal lijnen in de speech:', aantal_lijnen)
25 print('Aantal keer Romeo in de speech:', aantal_keer_Romeo)
```

Voer dit programma uit en noteer de uitvoer:

```
Aantal lijnen in de speech:
Aantal keer Romeo in de speech:
```

Theorie

Op lijn 8 van het programma zie je `with open('Romeo.txt', 'r') as speech:`. De precieze details van deze uitdrukking zijn onbelangrijk, maar in één zin zou je kunnen zeggen dat er hierdoor een datastream op gang gebracht wordt van de harde schijf naar het geheugen van de computer. Hoewel het strikt genomen niet helemaal correct is, mag je in de praktijk redeneren alsof de variabele `speech` de volledige inhoud van het bestand `Romeo.txt` bevat.

De parameter `'r'` op lijn 8 is optioneel. Deze `'r'` staat voor `read` en geeft aan dat we alleen willen lezen in dit bestand. Door de parameter `'r'` mee te geven, is het uitgesloten

dat we ook maar iets aan het bestand zouden veranderen. Deze parameter fungeert dus als veiligheid.

Nu het bestand ingelezen is in het geheugen, kan de processor ermee aan de slag. Op lijn 11 lees je `for lijn in speech:`. Je kent deze constructie uiteraard. Kennelijk kan je de variabele `speech` zien als een collectie van verschillende lijnen tekst. De constructie op lijn 11 itereert over al deze lijnen. De eerste waarde die de variabele `lijn` aanneemt in dit programma, is `'O Romeo, Romeo, wherefore art thou Romeo?'`.

Alle code die één niveau ingesprongen is, wordt uitgevoerd terwijl de datastroom naar `Romeo.txt` bestaat. Wanneer we op lijn 23 terug een niveau opschuiven naar links, wordt de datastroom naar `Romeo.txt` automatisch afgebroken. Je kan nadien niet meer uit de variabele `speech` lezen; het is alsof de inhoud verdwenen is.

12.3 Gegevens inlezen vanuit een csv-bestand

In de vorige opgave bevatte het tekstbestand `Romeo.txt` ongestructureerde tekst: een gedicht, een speech, een lijst getallen. Heel vaak wil je echter gegevens bewaren die uit meerdere *velden* bestaan, bijvoorbeeld de naam van een leerling samen met zijn/haar punten. Dit soort gegevens ken je ongetwijfeld uit een rekenblad zoals Excel: elke rij stelt daar één record voor (bijvoorbeeld één leerling), en elke kolom een veld van dat record (bijvoorbeeld de naam, of de punten).

Een `csv`-bestand (*Comma Separated Values*) is een tekstbestand dat diezelfde tabelvorm voorstelt, maar dan zonder de lay-out van een rekenblad. Een `csv`-bestand bevat enkel de pure gegevens. Elke lijn van het bestand komt overeen met één rij (record). De verschillende velden op die lijn worden gescheiden door een komma, die de plaats van een kolomgrens aangeeft. Zo is meteen duidelijk waar de naam Comma Separated Values vandaan komt.

Je kunt een `csv`-bestand probleemloos openen in Excel of Google Sheets, waar het automatisch als een tabel met kolommen wordt weergegeven.

Een `csv`-bestand is in de eerste plaats bedoeld om door een computer verwerkt te worden, niet om er “mooi” uit te zien voor een mens. Daarom wordt er gewoonlijk geen spatie gezet na een komma in een `csv`-bestand.

Je kunt de gegevens in een `csv`-bestand ook inlezen in Python. We gaan van de volledige tabel een lijst maken. Elke rij in de tabel zal overeenkomen met een element in de lijst.

Instap

- 3** Veronderstel dat er een tekstbestand `puntenlijst.csv` in dezelfde map staat als het Python-programma dat je schrijft. In dit bestand vind je de resultaten van de leerlingen in een klas op één toets:

```
Sophie,15
Naser,12.5
Nele,9
...
Adam,11
```

Schrijf een programma dat het aantal leerlingen in de klas telt, en dat het klasgemiddelde op deze toets berekent.

Je vindt een mogelijk programma in Dodona.

```
1 # bereken en toon het aantal leerlingen
2 # initialiseer de variabelen
3 totaal = 0
4 aantal_leerlingen = 0
5
6 # lees het bestand 'puntenlijst.csv' in
7 with open('puntenlijst.csv', 'r') as bestand:
8
9     # overloop elke lijn tekst in het bestand
10    for lijn in bestand:
11        # zet deze lijn om in een lijst
12        # splits de lijn op bij elke komma
13        # de lijn 'Sophie,15' wordt zo bv. omgezet in de lijst ['Sophie
14        # ', '15']
15        velden = lijn.strip().split(',')
16
17        # de naam van de leerling is het nulde element van de lijst
18        naam = velden[0]          # maar deze hebben we niet nodig in deze
19        opgave
20
21        # het resultaat van de leerling is het eerste element van de
22        lijst velden
23        punten = float(velden[1])
24
25        # verhoog de waarde van totaal met het resultaat van deze
26        leerling
27        totaal += punten
28        # verhoog de waarde van aantal_leerlingen met 1
29        aantal_leerlingen += 1
30
31 # het volledige bestand werd ingelezen en geanalyseerd
32 # de datastroom mag dus afgesloten worden
```

```
29
30 # bereken en toon het aantal leerlingen
31 print('Aantal leerlingen in de klas:', aantal_leerlingen)
32
33 # bereken en toon het klasgemiddelde
34 gemiddelde = totaal / aantal_leerlingen
35 print('Klasgemiddelde:', gemiddelde)
```

Voer dit programma uit en noteer de uitvoer:

```
Aantal leerlingen in de klas:
Klasgemiddelde:
```

Theorie

Op lijn 14 zie je `velden = lijn.strip().split(',')`. Deze uitdrukking doet twee zaken:

- In eerste instantie wordt de methode `strip()` toegepast op het object `lijn`. `lijn` is gedefinieerd op regel 10, waar het tekstbestand lijn per lijn wordt ingelezen. `lijn` is dus bv. de string `'Sophie, 15'`.

De methode `strip()` verwijdert eventuele ongewenste spaties bij het begin en op het einde van elke lijn. Eventuele spaties in het midden van de lijn blijven onaangeroerd. Belangrijker is echter dat `strip()` een nette manier is om rekening te houden met de overgang van een regel naar een volgende.

Toegepast op de string `'Sophie,15'` lijkt het alsof `strip()` niets doet: de uitvoer is precies gelijk aan de invoer, `'Sophie,15'`. Toegepast op de string `' Sophie,15 '` zie je wel effect: de uitvoer is gelijk aan `'Sophie,15'`.

- De methode `split(',')` wordt nu toegepast op het resultaat van de eerste stap. Deze methode hakt een string op in losse stukjes, waarbij de parameter aangeeft op welke manier twee stukken van elkaar gescheiden moeten worden. In onze situatie is het een komma (,) die als scheiding moet worden gezien tussen twee stukjes.

Het resultaat van deze methode is een lijst van strings. De komma's die de scheiding aangeven tussen twee stukken verdwijnen zelf bij deze actie.

De uitdrukking `'Sophie,15'.split(',')` geeft dus de lijst `['Sophie', '15']` terug.

Op lijnen 17 en 20 worden de naam van de leerling en diens resultaat uit deze lijst gehaald. Merk op dat het element op index 1 (bv. `'15'`) altijd een string is. Aangezien we willen werken met de getalwaarde van deze string, en het resultaat mogelijk een kommagetal kan zijn, moeten we deze omzetten naar een `float`.

12.4 Gegevens wegschrijven naar een tekstbestand

Tot nu toe hebben we uitsluitend gegevens ingelezen uit een tekstbestand. Soms kan het ook handig zijn om informatie te kunnen wegschrijven naar een tekstbestand.

Instap

- 4** Veronderstel dat er een tekstbestand `puntenlijst.csv` in dezelfde map staat als het Python-programma dat je schrijft. In dit bestand vind je de resultaten van de leerlingen in een klas op meerdere toetsen. Je mag aannemen dat alle toetsen op 20 gequoteerd zijn.

```
Sophie,15,11,18,12
Naser,12.5,15,11,8
Nele,9,15,4.5,10
...
Adam,11,13,10,6
```

Schrijf een programma dat het gemiddelde resultaat van al deze leerlingen berekent. Het gemiddelde wordt afgerond op één cijfer na de komma. Al deze resultaten worden weggeschreven naar een tekstbestand `rapportcijfers.csv`. Als het gemiddelde na afronding minstens 10/20 is, wordt er in een derde kolom `'geslaagd'` toegevoegd. Als dat niet het geval is, wordt er `'niet geslaagd'` toegevoegd.

Voor de opgegeven puntenlijst zou de uitvoer de volgende zijn:

```
Sophie,14.0,geslaagd
Naser,11.6,geslaagd
Nele,9.6,niet geslaagd
...
Adam,10.0,geslaagd
```

Je vindt een mogelijk programma in Dodona.

```
1 # open een datastroom naar het bestand puntenlijst.csv
2 # in dit bestand willen we alleen maar lezen, dus we geven de parameter
  'r' mee
3 # we verwijzen naar deze eerste datastroom als invoer
4
5 # open daarnaast ook datastroom naar het bestand rapportcijfers.csv
6 # we willen gegevens wegschrijven naar dit bestand
7 # we geven dus de parameter 'w' (write) mee
8 # we verwijzen naar deze tweede datastroom als uitvoer
```

```
9
10 with open('puntenlijst.csv', 'r') as invoer, open('rapportcijfers.csv',
    'w') as uitvoer:
11     # overloop elke lijn tekst in het bestand
12     for lijn in invoer:
13         # zet deze lijn om in een lijst van velden
14         # splits de lijn op bij elke komma
15         # de lijn 'Sophie,15,11,18,12' wordt zo omgezet in de lijst ['
Sophie', '15', '11', '18', '12']
16         velden = lijn.strip().split(',')
17
18         # de naam van de leerling is het nulde element van de lijst
velden
19         naam = velden[0]          # hebben we nu wel nodig!
20
21         # de punten van deze leerling zijn alle overige elementen van de
lijst velden
22         punten = velden[1:]
23
24         # bereken de som van alle punten van deze leerling
25         som = 0
26         for punt in punten:
27             som += float(punt)
28
29         # bereken het gemiddelde, afgerond op 1 cijfer na de komma
30         gemiddelde = som / len(punten)
31         gemiddelde = round(gemiddelde, 1)
32
33         # ga na of de leerling geslaagd is, op basis van het afgeronde
gemiddelde
34         if gemiddelde >= 10:
35             resultaat = 'geslaagd'
36         else:
37             resultaat = 'niet geslaagd'
38
39         # bouw de nieuwe lijn op die uitgevoerd moet worden
40         nieuwe_lijn = naam + ',' + str(gemiddelde) + ',' + resultaat
41         # schrijf deze lijn uitvoer weg via de datastroom uitvoer naar
rapportcijfers.csv
42         uitvoer.write(nieuwe_lijn + '\n')
```

Theorie

Op lijn 10 zie je `with open('puntenlijst.csv', 'r') as invoer, open('rapportcijfers.csv', 'w') as uitvoer`. In tegenstelling tot de vorige opgaves, waarbij we telkens slechts één bestand openden, worden hier *twee* bestanden tegelijk geopend binnen dezelfde `with`-constructie. Dat kan door de twee `open()`-aanroepen van elkaar te scheiden met een komma. Beide bestanden worden hierdoor automatisch gesloten zodra het ingesprongen blok eindigt, net zoals bij één enkel bestand.

Merk op dat de twee bestanden op een verschillende manier geopend worden:

- `open('puntenlijst.csv', 'r')` opent `puntenlijst.csv` met de parameter `'r'` (*read*): we willen enkel *lezen* uit dit bestand.
- `open('rapportcijfers.csv', 'w')` opent `rapportcijfers.csv` met de parameter `'w'` (*write*): we willen *schrijven* naar dit bestand.

Er worden dus twee verschillende datastromen op gang gebracht, in tegengestelde richting. De ene brengt gegevens van de harde schijf naar het geheugen (`invoer`), de andere brengt gegevens van het geheugen naar de harde schijf (`uitvoer`). Beide datastromen lopen gelijktijdig, zolang we ons binnen het `with`-blok bevinden.

Als `rapportcijfers.csv` nog niet bestaat, wordt het bestand automatisch aangemaakt door de parameter `'w'`. Bestaat het bestand wel al (bijvoorbeeld van een vorige keer dat je het programma uitvoerde), dan wordt de volledige inhoud ervan gewist vóór er iets nieuws wordt weggeschreven.

Op lijn 16 zie je opnieuw `velden = lijn.strip().split(',')`. Voor de lijn `'Sophie,15,11,18,12'` levert dit de lijst `['Sophie', '15', '11', '18', '12']` op.

Op lijn 22 zie je `punten = velden[1:]`. De lijst `punten` bevat dus alle elementen van de lijst `velden`, vanaf index 1 tot het einde. Toegepast op onze lijst levert dit `['15', '11', '18', '12']` op. Om op lijn 30 het gemiddelde te berekenen, moeten we weten hoeveel evaluaties er waren. Dat aantal is gelijk aan `len(punten)`.

Op lijn 40 wordt ten slotte de nieuwe lijn opgebouwd met `+`, de operator voor het samenvoegen (concateneren) van strings. Merk op dat `str(gemiddelde)` noodzakelijk is: `gemiddelde` is een getal (een `float`), en de `+`-operator laat niet toe om een string en een getal rechtstreeks samen te voegen.

Op lijn 42 zie je `uitvoer.write(nieuwe_lijn + '\n')`. De methode `.write()` schrijft een string weg naar het bestand dat aan `uitvoer` verbonden is. In tegenstelling tot `print()`, die automatisch een nieuwe lijn start na elke aanroep, zou `.write()` alle lijnen na elkaar op één en dezelfde regel zetten in `rapportcijfers.csv`. Om ervoor te zorgen dat elke lijn uitvoer ook op een aparte lijn terechtkomt in `rapportcijfers.csv`, voegen we op lijn 42 expliciet het teken `'\n'` toe, vóór deze wordt weggeschreven. Het teken `'\n'` is een *newline-teken* (letterlijk: “nieuwe lijn”). Het is een speciaal teken dat aangeeft waar één lijn eindigt en de volgende begint.

12.5 Overzicht

Je gebruikt bestanden wanneer je gegevens permanent wilt bewaren, ook nadat je programma gestopt is. In dit hoofdstuk hebben we het onderscheid tussen **geheugen** en **harde schijf** behandeld, en geleerd hoe je gegevens kan **inlezen** en **wegschrijven** met Python.

Computerarchitectuur

Eigenschap	RAM-geheugen	Harde schijf / SSD
Vluchtig (tijdelijk)	ja	nee
Snelheid	snel	traag
Capaciteit	beperkt	groot
Rechtstreeks toegankelijk door de processor	ja	nee

- De processor kan enkel rechtstreeks werken met gegevens in het RAM-geheugen.
- Om gegevens permanent te bewaren, moeten ze weggeschreven worden naar de harde schijf.

Bits en bytes

- Een **bit** is de kleinste eenheid van digitale informatie, en kan enkel de waarde 0 of 1 aannemen.
- Een **byte** bestaat uit 8 bits, en kan dus $2^8 = 256$ verschillende waarden aannemen.
- De (uitgebreide) **ASCII-code** koppelt elke byte aan een letter, cijfer, leesteken of ander karakter.

Soorten bestanden

- Een **tekstbestand** bevat platte tekst, zonder lay-out, en is rechtstreeks leesbaar door een mens (bv. `.txt`, `.csv`).
- Een **binair bestand** is niet rechtstreeks leesbaar door een mens, maar wel interpreteerbaar door een computer (bv. `.pdf`, `.jpg`).

Een tekstbestand inlezen

```
1 with open('bestand.txt', 'r') as naam:
2     for lijn in naam:
3         ...
```

- De parameter **'r'** staat voor *read*. Dit geeft aan dat je enkel wilt lezen uit het bestand.
- De constructie **with ... as ...**: opent een datastroom van de harde schijf naar het geheugen, en sluit deze automatisch af zodra het ingesprongen blok eindigt (ook als er een fout optreedt).
- Met **for lijn in naam**: itereer je over de afzonderlijke lijnen van het bestand.
- Elke **lijn** eindigt op een **newline-teken** (**'\n'**), behalve mogelijk de laatste lijn.
- Met **lijn.strip()** worden het newline-teken en eventuele overtollige spaties vooraan of achteraan verwijderd.

Een csv-bestand inlezen

Een csv-bestand stelt een tabel voor. Elke lijn is een record. De verschillende velden op een lijn worden gescheiden door een komma.

```
1 with open('bestand.txt', 'r') as naam:
2     for lijn in naam:
3         velden = lijn.strip().split(',')
4         ...
```

- De methode **split(',')** splitst een string op telkens wanneer er een komma voorkomt, en geeft een lijst van de afzonderlijke velden terug.
- Elk veld is een **string**, ook als het er als een getal uitziet.

Wegschrijven naar een tekstbestand

```
1 with open('bestand.txt', 'w') as naam:
2     naam.write(tekst + '\n')
3     ...
```

- De parameter **'w'** staat voor *write*. Dit geeft aan dat je wilt schrijven naar het bestand. Bestaat het bestand nog niet, dan wordt het aangemaakt. Bestaat het bestand wel al, dan wordt de inhoud ervan eerst gewist.

- Met de methode `write(x)` wordt de string `x` weggeschreven naar het bestand.
- De methode `write()` voegt, in tegenstelling tot de functie `print()`, zelf **geen** newline-teken toe. Je moet `'\n'` dus expliciet toevoegen als elke lijn op een aparte regel moet komen.

Meerdere bestanden tegelijk

- Je kunt verschillende bestanden tegelijk openen binnen dezelfde **with**-constructie, door de `open()`-aanroepen te scheiden met een komma. Zo kan je bijvoorbeeld tegelijk lezen uit het ene bestand en schrijven naar een ander.

```
1 with open('invoer.txt', 'r') as invoer, open('uitvoer.txt', 'w') as
   uitvoer:
2     ...
```

Veelgemaakte fouten

- Denk eraan dat elk veld uit een `csv`-bestand een **string** is, ook al ziet het er als een getal uit.
- Vergeet niet `.strip()` toe te passen, waardoor het newline-teken (`'\n'`) mee in het laatste veld van een lijn terechtkomt.
- Voeg `'\n'` toe bij `.write()` om alle lijnen op één regel te laten terechtkomen.
- Pas de functie `str()` toe op een getal vooraleer het met `+` samen te voegen met een string.

13.1 Instap

- 1** Je wilt een trap met n treden beklimmen. Je kunt telkens 1 of 2 treden tegelijk nemen. Hoeveel verschillende manieren zijn er om de trap te beklimmen?

We proberen deze opgave op te lossen voor kleine waarden van n :

- Voor $n = 0$:
- Voor $n = 1$:
- Voor $n = 2$:
- Voor $n = 3$:
- Voor $n = 4$:

Door te redeneren met kleine waarden van n en deze stelselmatig groter te laten worden, zie je dat er een bepaald patroon verscholen zit in de oplossing. Meer bepaald vind je voor een trap met n treden het aantal manieren als de som van het aantal manieren om een trap met $n - 1$ treden te beklimmen, en het aantal manieren om een trap met $n - 2$ treden te beklimmen.

Stel dat we een functie `aantal_manieren_trap(n)` zouden hebben die het antwoord geeft op deze opgave, dan zouden we dus kunnen schrijven dat

```
aantal_manieren_trap(n) = aantal_manieren_trap(n-1) +  
aantal_manieren_trap(n-2).
```

De oplossing van ons probleem is dus de som van de oplossingen van twee eenvoudigere problemen. Je zou dan kunnen denken dat we het probleem alleen maar verschuiven, want als we `aantal_manieren_trap(n)` niet kennen, hoe zouden we `aantal_manieren_trap(n-1)` en `aantal_manieren_trap(n-2)` dan wel kunnen berekenen?

Om deze vraag te beantwoorden, moet je je realiseren dat n een natuurlijk getal is. Er bestaat geen grootste natuurlijk getal, maar de verzameling $\mathbb{N}$ is wel naar onder begrensd. 0 en 1 zijn de twee kleinste natuurlijke getallen.

Dat alles maakt dat we de functie `aantal_manieren_trap(n)` op een subtiele manier kunnen implementeren:

```
1 def aantal_manieren_trap(n):  
2     # n is een natuurlijk getal  
3     if n <= 1:  
4         # voor een trap met 0 of 1 trede is het probleem al opgelost  
5         # er is maar 1 manier om een trap met 0 of 1 trede te beklimmen  
6         return 1  
7     else:  
8         # het aantal trappen is minstens 2  
9         # we noteren het patroon dat we ingezien hadden  
10        return aantal_manieren_trap(n - 1) + aantal_manieren_trap(n -  
11        2)  
12 print(aantal_manieren_trap(0))
```

Kopieer de code en voer het programma uit in de Sandbox. Pas daarna de code aan en noteer de uitvoer voor volgende waarden van n :

- `aantal_manieren_trap(0) =`

- `aantal_manieren_trap(1) =`

- `aantal_manieren_trap(2) =`

- `aantal_manieren_trap(3) =`

- `aantal_manieren_trap(4) =`

- `aantal_manieren_trap(10) =`
- `aantal_manieren_trap(20) =`
- `aantal_manieren_trap(35) =`

Je merkt dat de benodigde rekentijd snel toeneemt met n . Om `aantal_manieren_trap(35)` te berekenen, heeft Dodona misschien wel een halve minuut of meer nodig. Mogelijk krijg je de uitvoer zelfs nooit te zien omdat de uitvoering van je programma automatisch beëindigd wordt na een bepaalde tijd.

13.2 Theorie

Recursie is een algoritmische techniek waarbij een functie zichzelf aanroept om een probleem op te lossen. De achterliggende idee is dat je een groot probleem opsplijt in kleinere versies van hetzelfde probleem. Je blijft dit herhalen tot je een eenvoudig geval bereikt dat je direct (zonder recursie) kunt oplossen. De naam recursie verwijst naar het Latijnse werkwoord *re-currere*, dat *teruglopen* betekent.

Elke recursieve functie bestaat uit twee essentiële onderdelen:

- Basisstap* (stopvoorwaarde). Dit is het eenvoudigste geval dat je direct kunt oplossen zonder recursie toe te passen.
- Recursiestap*. In deze stap roept de functie zichzelf aan met een eenvoudigere versie van het probleem.

De basisstap is hierbij cruciaal. Zonder basisstap zou je programma nooit eindigen. Je komt immers terecht in een eindeloze lus. Voor het inleidend voorbeeld van Opgave 1:

- `aantal_manieren_trap(2) = aantal_manieren_trap(1) + aantal_manieren_trap(0)`
- `aantal_manieren_trap(1) = aantal_manieren_trap(0) + aantal_manieren_trap(-1)`
- ...
- `aantal_manieren_trap(-1000) = aantal_manieren_trap(-1001) + aantal_manieren_trap(-1002)`
- ...

13. RECURSIE

Visueel kunnen we de uitvoering van een recursief gedefinieerde functie weergeven in een *aanroepboom*:

```
aantal_manieren_trap(4)
├── aantal_manieren_trap(3)
│   ├── aantal_manieren_trap(2)
│   │   ├── aantal_manieren_trap(1)
│   │   └── aantal_manieren_trap(0)
│   └── aantal_manieren_trap(1)
└── aantal_manieren_trap(2)
    ├── aantal_manieren_trap(1)
    └── aantal_manieren_trap(0)
```

Je ziet dat sommige berekeningen, zoals die van `aantal_manieren_trap(2)`, dubbel gebeuren. Voor grotere waarden van n wordt dit probleem alleen maar groter. Zo wordt bij de uitvoering van `aantal_manieren_trap(10)` maar liefst 34 keer de waarde van `aantal_manieren_trap(2)` berekend. Bij `aantal_manieren_trap(20)` wordt de waarde van `aantal_manieren_trap(2)` 4181 keer berekend, en bij `aantal_manieren_trap(35)` is dit maar liefst 5 702 887 keer.

Recursie is dus een elegante manier om sommige problemen op te lossen, maar je ziet dat de uitvoering van een recursief gedefinieerde functie niet efficiënt gebeurt. In vergelijking met een iteratief gedefinieerde functie zal de uitvoeringstijd van een recursief gedefinieerde functie vaak veel sneller toenemen met n .*

Recursie is dus zeker niet geschikt voor alle problemen, maar dat neemt niet weg dat er wel degelijk erg efficiënte recursieve algoritmes bestaan. Recursie is een ideale techniek voor het doorlopen van structuren die uit onderdelen bestaan die zelf opnieuw dezelfde structuur hebben. Een aantal concrete toepassingen hiervan zijn:

- Een bestand zoeken in een mappenstructuur. Om alle bestanden in een map en alle onderliggende mappen te doorlopen, wordt een recursief proces gebruikt.
- De oplossing zoeken van een sudoku door alle mogelijkheden te overlopen.
- In de Griekse mythologie hielp Ariadne haar geliefde Theseus om uit het labirint te ontsnappen nadat hij de Minotaurus had verslagen. Ze gaf hem een draad die hij moest uitrollen terwijl hij het labirint binnenging, zodat hij de weg terug naar buiten kon vinden. Hoewel Theseus in letterlijke zin geen functie opriep, komt de achterliggende gedachte van Ariadne's strategie wel overeen met wat wij nu recursie noemen.

*Uiteraard bestaan er manieren om de rekentijd te beperken, maar dat gaat te ver voor deze cursus.

13.3 Opdrachten

Reeks 1

- 2** Je kunt op verschillende manieren de som berekenen van twee natuurlijke getallen.
- a. Schrijf een functie `som_direct(a, b)` die simpelweg $a + b$ berekent en teruggeeft.
 - b. Schrijf een functie `som_iteratief(a, b)` die de som van a en b berekent en teruggeeft op een iteratieve manier.
 - c. Schrijf een functie `som_rekursief(a, b)` die de som van a en b berekent en teruggeeft op een recursieve manier.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 3** Je kunt op verschillende manieren de macht berekenen van twee getallen.
- a. Schrijf een functie `macht_direct(a, b)` die simpelweg a^b berekent en teruggeeft.
 - b. Schrijf een functie `macht_iteratief(a, b)` die a^b berekent en teruggeeft op een iteratieve manier.
 - c. Schrijf een functie `macht_rekursief(a, b)` die a^b berekent en teruggeeft op een recursieve manier.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 4** In de wiskunde is de faculteit van een natuurlijk getal n gedefinieerd als volgt:

$$n! = n \cdot (n - 1) \cdot \dots \cdot 3 \cdot 2 \cdot 1.$$

- a. Schrijf een functie `faculteit_iteratief(n)` die op een iteratieve manier $n!$ berekent en teruggeeft.
- b. Schrijf een functie `faculteit_rekursief(n)` die op een recursieve manier $n!$ berekent en teruggeeft.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 5** De rij van Fibonacci is wellicht het typevoorbeeld van een wiskundige rij die heel gemakkelijk recursief kan worden gedefinieerd. Als we het n -de getal van Fibonacci noteren als F_n , kan je de rij van Fibonacci formeel definiëren als volgt:

13. RECURSIE

- de eerste en de tweede term van de rij zijn gelijk aan 1:

$$F_1 = F_2 = 1.$$

- elke volgende term is gelijk aan de som van de twee voorgaande termen:

$$F_n = F_{n-1} + F_{n-2} \quad (n \geq 3).$$

De rij van Fibonacci bestaat dus uit de getallen 1, 1, 2, 3, 5, 8, 13, 21, ...

- Schrijf een functie `fibonacci_iteratief(n)` die op een iteratieve manier F_n berekent en teruggeeft.
- Schrijf een functie `fibonacci_rekursief(n)` die op een recursieve manier F_n berekent en teruggeeft.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 6** We beschouwen de rij van Tribonacci, een variant op de rij van Fibonacci. Deze rij heeft drie variabelen a , b en c . Als we het n -de getal van Tribonacci noteren als T_n , kan je deze rij formeel definiëren als volgt:

- $T_1 = a$
- $T_2 = b$
- $T_3 = c$
- $T_n = T_{n-1} + T_{n-2} + T_{n-3} \quad (n \geq 4).$

Schrijf een functie `tribonacci_rekursief(a,b,c,n)` die op een recursieve manier T_n berekent en teruggeeft. Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 7** De Torens van Hanoi is een puzzelspel dat gespeeld wordt met een aantal ronde schijven. Het spel bestaat uit een plankje met daarop drie stokjes. Bij aanvang van het spel is op één van de stokjes een kegelvormige toren geplaatst van ronde schijven met een gat in het midden. De schijven hebben verschillende diameters. Ze zijn zo geplaatst dat er geen grotere schijf op een kleinere schijf ligt.

Het doel van het spel is om de complete toren van schijven te verplaatsen naar een ander stokje. Daarbij moeten de volgende regels in acht genomen worden:

- Er mag slechts 1 schijf tegelijk worden verplaatst.
- Er mag nooit een grotere schijf op een kleinere liggen.

Schrijf een functie `Hanoi(n)` die op een recursieve manier berekent en teruggeeft hoeveel stappen er nodig zijn om een toren van `n` schijven te verplaatsen. Misschien helpt het om het spel een paar keer online te spelen met verschillende waarden van `n`.

- 8** De oud-Griekse wiskundige Euclides heeft rond 300 v.C. een algoritme bedacht om de grootste gemene deler (ggd) van twee getallen te berekenen. Het algoritme steunt op het herhaaldelijk toepassen van de eigenschap:

De grootste gemene deler van `grootste_getal` en `kleinste_getal` is gelijk aan de grootste gemene deler van `kleinste_getal` en de rest bij deling van `grootste_getal` door `kleinste_getal`.

Voorbeeld 1:

$$\begin{aligned}
 \text{ggd}(372, 752) &= \text{ggd}(752, 372) \\
 &= \text{ggd}(372, 8) && \text{want } 752 = 2 \cdot 372 + 8 \\
 &= \text{ggd}(8, 4) && \text{want } 372 = 46 \cdot 8 + 4 \\
 &= \text{ggd}(4, 0) && \text{want } 8 = 2 \cdot 4 + 0 \\
 &= 4 && \text{want elk getal is een deler van nul}
 \end{aligned}$$

- Gebruik dit algoritme om de grootste gemene deler te bepalen van 1071 en 462.
- Schrijf een functie `euclides_iteratief(a, b)` die op een iteratieve manier de grootste gemene deler van `a` en `b` berekent en teruggeeft.
- Schrijf een functie `euclides_rekursief(a, b)` die op een recursieve manier de grootste gemene deler van `a` en `b` berekent en teruggeeft.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 9** Een palindroom is een woord dat hetzelfde wordt gelezen van voor naar achter als van achter naar voor. Voorbeelden zijn *pop*, *raar* of *lepel*.

Schrijf een functie `is_palindroom(woord)` die op een recursieve manier nagaat of `woord` een palindroom is. Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

Hints:

- `woord[0]` en `woord[1]` verwijzen respectievelijk naar wat wij het eerste en tweede karakter van `woord` noemen.

13. RECURSIE

- Analoot verwijzen `woord[-1]` en `woord[-2]` respectievelijk naar het laatste en het voorlaatste karakter van `woord`.
- `woord[a:b]` verwijst naar het deel van `woord` dat begint bij het `a`-de karakter, en dat stopt vlak voor het `b`-de karakter van `woord`. `'palindroom'[2:5]` geeft dus `'lin'` terug.

10 Marie wilt een bepaald bedrag betalen. Ze heeft een ruime voorraad munten van 1 en 2 en biljetten van 5 euro. Schrijf een functie `aantal_manieren_betalen(bedrag)` die op een recursieve manier berekent en teruggeeft op hoeveel manieren ze `bedrag` kan betalen. Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

14.1 Inleiding

Auteurs van handboeken en leerkrachten zorgen er vaak voor dat opgaves mooi uitkomen. Je kent het ongetwijfeld:

- de discriminant van een tweedegraadsvergelijking is zo goed als altijd een volkomen kwadraat;
- oplossingen van een goniometrische vergelijking zijn typisch rationale veelvouden van π ;
- de stroom door en de spanning over een weerstand komen netjes uit op 50 mA en 6,0 V;
- je hebt precies 2 mol nodig van een stof om een bepaalde reactie volledig te laten opgaan; ...

Vanuit didactisch standpunt is er zeker iets te zeggen voor deze aanpak. Als je antwoord niet mooi uitkomt, is de kans erg groot dat je oplossing niet helemaal juist is. Je kan dan je redenering nog eens nakijken, of controleren of je niet ergens een rekenfout gemaakt hebt. Zo kan je alsnog het correcte antwoord vinden, en tonen dat je de leerstof onder de knie hebt.

In de “echte” wereld is wiskunde de taal waarmee een model van de (fysische / chemische / economische / ...) werkelijkheid wordt beschreven. De werkelijkheid houdt natuurlijk geen rekening met de oplosbaarheid van het wiskundig model.

Een voorbeeld maakt dit duidelijk. Stel dat je wil berekenen hoe laat de Zon opgaat op een willekeurige dag, en op een willekeurige plaats op Aarde. In principe hoef je alleen maar volgende vergelijking op te lossen:

$$\cos H = -\tan \phi \cdot \tan \delta.$$

Hierbij is δ de declinatie van de Zon op de bewuste dag. ϕ is de breedteligging van de plaats. Je lost deze vergelijking op naar de hoek H , die dan weer omgezet kan worden naar een tijdstip.

Het slechte nieuws is dat deze vergelijking onmogelijk algebraïsch op te lossen is voor willekeurige waarden van δ en ϕ . Tegenwoordig is dat geen probleem: je berekent H heel gemakkelijk met een wetenschappelijke rekenmachine. Je lost de vergelijking dus niet algebraïsch, maar numeriek op.

Bedenk dan dat wetenschappelijke rekenmachines nog maar bestaan sinds halverwege de jaren 1970. Toch konden astronomen al eeuwen geleden berekenen wanneer de Zon zou opkomen. Zij hadden natuurlijk geen wetenschappelijke rekenmachine ter beschikking, maar gebruikten numerieke technieken om de vergelijking op te lossen.

14.2 Wat is numerieke wiskunde?

Numerieke wiskunde is de tak van de wetenschap die zich bezighoudt met het ontwikkelen van algoritmes om analytische problemen op te lossen. Denk daarbij aan volgende problemen:

- het oplossen van een vergelijking;
- het oplossen van een stelsel vergelijkingen;
- het bepalen van een extremum van een functie;
- het berekenen van een bepaalde integraal;
- ...

Je staat er misschien niet bij stil, maar een alledaagse toepassing als het voorspellen van het weer steunt volledig op het numeriek oplossen van een bijzonder complex wiskundig model.

14.3 Numerieke wiskunde in deze cursus

In dit hoofdstuk beperken we ons tot twee domeinen:

- het numeriek oplossen van een vergelijking;
- het numeriek oplossen van een bepaalde integraal.

We kiezen voor deze toepassingen om twee redenen. Ten eerste sluiten ze perfect aan op de wiskunde van de derde graad. Een tweede reden is dat er algoritmes bestaan die kunnen geïmplementeerd worden met de opgedane kennis van dit handboek.

Numerieke wiskunde is een specialiteit op zichzelf. Het is expliciet niet de bedoeling om in te gaan op aspecten als afrondingsfouten, nauwkeurigheid van de resultaten, of de efficiëntie van het algoritme.

14.4 Opdrachten

Algoritmes om een vergelijking op te lossen

1 De **bisectiemethode** is een eenvoudig maar toch erg betrouwbaar numeriek algoritme om de oplossing te vinden van de vergelijking $f(x) = 0$. De achterliggende idee van de bisectiemethode is dat we herhaaldelijk het interval halveren waarin met zekerheid een oplossing van de vergelijking te vinden is. Als we deze procedure maar vaak genoeg blijven herhalen, bekomen we een erg klein interval waarin de gezochte oplossing ligt.

Om zeker te zijn dat er een oplossing bestaat in een interval $[a, b]$, eisen we dat $f(a) \cdot f(b) < 0$. $f(a)$ en $f(b)$ hebben dan een verschillend teken. We eisen daarnaast dat $f(x)$ continu is in $[a, b]$. Als aan deze beide voorwaarden voldaan is, weten we zeker dat er minstens één oplossing van $f(x) = 0$ ligt in $[a, b]$.

Het algoritme verloopt als volgt:

- Bereken c , het midden van het interval $[a, b]$.
- Als $f(a)$ en $f(c)$ hetzelfde teken hebben, ben je niet zeker dat de vergelijking $f(x) = 0$ een oplossing heeft in $[a, c]$. Je moet dus verder zoeken in $[c, b]$.
- Als $f(a)$ en $f(c)$ een verschillend teken hebben, ligt de oplossing in $[a, c]$. Je moet dus verder zoeken in $[a, c]$.
- De breedte van het interval waarin je zoekt, halveert dus in elke stap. Na 10 stappen is de breedte nog $1/1024$ van het oorspronkelijke interval. Na 30 stappen heb je bij benadering nog maar $1/10^9$ van het oorspronkelijke interval. Als je begint met een interval van breedte 1, heb je na een 30-tal stappen 9 decimalen van de gezochte oplossing.
- Blijf deze methode toepassen tot c gelijk wordt aan de gezochte waarde van x .
- Het probleem is dat de vergelijking $f(x) = 0$ in het algemeen geen exacte oplossing heeft. c nadert naar x , en dus nadert $f(c)$ ook naar $f(x) = 0$. Je zal dus niet altijd een waarde van c kunnen vinden waarvoor $f(c)$ exact gelijk is aan nul.

- We stellen ons daarom tevreden wanneer $f(c)$ in absolute waarde kleiner is dan een waarde e (die we de tolerantie noemen), met e een willekeurig klein positief getal. Pas als dat het geval is, mag de lus onderbroken worden. De waarde van c op dat moment is dan een benadering voor de oplossing van de vergelijking $f(x) = 0$.
- a. Schrijf een functie `midden(a, b)` die het midden van interval $[a, b]$ teruggeeft.
 - b. Schrijf een functie `bisectie_iteratief(f, a, b, e)` die de vergelijking $f(x) = 0$ numeriek oplost door de bisectiemethode op een iteratieve manier toe te passen in het interval $[a, b]$ met tolerantie e .
 - c. Schrijf een functie `bisectie_rekursief(f, a, b, e)` die de vergelijking $f(x) = 0$ numeriek oplost door de bisectiemethode op een recursieve manier toe te passen in het interval $[a, b]$ met tolerantie e .

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 2** Net als de bisectiemethode is ook de **Regula falsi** een algoritme om een nulwaarde van een functie $f(x)$ te zoeken in een interval $[a, b]$. Opnieuw geldt dat $f(a)$ en $f(b)$ steeds een tegengesteld teken moeten hebben ($f(a) \cdot f(b) < 0$), en dat $f(x)$ continu moet zijn in $[a, b]$. Als aan deze beide voorwaarden voldaan is, weten we zeker dat er minstens één oplossing van $f(x) = 0$ ligt in $[a, b]$.

Het algoritme verloopt als volgt:

- Bepaal het voorschrift van de rechte r door de punten $A(a, f(a))$ en $B(b, f(b))$. Aangezien $f(a)$ en $f(b)$ een tegengesteld teken hebben, moet r de x -as snijden.
 - Bepaal de x -coördinaat van het snijpunt $C(c, 0)$ van de rechte r met de x -as.
 - Bereken $f(c)$. Als $f(a)$ en $f(c)$ hetzelfde teken hebben, ben je niet zeker dat de vergelijking $f(x) = 0$ een oplossing heeft in $[a, c]$. Je moet dus verder zoeken in $[c, b]$.
 - Als $f(a)$ en $f(c)$ een verschillend teken hebben, heeft $f(x)$ zeker minstens één nulwaarde in $[a, c]$. Je moet dus verder zoeken in $[a, c]$.
 - Blijf deze methode toepassen tot wanneer je een waarde van c gevonden hebt waarvoor $f(c)$ in absolute waarde kleiner is dan de tolerantie e . De waarde van c op dat moment is dan een benadering voor de oplossing van de vergelijking $f(x) = 0$.
- a. Neem een blad papier en noteer het voorschrift van de rechte r door de punten $A(a, f(a))$ en $B(b, f(b))$. Stel dit voorschrift gelijk aan nul, en los op naar x . Deze uitdrukking bepaalt de waarde van c in elke stap van het algoritme. Schrijf vervolgens een functie `bepaal_c(f, a, b)` die de waarde van c berekent en teruggeeft.

- b. Schrijf een functie `regula_falsi_iteratief(f, a, b, e)` die de vergelijking $f(x) = 0$ numeriek oplost door het regula falsi-algoritme op een iteratieve manier toe te passen in het interval $[a, b]$ met tolerantie e .
- c. Schrijf een functie `regula_falsi_rekursief(f, a, b, e)` die de vergelijking $f(x) = 0$ numeriek oplost door het regula falsi-algoritme op een recursieve manier toe te passen in het interval $[a, b]$ met tolerantie e .

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

Algoritmes om een bepaalde integraal te berekenen

3 Je hebt bij wiskunde geleerd dat je een bepaalde integraal kan beschouwen als de som van een oneindig aantal termen. De Duitse wiskundige Bernhard Riemann (1826-1866) bedacht immers de volgende methode om een benadering te vinden voor de (georiënteerde) oppervlakte onder de grafiek van $f(x)$ in het interval $[a, b]$:

- verdeel het interval $[a, b]$ in n gelijke deelintervallen met breedte Δx ;
- kies een willekeurige waarde x_i in het i -de deelinterval;
- bereken de functiewaarde $f(x_i)$;
- $\sum_{i=1}^n f(x_i) \cdot \Delta x = f(x_1) \cdot \Delta x + f(x_2) \cdot \Delta x + \dots + f(x_n) \cdot \Delta x$ is dan een benadering voor de exacte oppervlakte $\int_a^b f(x) dx$.

- a. Schrijf een functie `Delta_x(a, b, n)` die de breedte Δx van elk deelinterval berekent wanneer je $[a, b]$ verdeelt in n gelijke deelintervallen.
- b. Schrijf een functie `x_i(x1, x2, type)` die een waarde teruggeeft in het deelinterval $[x_1, x_2]$:
- als `type` gelijk is aan `'LINKS'`, geeft de functie de waarde van `x1` terug;
 - als `type` gelijk is aan `'MIDDEN'`, geeft de functie het midden van $[x_1, x_2]$ terug;
 - als `type` gelijk is aan `'RECHTS'`, geeft de functie de waarde van `x2` terug.
- c. Schrijf een functie `Riemannsom(f, a, b, n, type)` die de waarde van $\sum_{i=1}^n f(x_i) \cdot \Delta x$ teruggeeft, waarbij Δx en x_i berekend worden zoals hierboven beschreven.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

- 4** Je hebt bij wiskunde geleerd dat je een bepaalde integraal kan beschouwen als de som van een oneindig aantal termen. De precieze details van deze riemannsom zijn in deze opgave niet van belang. Belangrijker is te beseffen dat deze methode grafisch neerkomt op het optellen van de oppervlakte van een groot aantal rechthoekjes.

In feite benaderen we $f(x)$ in elk deelinterval door een functie van de nulde graad (een constante functie). Dit is een vrij grove benadering, die weliswaar nadert naar de correcte georiënteerde oppervlakte naarmate het aantal deelintervallletjes toeneemt.

We gaan in deze opgave een kleine verfijning doorvoeren aan de riemannsom die je kent. Net zoals bij een riemannsom verdeel je ook hier het interval $[a, b]$ in n gelijke deelintervallletjes met breedte Δx . In plaats van de georiënteerde oppervlakte te beschouwen als de som van rechthoekjes, berekenen we nu echter de som van de oppervlaktes van een groot aantal trapeziums. Zo benaderen we $f(x)$ in elk deelinterval door een functie van de eerste graad. Deze **trapeziumregel** geeft voor een zelfde aantal deelintervallletjes in het algemeen een betere benadering dan de riemannsom.

In de lagere school heb je (hopelijk) geleerd dat je de oppervlakte van een trapezium met grote basis B , kleine basis b en hoogte h kan berekenen als $\frac{B+b}{2} \cdot h$. Als we het i -de deelinterval definiëren als $[x_i, x_{i+1}]$, vind je de oppervlakte van de i -de deeltrapezium dus als $\frac{f(x_i) + f(x_{i+1})}{2} \cdot \Delta x$. De totale oppervlakte van alle deeltrapeziums wordt dan ook gegeven door $\sum_{i=1}^n \frac{f(x_i) + f(x_{i+1})}{2} \cdot \Delta x$.

- Schrijf een functie `Delta_x(a, b, n)` die de breedte Δx van elk deelinterval berekent wanneer je $[a, b]$ verdeelt in n gelijke deelintervallen.
- Schrijf een functie `x_i(a, i, Delta_x)` die de ondergrens van het i -de deelinterval $[x_i, x_{i+1}]$ teruggeeft. Denk ook na hoe je de bovengrens van het i -de deelinterval $[x_i, x_{i+1}]$ kan berekenen. Je mag daarvoor natuurlijk een functie schrijven, maar dat hoeft niet.
- Schrijf een functie `trapeziumregel(f, a, b, n)` die de waarde van $\sum_{i=1}^n \frac{f(x_i) + f(x_{i+1})}{2} \cdot \Delta x$ teruggeeft, waarbij Δx en x_i berekend worden zoals hierboven beschreven.

Test je code in Dodona. Let daarbij op dat je geen hoofdprogramma ingeeft.

15

Sorteeralgoritmen

15.1 Opdrachten